TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

Documento de analisis, requisitos y arquitectura

Vision, mapa de impacto, historias, requisitos, drivers, arquitectura, decisiones, datos, planificacion, pruebas y seguridad

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Contenido

Contenido....2
1. Vision y contexto....6
Declaracion de Vision....6
Desglose del posicionamiento....6
Objetivos del producto (derivados del Mapa de Impacto, ver *documento 2 (Mapa de impacto)*)....7
Alcance del MVP....7
Objetivo medible....7
Riesgos de producto....7
Stakeholders....8
2. Mapa de impacto....9
Why - Objetivo....9
Actores....9
Who / How / What....9
Historias de usuario por valor....11
3. Historias de usuario....14
Traduccion de mensajes....14
Gestion de conversaciones....14
Cuenta de usuario....15
Participacion del interlocutor....16
Reportes....16
Administracion de idiomas y regiones....16
4. Criterios de aceptacion....17
Traduccion de mensajes....17
Gestion de conversaciones....18
Cuenta de usuario....19
Participacion del interlocutor....20
Reportes....20
Administracion de idiomas y regiones....20
5. Requisitos funcionales y no funcionales....22
1. Actores....22
2. Requisitos funcionales....22
3. Requisitos no funcionales....24
4. Trazabilidad impacto -> requisito -> driver....25
5. Fuera del alcance del MVP....25
6. Drivers de arquitectura....26
1. Drivers....26
2. Como se responde y como se comprueba....26
7. Arquitectura....28
Como leer esta entrega....28
1. Estilo arquitectonico....29
2. Como la arquitectura responde a los drivers....31
3. Vista de contexto (C4 Nivel 1)....32

4. Vista de contenedores (C4 Nivel 2)....32
5. Vista de componentes (C4 Nivel 3)....33
6. Vista de datos....34
7. Vista dinamica: traducir un mensaje....35
8. Vista de despliegue....36
9. Patrones aplicados....36
10. Indice de decisiones (ADR)....37
11. Riesgos tecnicos y mitigaciones....38
12. Trazabilidad global....39
8. Stack tecnologico....40
Vista general....40
Por que cada tecnologia y que alternativas se evaluaron....40
Estructura del codigo (capas)....41
Como correrlo....42
9. Diagramas C4....43
Nivel 1 - Contexto del sistema....43
Nivel 2 - Contenedores....44
Tabla de elementos....44
Comunicaciones principales....45
10. Componentes de la API....46
Diagrama de componentes....46
Catalogo de componentes....47
Bounded Contexts....49
Trazabilidad: requerimiento -> componente....50
Limitaciones conocidas y evolucion....52
11. Decisiones de arquitectura (ADR)....53
ADR-001: App movil con Expo (React Native) y NativeWind....54
ADR-002: Backend monolitico en Node.js con Express, API REST sincrona....55
ADR-003: SQLite como base de datos inicial....55
ADR-004: Traduccion con Qwen local, Claude Sonnet de respaldo y motor de reglas....56
ADR-005: Voz: dictado en el cliente y expo-speech para la salida....57
ADR-006: Autenticacion con JWT de acceso corto y token de refresco rotativo....57
ADR-007: Interfaz basada en el template Luna....58
ADR-008: Ajustes al modelo de dominio al implementarlo....58
ADR-009: Identificadores del dominio en espanol (lenguaje ubicuo)....59
ADR-010: Proteccion de datos personales....59
ADR-011: API v1 segun la convencion REST y documentada con OpenAPI....60
ADR-012: SurrealDB como fuente de verdad, detras de un repositorio....60
ADR-013: Arquitectura hexagonal (puertos y adaptadores) dentro del monolito modular....63
ADR-014: Voz en Expo Go con grabacion en el celular, Whisper en el servidor y entonacion por texto y voz 64
ADR-015: Voz natural con emocion en el servidor (puerto Sintetizador)....64
ADR-016: VoiceStudio local como primer motor de voz con emocion....65
ADR-017: Piramide de pruebas en tres niveles, todos en el CI....66

ADR-018: Escaneo de seguridad con Trivy en el CI....66
ADR-019: ElevenLabs (nube) como motor de voz; se revierte VoiceStudio....67
ADR-020: Claude Sonnet en Google Vertex AI como motor de traduccion principal....68
12. Modelo de datos....70
Objetos de dominio....70
Usuario....70
Conversacion....72
Lenguaje....73
LenguajePorConversacion....74
MensajeOriginal....75
MensajeDestino....76
Canal....76
Entonacion....77
Region....78
Pais....78
LenguajeRegion (relacion)....79
Tablas de soporte (no son objetos de dominio)....79
Correspondencia con la base de datos....80
Nota metodologica sobre combinaciones unicas....80
13. Especificacion de vistas y UI....81
Customer Journey Map....81
Etapas del Customer Journey Map en la app....82
Diferencias con el diseno original (decididas al construir)....83
14. Planificacion de sprints....84
Supuestos....84
Historias por prioridad....84
Sprints....85
Por que el sprint 1 tiene pocas historias....86
Cronograma del trabajo de grado....86
Relacion con la prueba de concepto del 14/10/2026....86
Criterios de la planificacion....86
15. Plan de liberaciones....87
Que recibe el usuario en cada liberacion....87
Nota....87
16. Matriz de sensibilidad de datos....88
Envio a terceros....89
Retencion....89
Controles de infraestructura en produccion....89
17. Politica de tratamiento de datos personales....91
18. Piramide de pruebas....92
1. Resumen....92
2. Por que una piramide....93
3. Base: pruebas unitarias....93
4. Medio: pruebas de integracion....94

5. Punta: pruebas de punta a punta..... 94
6. Como correr todo en local..... 94
7. En el CI (.github/workflows/ci.yml)..... 95
8. Reglas del equipo..... 95
19. Seguridad con Trivy..... 96
1. Resultado actual..... 96
2. Dependencias corregidas en el frontend..... 96
3. Excepcion aceptada..... 97
4. Como funciona..... 97
5. Como correrlo en local..... 98
6. Que hacer cuando Trivy falla..... 98
7. Relacion con la arquitectura..... 98
8. Historial de escaneos..... 98
20. OWASP Top 10 para aplicaciones con LLM..... 100
1. Resumen..... 100
2. Detalle por riesgo..... 101
3. Brechas y plan para cerrarlas..... 105
4. Hallazgo adicional (fuera de la lista OWASP)..... 105
5. Como se relaciona con lo demas..... 106

1. Vision y contexto

Fase 1 - Definicion del contexto

Version: 1.0 (29/10/2026) **Fuente:** `anexos/Vision.xlsx` (plantilla de posicionamiento de Geoffrey Moore).

Declaracion de Vision

> *Para cualquier persona que necesita expresarse verbal o textualmente en un idioma que no conoce o domina, incluyendo no solo su mensaje sino tambien cualquier modismo o jerga cultural utilizada sin perder el significado o contexto del mismo,* ***TalkLate*** *es una aplicacion movil que esta disenada para ofrecer una experiencia de traduccion que va mas alla de la conversion literal de palabras, ya que considera el contexto, las emociones, los matices culturales y las intenciones detras del lenguaje, permitiendo una comunicacion mas natural y significativa. A diferencia de Meta Seamless, DeepL Translator, TimeKettle, Lilt y Kudo AI, nuestro producto se distingue por su enfoque en la* ***conexion profunda****, permitiendo a los usuarios expresarse autenticamente en otro idioma, con foco inicial en ciertas regiones y acentos especificos del ingles y espanol, y con una vision evolutiva para soportar con el paso del tiempo otros lenguajes con sus respectivos modismos y caracteristicas, sin perder la espontaneidad ni la riqueza de su comunicacion original.*

Desglose del posicionamiento

Elemento	Definicion
Cliente/publico objetivo	Cualquier persona que necesita expresarse verbal o textualmente en un idioma que no conoce o domina.
Necesidad / oportunidad	Comunicar no solo el mensaje literal, sino tambien modismos o jerga cultural, sin perder significado ni contexto.
Nombre del producto	TalkLate
Tipo de producto	Aplicacion movil
Beneficio principal	Traduccion que va mas alla de la conversion literal palabra por palabra: considera contexto, emociones, matices culturales e intenciones.
Alternativas existentes	Meta Seamless, DeepL Translator, TimeKettle, Lilt, Kudo AI (a evaluar tambien: Semihuman AI).
Diferenciacion principal	"Conexion profunda": permitir a los usuarios expresarse autenticamente en otro idioma. Foco inicial en regiones y acentos especificos del ingles y espanol, con vision evolutiva hacia mas idiomas sin perder espontaneidad ni riqueza comunicativa.

¿Que es una "conexion profunda"?

Es la capacidad de comunicarse con una persona o grupo que habla un idioma ajeno al propio, sin limitarse a una comunicacion formal que impida expresarse autenticamente. TalkLate no solo traduce palabras:

interpreta jerga local, acentos y variaciones linguisticas regionales, brindando una comunicacion mas natural y humana — culturalmente ajustada en cada interaccion multilingue.

Objetivos del producto (derivados del Mapa de Impacto, ver *documento 2 (Mapa de impacto)*)

1. Traduccion contextual en tiempo real, con claridad del contexto y del lenguaje geograficamente.
2. Traduccion que preserva tono, estilo, intencion y modismos del hablante original — evitando traducciones planas o literales.
3. Historial y gestion de conversaciones traducidas, reutilizable como recurso de aprendizaje.
4. Un modelo de IA que evoluciona con el tiempo: nuevos idiomas, acentos y capacidades, sin sacrificar calidad ni naturalidad.

Alcance del MVP

- **Tiempo real** significa traduccion **por turno**: cada mensaje se traduce y se entrega en pocos segundos (3 segundos o menos por mensaje de texto y 5 segundos o menos por mensaje de voz, DRV-01). No es traduccion simultanea mientras la persona habla.
- **Acentos y variaciones regionales:** en el MVP la region de origen se reconoce por el **vocabulario** (por ejemplo "parce", "wey", "che") y el motor adapta los modismos a la region de destino. Reconocer el acento por el **sonido** de la voz es evolucion futura.
- **Emociones y tono:** se estima el tono del mensaje (neutral, amigable, alegre, preocupado, enojado, formal) a partir de las palabras y, en los mensajes hablados, de la voz (volumen, variacion y velocidad). La traduccion conserva ese tono y la voz de salida lo replica.
- **Voz:** la app graba el mensaje y el servidor lo transcribe con Whisper; la traduccion se escucha con una voz neuronal con emocion (ElevenLabs, con Azure Speech u OpenAI como respaldo).
- **Idiomas:** espanol e ingles activos al inicio; otros idiomas (por ejemplo frances y portugues) se agregan desde la administracion, sin tocar codigo (vision evolutiva).
- **Motor de IA:** la traduccion la hace **Claude Sonnet**, servido por Google Vertex AI y llamado desde el **servidor** de TalkLate (no desde el celular); el motor de reglas es un respaldo parcial y Qwen local (Ollama) queda como adaptador alternativo (ADR-020).

Objetivo medible

Ver el "Why" SMART en *documento 2 (Mapa de impacto)*: al cierre de la liberacion 3 (08/01/2027), 10 personas de prueba sostienen conversaciones de 10 mensajes o mas con alguien que habla otro idioma, con menos del 20 % de traducciones reportadas como malas y 3 segundos o menos por traduccion.

Riesgos de producto

- La necesidad no es solo comunicarse: es comunicarse de forma autentica y no solo formal. El producto se mide por esa diferencia (tono, emocion y modismos), no por la traduccion literal.

- El mercado de traduccion con IA cambia rapido (Meta Seamless, DeepL, TimeKettle, Semihuman AI); la diferenciacion se sostiene en la adaptacion cultural por region.
- El foco inicial en acentos/regiones especificas de ingles y espanol implica que el MVP no cubre todos los idiomas ni todas las variantes regionales desde el dia uno; la expansion es explicitamente evolutiva, no un compromiso del lanzamiento inicial.

Stakeholders

- **Equipo de desarrollo:** Julian David Diaz Medina, Diego Alejandro Franco Atehortua.
- **Director del trabajo de grado:** Wider Farid Sanchez Garzon.

2. Mapa de impacto

Fase 2 - Especificacion de requerimientos

Version: 1.0 (29/10/2026). Hoja de calculo de apoyo: `anexos/Mapa_de_Impacto.xlsx`. **Metodologia:** Impact Mapping (Gojko Adzic): Why (objetivo) -> Who (actores, en terminos positivos) -> How (impacto SMART: el beneficio que obtiene el actor) -> What (entregable = epica, que agrupa modulos del sistema y se abre en historias de usuario).

Why - Objetivo

Objetivo (SMART): al cierre de la liberacion 3 (08/01/2027), al menos 10 personas de prueba sostienen cada una una conversacion de 10 mensajes o mas con alguien que habla otro idioma (espanol-ingles), con menos del 20 % de traducciones reportadas como malas y un tiempo por traduccion de 3 segundos o menos (p95). Vision: ver el documento 1 (Vision y contexto).

Actores

- **Cliente Traductor:** dueno del dispositivo; usa la app para comunicarse en un idioma que no domina.
- **Interlocutor:** la otra persona de la conversacion; no instala la app ni crea cuenta, responde con el celular del Cliente Traductor.
- **Administrador:** mantiene el catalogo de idiomas y regiones y revisa la calidad de las traducciones.

> *La Inteligencia Artificial ya no se modela como actor: no es una persona que obtenga un beneficio, es parte del sistema (el motor de traduccion, ADR-004).*

Who / How / What

Ordenado de mayor a menor valor, el mismo criterio con que se llenan los sprints. "Desde P" es la prioridad de la historia mas valiosa de ese impacto (ver *documento 14 (Planificacion de sprints)*).

#	Valor	Actor	Impacto (How)	Problema si no se logra	Beneficio	Entregable (epica)	Indicador SMART
1	Alta (desde P1)	Cliente Traductor	Interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos.	Si no se traduce un mensaje en tiempo real no se puede sostener una conversacion con otra persona cuando no se conoce el idioma destino	Tener una conversacion fluida con una persona que habla otro idioma	Traduccion de mensajes	Liberacion 1 (13/11/2026): el 80 % de los modismos del glosario de prueba se traducen con su equivalente cultural y en 3 segundos o menos (p95). Se mide con npm run medir.
2	Alta (desde P3)	Cliente Traductor	Sostener una conversacion natural, hablando y escuchando como lo haria en su propio idioma.	Si la traduccion solo sale en texto o con una voz que suena robotica, la otra persona no entiende bien lo que el usuario quiso decir y todo se siente forzado.	Hablar y escuchar de forma natural, como en el propio idioma.	Traduccion de mensajes	Liberacion 1: el 100 % de los mensajes enviados por voz se escuchan traducidos; 7 de cada 10 personas de prueba califican la voz como comprensible.

#	Valor	Actor	Impacto (How)	Problema si no se logra	Beneficio	Entregable (epica)	Indicador SMART
3	Alta (desde P6)	Cliente Traductor	Empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio.	Si el usuario tiene que configurar todo y decir su idioma antes de hablar, pierde tiempo y la conversacion no arranca de inmediato.	Empezar a hablar de una vez.	Gestion de conversaciones / Administracion de idiomas y regiones	Liberaciones 1 y 2: en la prueba con usuarios, una persona nueva crea la conversacion y envia su primer mensaje en menos de 1 minuto; el idioma de origen se reconoce bien en el 90 % de los primeros mensajes.
4	Alta (desde P8)	Cliente Traductor	Mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada.	Si la conversacion no guarda la configuracion ni el hilo, cada mensaje queda suelto y se pierde el contexto de lo que se ha dicho.	Conversar de corrido, con todo el hilo en un solo lugar.	Gestion de conversaciones	Liberacion 2 (11/12/2026): en la prueba con usuarios hay conversaciones de 10 mensajes o mas sin reconfigurar idiomas.
5	Alta (desde P11)	Interlocutor	Participar en la conversacion en su propio idioma y en igualdad de condiciones, sin instalar la aplicacion ni crear una cuenta.	Si la otra persona tiene que instalar la app o crear una cuenta para contestar, la conversacion no arranca o ella queda en desventaja.	Que la otra persona hable en su idioma con el celular del usuario, sin cuenta.	Participacion del interlocutor	Liberacion 2: en la prueba con usuarios, la otra persona responde en su idioma con el celular del usuario, sin cuenta, en el 100 % de los casos.
6	Media (desde P14)	Cliente Traductor	Aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas.	Si las conversaciones no se guardan, al cerrar la app se pierde lo que se dijo y el usuario no puede repasar las frases ni aprender de ellas.	Volver a conversaciones pasadas para repasar y ganar confianza en el otro idioma.	Gestion de conversaciones	Liberacion 2: el 100 % de las conversaciones quedan en el historial y se pueden reabrir; en la encuesta final, 7 de cada 10 personas dicen que repasarlas les ayudo.
7	Media (desde P16)	Cliente Traductor	Tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver.	Si no hay una cuenta, las conversaciones no quedan ligadas a nadie: cualquiera con el celular las podria ver y, si el usuario cambia de celular, las pierde.	Tener las conversaciones guardadas y que sean solo del usuario.	Cuenta de usuario	Liberacion 3 (08/01/2027): 0 accesos de una persona a conversaciones de otra en las pruebas de seguridad; la persona puede eliminar su cuenta con todos sus datos.
8	Media (desde P18)	Cliente Traductor	Contribuir a mejorar la calidad del servicio senalando las traducciones que no fueron correctas o naturales.	Si el usuario no puede avisar cuando una traduccion sale mal, el error se queda y se repite, y termina desconfiando de la app.	Ayudar a que las traducciones mejoren con el uso.	Reportes	Liberacion 3: cualquier traduccion se reporta en 2 toques o menos y el reporte llega al administrador.
9	Baja (desde P23)	Administrador	Decidir que mejoras priorizar en el servicio a partir de los reportes de mala traduccion que envian los clientes.	Si nadie revisa los reportes de mala traduccion, los errores no se corrigen y no se sabe que mejorar primero.	Mejorar primero lo que mas afecta a los usuarios.	Reportes	Liberacion 3: el 100 % de los reportes abiertos se revisan (revisado o descartado) en menos de una semana.

#	Valor	Actor	Impacto (How)	Problema si no se logra	Beneficio	Entregable (epica)	Indicador SMART
10	Baja (desde P24)	Administrador	Ampliar la oferta de idiomas y regiones con calidad confiable, activando solo los que ya traducen bien.	Si no se pueden agregar ni ajustar idiomas y regiones, la app se queda con lo que trajo al inicio y no le sirve a mas gente.	Poder crecer a mas idiomas y regiones.	Administracion de idiomas y regiones	Liberacion 3: un idioma o region se agrega y activa sin cambiar codigo; solo se activan idiomas que pasan el glosario de prueba con 80 % o mas.

> *Un impacto puede abrirse en varios entregables y un entregable puede aparecer en varios impactos. Cada historia de usuario dice a que impacto aporta en su "Para".*

Historias de usuario por valor

Mismo orden que *documento 14 (Planificacion de sprints)* (prioridad 1 = mas valor).

1. **HU0003** - Como Cliente Traductor, necesito poder traducir mi mensaje al idioma destino en tiempo real, para interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos. *(Epica: Traduccion de mensajes · Talla XXL · Sprint 1)*
2. **HU0002** - Como Cliente Traductor, necesito poder ingresar mi mensaje escribiendo, para interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos. *(Epica: Traduccion de mensajes · Talla S · Sprint 1)*
3. **HU0001** - Como Cliente Traductor, necesito poder ingresar mi mensaje hablando, para interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos. *(Epica: Traduccion de mensajes · Talla L · Sprint 1)*
4. **HU0004** - Como Cliente Traductor, necesito poder escuchar el mensaje traducido, para sostener una conversacion natural, hablando y escuchando como lo haria en su propio idioma. *(Epica: Traduccion de mensajes · Talla M · Sprint 1)*
5. **HU0005** - Como Cliente Traductor, necesito poder recibir la traduccion por el mismo canal que use, para sostener una conversacion natural, hablando y escuchando como lo haria en su propio idioma. *(Epica: Traduccion de mensajes · Talla S · Sprint 1)*
6. **HU0006** - Como Cliente Traductor, necesito poder seleccionar el idioma y la region destino, para empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio. *(Epica: Gestion de conversaciones · Talla M · Sprint 1)*
7. **HU0007** - Como Cliente Traductor, necesito poder reconocer el idioma origen, para empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio. *(Epica: Gestion de conversaciones · Talla L · Sprint 2)*
8. **HU0008** - Como Cliente Traductor, necesito poder crear una conversacion, para mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada. *(Epica: Gestion de conversaciones · Talla S · Sprint 2)*

9. **HU0009** - Como Cliente Traductor, necesito poder configurar los parametros una sola vez, para mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada. *(Epica: Gestion de conversaciones · Talla S · Sprint 2)*
10. **HU0010** - Como Cliente Traductor, necesito poder mantener el hilo continuo, para mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada. *(Epica: Gestion de conversaciones · Talla M · Sprint 2)*
11. **HU0019** - Como Interlocutor, necesito poder recibir el mensaje traducido, para participar en la conversacion en su propio idioma y en igualdad de condiciones, sin instalar la aplicacion ni crear una cuenta. *(Epica: Participacion del interlocutor · Talla S · Sprint 2)*
12. **HU0020** - Como Interlocutor, necesito poder responder en mi idioma, para participar en la conversacion en su propio idioma y en igualdad de condiciones, sin instalar la aplicacion ni crear una cuenta. *(Epica: Participacion del interlocutor · Talla M · Sprint 2)*
13. **HU0023** - Como Cliente Traductor, necesito poder consultar los idiomas disponibles, para empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio. *(Epica: Administracion de idiomas y regiones · Talla XS · Sprint 2)*
14. **HU0011** - Como Cliente Traductor, necesito poder consultar mi historial, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. *(Epica: Gestion de conversaciones · Talla M · Sprint 2)*
15. **HU0012** - Como Cliente Traductor, necesito poder abrir una conversacion anterior, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. *(Epica: Gestion de conversaciones · Talla S · Sprint 2)*
16. **HU0015** - Como Cliente Traductor, necesito poder registrarme en la aplicacion, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. *(Epica: Cuenta de usuario · Talla L · Sprint 3)*
17. **HU0016** - Como Cliente Traductor, necesito poder iniciar sesion, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. *(Epica: Cuenta de usuario · Talla M · Sprint 3)*
18. **HU0021** - Como Cliente Traductor, necesito poder reportar una mala traduccion, para contribuir a mejorar la calidad del servicio senalando las traducciones que no fueron correctas o naturales. *(Epica: Reportes · Talla M · Sprint 3)*
19. **HU0013** - Como Cliente Traductor, necesito poder renombrar una conversacion, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. *(Epica: Gestion de conversaciones · Talla XS · Sprint 2)*
20. **HU0014** - Como Cliente Traductor, necesito poder eliminar una conversacion, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. *(Epica: Gestion de conversaciones · Talla XS · Sprint 2)*
21. **HU0017** - Como Cliente Traductor, necesito poder recuperar mi contrasena, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. *(Epica: Cuenta de usuario · Talla L · Sprint 3)*

22. **HU0018** - Como Cliente Traductor, necesito poder cerrar sesion, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. *(Epica: Cuenta de usuario · Talla XS · Sprint 2)*
23. **HU0022** - Como Administrador, necesito poder revisar los reportes de mala traduccion, para decidir que mejoras priorizar en el servicio a partir de los reportes de mala traduccion que envian los clientes. *(Epica: Reportes · Talla M · Sprint 3)*
24. **HU0024** - Como Administrador, necesito poder agregar un idioma o una region, para ampliar la oferta de idiomas y regiones con calidad confiable, activando solo los que ya traducen bien. *(Epica: Administracion de idiomas y regiones · Talla M · Sprint 3)*
25. **HU0025** - Como Administrador, necesito poder activar o desactivar un idioma o una region, para ampliar la oferta de idiomas y regiones con calidad confiable, activando solo los que ya traducen bien. *(Epica: Administracion de idiomas y regiones · Talla XS · Sprint 3)*
26. **HU0026** - Como Cliente Traductor, necesito poder eliminar mi cuenta, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. *(Epica: Cuenta de usuario · Talla XS · Sprint 3)*

3. Historias de usuario

Salen de la hoja Entregables de `anexos/Mapa_de_Impacto.xlsx`. Cada entregable es una epica (modulo del sistema) que se abre en varias historias. Formato: Como / Necesito poder / Para. Prioridad por valor para el negocio; talla y horas por historia; sprint de un mes (120 horas).

Traduccion de mensajes

HU0001 - Como Cliente Traductor, necesito poder ingresar mi mensaje hablando, para interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos. Prioridad 3 · Talla L (24 h) · Sprint 1 · Valor: Es la forma natural de usar el producto: hablar y ser entendido sin escribir.

HU0002 - Como Cliente Traductor, necesito poder ingresar mi mensaje escribiendo, para interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos. Prioridad 2 · Talla S (8 h) · Sprint 1 · Valor: Es la entrada mas simple y la que permite usar la traduccion desde el primer dia.

HU0003 - Como Cliente Traductor, necesito poder traducir mi mensaje al idioma destino en tiempo real, para interactuar facil y rapidamente con personas que hablan otro idioma, traduciendo cada mensaje en tiempo real y conservando el contexto cultural, el tono y los modismos. Prioridad 1 · Talla XXL (48 h) · Sprint 1 · Valor: Es el corazon del producto: sin esto TalkLate no entrega su valor diferencial.

HU0004 - Como Cliente Traductor, necesito poder escuchar el mensaje traducido, para sostener una conversacion natural, hablando y escuchando como lo haria en su propio idioma. Prioridad 4 · Talla M (16 h) · Sprint 1 · Valor: Permite que la otra persona reciba el mensaje de forma comoda y natural.

HU0005 - Como Cliente Traductor, necesito poder recibir la traduccion por el mismo canal que use, para sostener una conversacion natural, hablando y escuchando como lo haria en su propio idioma. Prioridad 5 · Talla S (8 h) · Sprint 1 · Valor: Mantiene la experiencia coherente: si hablo, escucho; si escribo, leo.

Gestion de conversaciones

HU0006 - Como Cliente Traductor, necesito poder seleccionar el idioma y la region destino, para empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio. Prioridad 6 · Talla M (16 h) · Sprint 1 · Valor: Define a que persona y lugar se adapta el mensaje: de ahi sale el matiz cultural.

HU0007 - Como Cliente Traductor, necesito poder reconocer el idioma origen, para empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio. Prioridad 7 · Talla L (24 h) · Sprint 2 · Valor: Evita configurar a mano el idioma cada vez y acelera la conversacion.

HU0008 - Como Cliente Traductor, necesito poder crear una conversacion, para mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada. Prioridad 8 · Talla S (8 h) · Sprint 2 · Valor: Agrupa los mensajes de una misma charla para no perder el contexto.

HU0009 - Como Cliente Traductor, necesito poder configurar los parametros una sola vez, para mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada. Prioridad 9 · Talla S (8 h) · Sprint 2 · Valor: Ahorra repetir la configuracion en cada mensaje.

HU0010 - Como Cliente Traductor, necesito poder mantener el hilo continuo, para mantener el contexto de toda la charla, de modo que cada respuesta se traduzca dentro del mismo hilo sin reconfigurar nada. Prioridad 10 · Talla M (16 h) · Sprint 2 · Valor: Hace posible un dialogo fluido, que es lo que busca el usuario.

HU0011 - Como Cliente Traductor, necesito poder consultar mi historial, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. Prioridad 14 · Talla M (16 h) · Sprint 2 · Valor: Permite repasar y aprender de conversaciones pasadas.

HU0012 - Como Cliente Traductor, necesito poder abrir una conversacion anterior, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. Prioridad 15 · Talla S (8 h) · Sprint 2 · Valor: Permite retomar una conversacion donde se dejo.

HU0013 - Como Cliente Traductor, necesito poder renombrar una conversacion, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. Prioridad 19 · Talla XS (4 h) · Sprint 2 · Valor: Ayuda a ubicar conversaciones, pero no cambia el valor central.

HU0014 - Como Cliente Traductor, necesito poder eliminar una conversacion, para aprender y ganar confianza en el otro idioma apoyandose en conversaciones pasadas. Prioridad 20 · Talla XS (4 h) · Sprint 2 · Valor: Da control sobre los datos propios; es de apoyo.

Cuenta de usuario

HU0015 - Como Cliente Traductor, necesito poder registrarme en la aplicacion, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. Prioridad 16 · Talla L (24 h) · Sprint 3 · Valor: Guarda las conversaciones del usuario y protege sus datos; el valor central no depende de esto.

HU0016 - Como Cliente Traductor, necesito poder iniciar sesion, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. Prioridad 17 · Talla M (16 h) · Sprint 3 · Valor: Da acceso seguro a lo guardado.

HU0017 - Como Cliente Traductor, necesito poder recuperar mi contrasena, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. Prioridad 21 · Talla L (24 h) · Sprint 3 · Valor: Evita perder la cuenta; es de apoyo.

HU0018 - Como Cliente Traductor, necesito poder cerrar sesion, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. Prioridad 22 · Talla XS (4 h) · Sprint 2 · Valor: Protege la cuenta en un dispositivo compartido; es de apoyo. Por valor cabe en el sprint 2, aunque iniciar sesion llegue en el sprint 3 (no se planifica por dependencia).

HU0026 - Como Cliente Traductor, necesito poder eliminar mi cuenta, para tener las conversaciones y la configuracion guardadas en una cuenta segura que solo el usuario puede ver. Prioridad 26 · Talla XS (4 h) · Sprint 3 · Valor: Cumple el derecho al olvido (Ley 1581 de 2012): la persona controla sus datos.

Participacion del interlocutor

HU0019 - Como Interlocutor, necesito poder recibir el mensaje traducido, para participar en la conversacion en su propio idioma y en igualdad de condiciones, sin instalar la aplicacion ni crear una cuenta. Prioridad 11 · Talla S (8 h) · Sprint 2 · Valor: Hace que la otra persona participe sin instalar ni operar la app.

HU0020 - Como Interlocutor, necesito poder responder en mi idioma, para participar en la conversacion en su propio idioma y en igualdad de condiciones, sin instalar la aplicacion ni crear una cuenta. Prioridad 12 · Talla M (16 h) · Sprint 2 · Valor: Cierra el dialogo en ambos sentidos.

Reportes

HU0021 - Como Cliente Traductor, necesito poder reportar una mala traduccion, para contribuir a mejorar la calidad del servicio senalando las traducciones que no fueron correctas o naturales. Prioridad 18 · Talla M (16 h) · Sprint 3 · Valor: Alimenta la mejora de calidad con casos reales.

HU0022 - Como Administrador, necesito poder revisar los reportes de mala traduccion, para decidir que mejoras priorizar en el servicio a partir de los reportes de mala traduccion que envian los clientes. Prioridad 23 · Talla M (16 h) · Sprint 3 · Valor: Permite priorizar que mejorar primero.

Administracion de idiomas y regiones

HU0023 - Como Cliente Traductor, necesito poder consultar los idiomas disponibles, para empezar a conversar en menos de un minuto, configurando idioma y region destino una sola vez y sin tener que indicar el idioma propio. Prioridad 13 · Talla XS (4 h) · Sprint 2 · Valor: Muestra con que idiomas se puede hablar antes de empezar.

HU0024 - Como Administrador, necesito poder agregar un idioma o una region, para ampliar la oferta de idiomas y regiones con calidad confiable, activando solo los que ya traducen bien. Prioridad 24 · Talla M (16 h) · Sprint 3 · Valor: Hace crecer el servicio a mas personas.

HU0025 - Como Administrador, necesito poder activar o desactivar un idioma o una region, para ampliar la oferta de idiomas y regiones con calidad confiable, activando solo los que ya traducen bien. Prioridad 25 · Talla XS (4 h) · Sprint 3 · Valor: Controla que idiomas se ofrecen con calidad suficiente.

4. Criterios de aceptacion

Cada criterio se comprueba en la app o en las pruebas automaticas de `backend/test`. Tambien estan en la hoja Entregables de `anexos/Mapa_de_Impacto.xlsx`.

Traduccion de mensajes

HU0001 Ingresar mi mensaje hablando

1. Dado que abro una conversacion, entonces veo la burbuja de voz; cuando la toco y hablo, la burbuja muestra que escucha y, al terminar de hablar (1.5 s de silencio o un segundo toque), el mensaje se transcribe y se envia por el canal Voz.
2. Se detecta la entonacion del mensaje por las palabras y por la voz (volumen, variacion y velocidad), y queda registrada en el mensaje.
3. Cuando el microfono no esta permitido, la grabacion es muy corta o no se entiende el audio, entonces veo un aviso claro y puedo escribir el mensaje desde el modo chat.

HU0002 Ingresar mi mensaje escribiendo

1. Dado que estoy en una conversacion, cuando escribo un mensaje y lo envio, entonces aparecen mi mensaje original y su traduccion.
2. No se puede enviar un mensaje vacio ni de mas de 5000 caracteres.

HU0003 Traducir mi mensaje al idioma destino en tiempo real

1. Dado un mensaje con un modismo (ej. 'Hay que ponerse las pilas con el trabajo'), cuando lo envio, entonces la traduccion usa el equivalente cultural en la region destino ('We need to get cracking on the work') y no una version literal.
2. Cuando se adapto un modismo, entonces puedo ver una nota cultural que lo explica; si quedaron palabras sin traducir, la traduccion se marca como aproximada.
3. Cuando el servicio no puede traducir, entonces veo un mensaje de error en vez de un resultado vacio.

HU0004 Escuchar el mensaje traducido

1. Dado un mensaje traducido, cuando toco 'Escuchar', entonces se reproduce con una voz natural del idioma y pais destino.
2. La voz replica la emocion detectada (enojo, alegria, preocupacion, amabilidad o formalidad) y la velocidad con que hablo la persona.
3. Si el motor de voz natural no esta disponible, la traduccion se escucha igual con la voz del celular.

HU0005 Recibir la traduccion por el mismo canal que use

1. Dado que envie el mensaje por voz, entonces la traduccion se reproduce sola y tambien queda escrita.
2. Dado que envie el mensaje por texto, entonces la traduccion solo se muestra escrita.

Gestion de conversaciones

HU0006 Seleccionar el idioma y la region destino

1. Dado que creo una conversacion, cuando elijo un idioma de destino, entonces solo veo las regiones validas para ese idioma.
2. No puedo empezar la conversacion si falta el idioma o la region destino.

HU0007 Reconocer el idioma origen

1. Dado que no elegi mi idioma, cuando envio mi primer mensaje, entonces el sistema lo reconoce y lo muestra en el encabezado de la conversacion.
2. Una vez reconocido, el idioma de origen no cambia en esa conversacion.
3. Cuando el mensaje es muy corto para reconocer el idioma, entonces se me pide escribir un poco mas.

HU0008 Crear una conversacion

1. Cuando creo una conversacion sin nombre, entonces se llama 'Nueva Conversacion'.
2. La conversacion creada aparece en el menu lateral y en el historial.

HU0009 Configurar los parametros una sola vez

1. Dado que ya elegi idioma y region destino al crear la conversacion, entonces los mensajes siguientes no me piden configurar nada.
2. El destino de la conversacion se mantiene mientras la uso.

HU0010 Mantener el hilo continuo

1. Cuando envio varios mensajes seguidos, entonces cada uno se traduce y queda en el mismo hilo, en orden.
2. Cuando reabro la conversacion mas tarde, entonces veo todos los mensajes anteriores.

HU0011 Consultar mi historial

1. Cuando abro el historial, entonces veo mis conversaciones con su nombre, idiomas, numero de mensajes y ultimo mensaje, las mas recientes primero.
2. Puedo buscar una conversacion por su nombre.
3. Solo veo mis propias conversaciones.

HU0012 Abrir una conversacion anterior

1. Cuando toco una conversacion del historial, entonces veo sus mensajes originales y traducidos en orden.
2. No puedo abrir la conversacion de otra persona.

HU0013 Renombrar una conversacion

1. Cuando cambio el nombre, entonces debe tener entre 3 y 80 caracteres, sin caracteres especiales ni palabras ofensivas; si no, veo el motivo.
2. El nombre nuevo se ve en el encabezado, el menu y el historial.

HU0014 Eliminar una conversacion

1. Cuando elijo eliminar, entonces la app me pide confirmar y me avisa que no se puede deshacer.
2. Al confirmar, la conversacion se borra con sus mensajes y reportes.

Cuenta de usuario

HU0015 Registrarme en la aplicacion

1. Cuando me registro con datos validos, entonces la cuenta se crea y entro a la app.
2. La contrasena debe tener 8 o mas caracteres, una mayuscula, un numero y un simbolo; el correo debe tener formato valido.
3. Debo tener 13 anos o mas; si no, veo 'debes tener al menos 13 anos'.
4. Si el correo ya esta registrado, veo un aviso para iniciar sesion.
5. Debo aceptar la politica de tratamiento de datos (Ley 1581 de 2012); si no, no se crea la cuenta.

HU0016 Iniciar sesion

1. Con correo y contrasena correctos entro a la app; con datos incorrectos veo 'Usuario o contrasena incorrectos'.
2. Despues de varios intentos fallidos seguidos, el acceso se bloquea unos minutos.
3. Al cerrar y abrir la app sigo con la sesion iniciada.

HU0017 Recuperar mi contrasena

1. Cuando pido recuperar la contrasena con un correo registrado, entonces recibo un codigo de 6 digitos que vence en 15 minutos.
2. Con un codigo correcto y una contrasena nueva que cumpla las reglas, la contrasena cambia; el codigo solo sirve una vez.
3. Con un codigo incorrecto o vencido veo que debo pedir uno nuevo.
4. Si el correo no esta registrado, veo que no hay una cuenta asociada.

HU0018 Cerrar sesion

1. Cuando cierro sesion, entonces vuelvo a la pantalla de bienvenida.
2. El acceso anterior deja de funcionar aunque alguien lo conserve.

HU0026 Eliminar mi cuenta

1. Dado que estoy en mi perfil, cuando elijo eliminar mi cuenta, entonces la app me pide mi contrasena y me avisa que no se puede deshacer.

2. Al confirmar, se borran mi cuenta, mis conversaciones, mis mensajes y mis reportes, y vuelvo a la pantalla de bienvenida.
3. Con una contrasena incorrecta la cuenta no se elimina.

Participacion del interlocutor

HU0019 Recibir el mensaje traducido

1. Dado que la otra persona responde (se elige 'Otra persona' y se le pasa el celular), entonces su mensaje se traduce a mi idioma y se muestra a la izquierda, junto a su original.
2. La otra persona no necesita instalar la app ni crear una cuenta.

HU0020 Responder en mi idioma

1. Cuando la otra persona escribe en su propio idioma, entonces su respuesta se traduce al idioma de la conversacion.
2. Para responder ya debe haberse reconocido mi idioma con mi primer mensaje.

Reportes

HU0021 Reportar una mala traduccion

1. Cuando reporto una traduccion con un motivo de 3 a 500 caracteres, entonces el reporte se guarda y el mensaje queda marcado como 'Reportado'.
2. Solo puedo reportar una vez la misma traduccion y solo las de mis conversaciones.

HU0022 Revisar los reportes de mala traduccion

1. Dado que soy administrador, cuando abro los reportes, entonces veo el original, la traduccion, el motivo y la sugerencia, y puedo filtrar por abiertos, revisados o descartados.
2. Puedo marcar un reporte como revisado o descartado.
3. Un cliente que no es administrador no puede ver ni cambiar reportes.

Administracion de idiomas y regiones

HU0023 Consultar los idiomas disponibles

1. Cuando abro la configuracion de una conversacion, entonces veo solo los idiomas activos con sus regiones.
2. Los idiomas desactivados no aparecen en la lista.

HU0024 Agregar un idioma o una region

1. Dado que soy administrador, cuando agrego un idioma, entonces necesita un nombre unico y un codigo de 2 a 5 letras, y queda desactivado hasta que lo active.
2. Una region no puede repetirse dentro del mismo pais; si el pais no existe, debo indicar su codigo de 2 letras.

3. Un cliente que no es administrador no puede agregar idiomas ni regiones.

HU0025 Activar o desactivar un idioma o una region

1. Cuando activo un idioma, entonces aparece en el catalogo para crear conversaciones; cuando lo desactivo, deja de aparecer.
2. Las conversaciones ya creadas no se ven afectadas.

5. Requisitos funcionales y no funcionales

Fase 2 - Especificacion de requerimientos

Version: 1.0 (29/10/2026). Cada requisito funcional sale de una historia de usuario (*documento 3 (Historias de usuario)*) y cita la regla de negocio del modelo de datos (*documento 12 (Modelo de datos)*) que aplica. Los criterios de aceptacion estan en *documento 4 (Criterios de aceptacion)*.

Resumen: 26 requisitos funcionales, 11 requisitos no funcionales y 8 drivers de arquitectura (*documento 6 (Drivers de arquitectura)*).

1. Actores

Actor	Descripcion	Necesita cuenta
Cliente Traductor	Dueno del celular; usa la app para hablar con alguien en un idioma que no domina.	Si
Interlocutor	La otra persona de la conversacion; responde en su idioma con el celular del usuario.	No
Administrador	Mantiene el catalogo de idiomas y regiones y revisa los reportes de mala traduccion.	Si (rol administrador)

2. Requisitos funcionales

Ordenados por prioridad (1 = mas valor), el mismo orden de los sprints.

Prio	ID	Requisito	Historia	Epica	Sprint	Regla o fuente
1	REQ-TRAD-003	El sistema debe traducir cada mensaje al idioma y region destino conservando tono, intencion y modismos (equivalente cultural, no literal), con nota cultural cuando adapte un modismo y marca de aproximada si quedan palabras sin traducir.	HU0003	Traduccion de mensajes	1	Conversacion: Traducir Mensaje; Region: aplicar matices
2	REQ-TRAD-002	El sistema debe permitir ingresar un mensaje escrito de 1 a 5000 caracteres, no vacio.	HU0002	Traduccion de mensajes	1	MensajeOriginal
3	REQ-TRAD-001	El sistema debe permitir ingresar un mensaje hablando: grabarlo en el celular, transcribirlo en el servidor (Whisper) y detectar la entonacion por las palabras y por la voz (volumen, variacion y velocidad). El mensaje se marca con el canal Voz.	HU0001	Traduccion de mensajes	1	Canal
4	REQ-TRAD-004	El sistema debe reproducir la traduccion con una voz natural del idioma y pais destino que replique la emocion detectada (enojo, alegria, preocupacion, amabilidad, formalidad) y la velocidad del hablante.	HU0004	Traduccion de mensajes	1	Entonacion
5	REQ-TRAD-005	El sistema debe entregar la traduccion por el mismo canal del mensaje original: audio (y texto) si fue por voz, texto si fue escrito.	HU0005	Traduccion de mensajes	1	Canal: Entregar mensaje traducido
6	REQ-CONV-001	El sistema debe exigir idioma y region destino, y la region debe ser valida para ese idioma.	HU0006	Gestion de conversaciones	1	Conversacion-RN-1

Prio	ID	Requisito	Historia	Epica	Sprint	Regla o fuente
7	REQ-CONV-002	El sistema debe reconocer el idioma de origen con el primer mensaje si no se eligio, y no cambiarlo en esa conversacion.	HU0007	Gestion de conversaciones	2	Conversacion-RN-2
8	REQ-CONV-003	El sistema debe crear conversaciones con nombre por defecto "Nueva Conversacion" y validar que el par de idiomas este soportado.	HU0008	Gestion de conversaciones	2	Conversacion-RN-3
9	REQ-CONV-004	El sistema debe conservar la configuracion de idiomas de la conversacion para todos sus mensajes.	HU0009	Gestion de conversaciones	2	Conversacion: Configurar Parametros
10	REQ-CONV-005	El sistema debe guardar cada mensaje original y su traduccion en el hilo de la conversacion, en orden.	HU0010	Gestion de conversaciones	2	LenguajePorConversacion
11	REQ-INT-001	El sistema debe mostrar y reproducir la traduccion para la otra persona sin pedirle cuenta ni instalar la app.	HU0019	Participacion del interlocutor	2	Impacto del Interlocutor
12	REQ-INT-002	El sistema debe traducir la respuesta de la otra persona al idioma del usuario dentro de la misma conversacion.	HU0020	Participacion del interlocutor	2	Conversacion-RN-2
13	REQ-IDIOMA-001	El sistema debe mostrar solo los idiomas activos con sus regiones.	HU0023	Administracion de idiomas y regiones	2	Lenguaje: Listar; Region: Listar
14	REQ-CONV-006	El sistema debe listar solo las conversaciones del usuario, paginadas y ordenadas por actividad reciente.	HU0011	Gestion de conversaciones	2	Conversacion: Consultar historial
15	REQ-CONV-007	El sistema debe mostrar los mensajes originales y traducidos de una conversacion del usuario, y negar el acceso a las de otra persona.	HU0012	Gestion de conversaciones	2	Conversacion: Consultar
16	REQ-USR-001	El sistema debe registrar usuarios con nombre, apellido, correo unico, contrasena segura, fecha de nacimiento (13 anos o mas) y aceptacion de la politica de tratamiento de datos.	HU0015	Cuenta de usuario	3	Usuario-RN-1 a RN-4; Ley 1581
17	REQ-USR-002	El sistema debe iniciar sesion con correo y contrasena, bloquear temporalmente tras intentos fallidos y mantener la sesion con un token de refresco.	HU0016	Cuenta de usuario	3	Usuario-RN-5
18	REQ-REP-001	El sistema debe permitir reportar una traduccion propia una sola vez, con un motivo de 3 a 500 caracteres.	HU0021	Reportes	3	Lenguaje: Generar reporte
19	REQ-CONV-008	El sistema debe permitir renombrar con 3 a 80 caracteres, sin caracteres especiales ni palabras ofensivas, sin cambiar el identificador.	HU0013	Gestion de conversaciones	2	Conversacion-RN-4, RN-5
20	REQ-CONV-009	El sistema debe eliminar la conversacion con sus mensajes y reportes, previa confirmacion.	HU0014	Gestion de conversaciones	2	Conversacion: Eliminar
21	REQ-USR-003	El sistema debe enviar un codigo de 6 digitos valido por 15 minutos y de un solo uso para definir una nueva contrasena segura.	HU0017	Cuenta de usuario	3	Usuario-RN-6 a RN-9
22	REQ-USR-004	El sistema debe invalidar el token de acceso y el de refresco al cerrar sesion.	HU0018	Cuenta de usuario	2	Usuario-RN-10
23	REQ-REP-002	El sistema debe permitir al administrador listar, filtrar y marcar reportes como revisados o descartados.	HU0022	Reportes	3	Rol administrador

Prio	ID	Requisito	Historia	Epica	Sprint	Regla o fuente
24	REQ-IDIOMA-002	El sistema debe permitir al administrador agregar idiomas (nombre y codigo unicos) y regiones (unicas dentro del pais).	HU0024	Administracion de idiomas y regiones	3	Combinaciones unicas
25	REQ-IDIOMA-003	El sistema debe permitir al administrador activar o desactivar idiomas sin afectar las conversaciones existentes.	HU0025	Administracion de idiomas y regiones	3	Lenguaje.activo
26	REQ-USR-005	El sistema debe eliminar la cuenta y todos sus datos cuando el usuario lo pida y confirme con su contrasena.	HU0026	Cuenta de usuario	3	Ley 1581 (derecho al olvido)

3. Requisitos no funcionales

ID	Atributo de calidad	Requisito y criterio medible	Driver	Como se verifica
RNF-001	Desempeno	"Tiempo real" es traduccion por turno: texto en 3 s o menos (p95); voz en 5 s o menos, contando transcripcion y audio de salida.	DRV-01	`npm run medir`, `npm run medir:voz` y evento de log `traduccion` (`duracion_ms`)
RNF-002	Exactitud cultural	El 80 % de los modismos del glosario de prueba se traducen con su equivalente cultural, no literal.	DRV-02	`npm run medir` con el motor de produccion (Claude en Vertex)
RNF-003	Seguridad y privacidad	Datos personales segun *documento 16 (Matriz de sensibilidad de datos)*: mensajes cifrados en reposo (AES-GCM), acceso solo del dueno, logs sin datos personales, consentimiento y eliminacion de cuenta (Ley 1581 de 2012).	DRV-03	Pruebas `api.test.js` y `seguridad.test.js`
RNF-004	Extensibilidad	Se agregan idiomas y regiones sin cambiar codigo.	DRV-04	Prueba del administrador
RNF-005	Disponibilidad	Si un motor de IA falla, el servicio sigue: traduccion Claude (Vertex) -> reglas, que solo adaptan los modismos que conocen (sin coincidencias responden un error claro, no una traduccion palabra por palabra); voz ElevenLabs -> Azure -> OpenAI -> voz del celular.	DRV-05	`motores.test.js`
RNF-006	Seguridad (abuso)	30 mensajes por minuto por usuario, 8 intentos de inicio de sesion fallidos, 5 solicitudes de recuperacion cada 15 minutos.	DRV-06	Pruebas de seguridad
RNF-007	Portabilidad	Una sola base de codigo para iOS, Android y web.	DRV-07	Compilacion web y Expo Go
RNF-008	Mantenibilidad	Cobertura de pruebas de 80 % o mas en la logica de negocio; piramide de pruebas (unitarias, integracion y punta a punta) en el CI; arquitectura hexagonal con puertos verificados al arrancar y reglas de dependencia validadas con dependency-cruiser.	DRV-08	`npm run coverage`, `npm run arch:check` y CI
RNF-009	Interoperabilidad	La API sigue la convencion REST del proyecto y esta documentada en OpenAPI 3.0.	DRV-08	`backend/openapi.yaml` y `/api/v1/docs`

ID	Atributo de calidad	Requisito y criterio medible	Driver	Como se verifica
RNF-010	Naturalidad de la voz	La voz de salida replica la emocion detectada (enojo, alegria, preocupacion, amabilidad, formalidad); 7 de cada 10 personas de prueba la califican como comprensible y natural.	DRV-02	Prueba de escucha con usuarios y audios de `npm run medir:voz`
RNF-011	Seguridad de la cadena de suministro	0 vulnerabilidades HIGH o CRITICAL en las dependencias (de produccion y de desarrollo) y 0 secretos en el codigo, salvo excepciones documentadas con fecha de vencimiento.	DRV-03	Trivy en CI en cada pull request y cada lunes (`.github/workflows/seguridad.yml`)

4. Trazabilidad impacto -> requisito -> driver

Impacto (mapa de impacto)	Requisitos funcionales	Drivers y RNF
1. Interactuar facil y rapido con quien habla otro idioma, conservando cultura, tono y modismos	REQ-TRAD-001, 002, 003	DRV-01 / RNF-001, DRV-02 / RNF-002, DRV-05 / RNF-005
2. Sostener una conversacion natural, hablando y escuchando	REQ-TRAD-004, 005	DRV-01 / RNF-001, DRV-02 / RNF-010
3. Empezar a conversar en menos de un minuto	REQ-CONV-001, 002, REQ-IDIOMA-001	DRV-04 / RNF-004
4. Mantener el contexto de toda la charla	REQ-CONV-003, 004, 005	DRV-08 / RNF-008
5. Que el interlocutor participe sin instalar la app	REQ-INT-001, 002	DRV-07 / RNF-007
6. Aprender con conversaciones pasadas	REQ-CONV-006, 007, 008, 009	DRV-03 / RNF-003
7. Cuenta segura que solo el usuario ve	REQ-USR-001 a 005	DRV-03 / RNF-003, DRV-06 / RNF-006
8. Senalar traducciones incorrectas	REQ-REP-001	DRV-02 / RNF-002
9. Priorizar mejoras con los reportes	REQ-REP-002	DRV-03 / RNF-003
10. Ampliar idiomas y regiones con calidad	REQ-IDIOMA-002, 003	DRV-04 / RNF-004

5. Fuera del alcance del MVP

- Compartir o exportar conversaciones y editar los datos del perfil: no tienen regla en el modelo de dominio; quedan para una version futura.
- Detectar el acento por el sonido de la voz: en el MVP la region de origen se reconoce por el vocabulario.
- Reentrenar el modelo de IA con los reportes: los reportes se guardan y se revisan; el reentrenamiento es evolucion futura.

6. Drivers de arquitectura

Fase 3 - Arquitectura

Version: 1.0 (29/10/2026)

Un driver es un requisito que obliga a tomar decisiones de arquitectura. Cada driver tiene una meta medible, la forma de medirla y las decisiones (ADR, *documento 11 (Decisiones de arquitectura (ADR))*) que lo responden. Los dos que definen el producto son DRV-01 (tiempo por turno) y DRV-02 (calidad cultural y emocional).

1. Drivers

ID	Driver	Meta medible	Origen
DRV-01	Tiempo de respuesta por turno	Texto: 3 s o menos (p95). Voz: 5 s o menos, contando transcripcion y audio de salida	RNF-001, vision
DRV-02	Calidad cultural y emocional	80 % de los modismos del glosario con su equivalente cultural; la emocion detectada se escucha en la voz de salida	RNF-002, RNF-010, vision
DRV-03	Privacidad de los datos	0 accesos a datos de otro usuario; mensajes cifrados en reposo; logs sin datos personales; consentimiento y borrado de cuenta (Ley 1581 de 2012)	RNF-003
DRV-04	Crecimiento de idiomas	Agregar un idioma o una region sin cambiar codigo	RNF-004, impacto del Administrador
DRV-05	Continuidad ante fallas de IA	Si falla un motor de traduccion o de voz, el servicio sigue con el siguiente	RNF-005
DRV-06	Costo y abuso	Whisper local sin costo por mensaje; la traduccion (Claude en Vertex) y la voz (ElevenLabs) cuestan por uso, por eso hay limite de 30 mensajes por minuto y 40 voces por minuto por usuario	RNF-006
DRV-07	Una base de codigo para varias plataformas	Mismo codigo para iOS, Android y web	RNF-007
DRV-08	Facilidad de prueba y mantenimiento	Reglas probadas sin internet ni claves; cobertura de 80 % o mas; reglas de dependencia entre capas verificables con dependency-cruiser	RNF-008, RNF-009

2. Como se responde y como se comprueba

Driver	Decisiones	Como se mide	Resultado actual
DRV-01	ADR-002 (REST por turno), ADR-004 (motores en cadena), ADR-014 (voz con Whisper), ADR-019 (voz en la nube), ADR-020 (traduccion con Claude en Vertex)	`npm run medir` (texto) y `npm run medir:voz` (voz) en `backend/`; evento de log `traduccion` con `duracion_ms`	Pendiente: medir con los motores de produccion (Claude en Vertex, Whisper y ElevenLabs) en el equipo de despliegue; Claude y ElevenLabs son llamadas a la nube y su latencia hay que comprobarla

Driver	Decisiones	Como se mide	Resultado actual
DRV-02	ADR-004 (prompt de equivalencia cultural), ADR-014 (entonacion por texto y voz), ADR-015 y ADR-019 (voz con emocion), ADR-020 (transcreacion cultural)	`npm run medir` con un glosario de prueba separado del usado para construir las reglas; prueba de escucha con usuarios	El motor de reglas logra 14 de 15 modismos; el resultado que cuenta para el driver es el del motor de produccion (Claude en Vertex) con un glosario separado, que esta pendiente
DRV-03	ADR-006 (autenticacion), ADR-010 (seguridad y datos personales), ADR-018 (Trivy)	Pruebas de integracion y unitarias de seguridad; escaneo de Trivy en cada pull request y cada semana	Cumple: aislamiento entre usuarios, cifrado en reposo, consentimiento, borrado de cuenta y logs sin datos personales. Ojo: el texto sale a terceros (Google Cloud para traducir y ElevenLabs para la voz); ver *documento 17 (Politica de tratamiento de datos personales)*
DRV-04	ADR-003 (capas), ADR-008 (modelo de dominio)	Prueba del administrador que activa un idioma y agrega una region	Cumple
DRV-05	ADR-004 (Circuit Breaker), ADR-015 (cadena de voz)	`backend/test/motores.test.js`	Cumple de forma parcial: Claude (Vertex) -> reglas, que solo adaptan los modismos que conocen y sin coincidencias responden un error claro; ElevenLabs -> Azure -> OpenAI -> voz del celular (ver ADR-020)
DRV-06	ADR-004, ADR-010, ADR-016	Limites en `src/adaptadores/http/limites.js` y pruebas de seguridad	Cumple
DRV-07	ADR-001 (Expo)	Compilacion web y ejecucion en Expo Go	Cumple en web y en Expo Go
DRV-08	ADR-002, ADR-003, ADR-013 (hexagonal), ADR-017 (piramide de pruebas)	Piramide de pruebas en CI: unitarias, integracion con `npm run coverage`, punta a punta de la API y de la app web; `npm run arch:check` (dependency-cruiser) para las capas	Cumple: 78 unitarias, 29 de integracion (18 de la API contra SQLite y SurrealDB) y 4 de punta a punta; cobertura de lineas de 96.5 %; `arch:check` sin violaciones

7. Arquitectura

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026)

Este es el documento de entrada a la arquitectura de TalkLate. Describe la **arquitectura construida** y **justifica cada decision con su ADR** (*documento 11 (Decisiones de arquitectura (ADR))*) y el driver que la obliga (*documento 6 (Drivers de arquitectura)*). Cada afirmacion apunta al codigo real (`backend/src`, `frontend/`).

Como leer esta entrega

Documento	Vista C4 / tema	Contenido
documento 7 (Arquitectura) (este)	Sintesis y justificacion	Todas las vistas amarradas y justificadas por ADR
documento 6 (Drivers de arquitectura)	Drivers	Requisitos que obligan decisiones (DRV-01 a DRV-08)
documento 8 (Stack tecnologico)	Stack	Tecnologia por capa y alternativas evaluadas
documento 9 (Diagramas C4)	C4 Niveles 1 y 2	Contexto del sistema y contenedores
documento 10 (Componentes de la API)	C4 Nivel 3 (API)	Componentes internos de la API TalkLate
documento 11 (Decisiones de arquitectura (ADR))	Decisiones	18 ADR con contexto, alternativas y consecuencias
documento 12 (Modelo de datos)	Datos	Modelo de dominio, reglas y correspondencia con la base
`diagramas/arquitectura_talklate.html`	Vista general interactiva	Diagrama de arquitectura (Archify) con el flujo de voz
`diagramas/arquitectura_hexagonal.html`	Vista interactiva del backend	Nucleo, 9 puertos y adaptadores (Archify)

1. Estilo arquitectonico

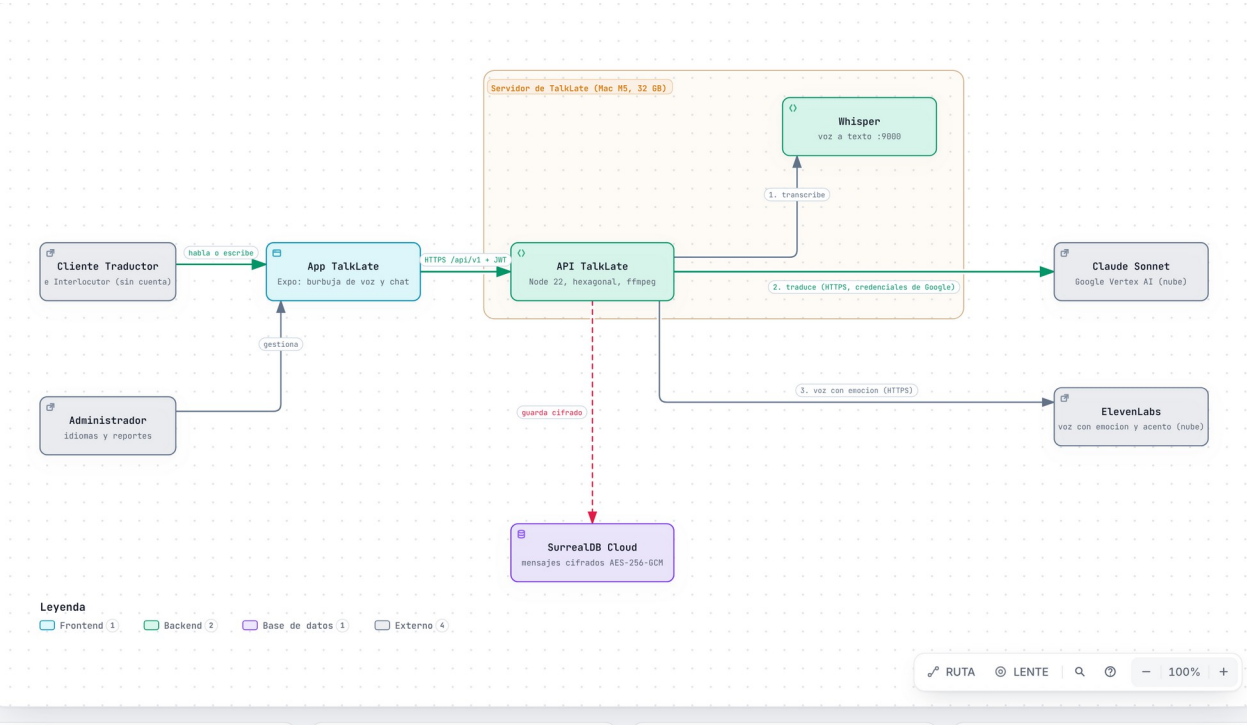


Figura 1. *Vista general de la arquitectura de TalkLate con el flujo de voz (Archify)*

TalkLate es una arquitectura **cliente-servidor** de tres contenedores:

- **App movil** multiplataforma en Expo / React Native, una sola base de codigo para iOS, Android y web (ADR-001, responde a DRV-07).
- **API TalkLate**, un **monolito modular** en Node.js 22 + Express con API REST sincrona `/api/v1` (ADR-002, ADR-011, responde a DRV-01, DRV-08).
- **Base de datos** SurrealDB 3.x, detras de un patron **Repository** que permite usar SQLite en pruebas (ADR-012, responde a DRV-04, DRV-08).

El corazon del producto -traducir conservando tono, intencion, emocion y modismos- vive en tres subsistemas que dependen de motores de IA, cada uno detras de un puerto y con respaldo:

Subsistema	Puerto	Motores (en orden)	ADR	Drivers
Transcripcion de voz	`Transcriptor`	Whisper (API compatible con OpenAI: whisper.cpp o faster-whisper locales)	ADR-014	DRV-01, DRV-06
Traduccion cultural	`MotorTraduccion`	Claude Sonnet en Google Vertex AI -> motor de reglas, con Circuit Breaker (Qwen local queda como adaptador alternativo)	ADR-004, ADR-020	DRV-01, DRV-02, DRV-05
Voz con emocion	`Sintetizador`	ElevenLabs -> Azure Speech -> OpenAI -> voz del celular	ADR-015, ADR-019	DRV-02, DRV-05, DRV-06
Genero de quien habla	`DetectorGenero`	Pitch del audio con ffmpeg; si no se decide, lo infiere el modelo desde el texto	-	DRV-02

La entonacion de un mensaje hablado se estima en el dominio (`dominio/prosodia.js`) con el volumen, la variacion y la velocidad de la voz, y se combina con el tono de las palabras. Ese tono viaja al prompt de traduccion y al sintetizador, que lo convierte en emocion audible (`dominio/voz.js`).

Por dentro, la API sigue una **arquitectura hexagonal (puertos y adaptadores)** (ADR-013): el dominio y los casos de uso estan en el centro y no dependen de Express, de la base ni de los motores de IA; cada dependencia externa entra por un puerto (`Repositorio`, `MotorTraduccion`, `Transcriptor`, `Sintetizador`, `Notificador`, `Cifrador`, `Hasheador`, `FirmadorTokens`) y se conecta con un adaptador. Las reglas de dependencia entre capas se verifican con dependency-cruiser (`npm run arch:check`). El chequeo corre en el CI en cada pull request y hoy no marca ninguna violacion: el hash de contrasenas (bcrypt) y los tokens (JWT) tambien entran por puertos (`Hasheador` y `FirmadorTokens`).

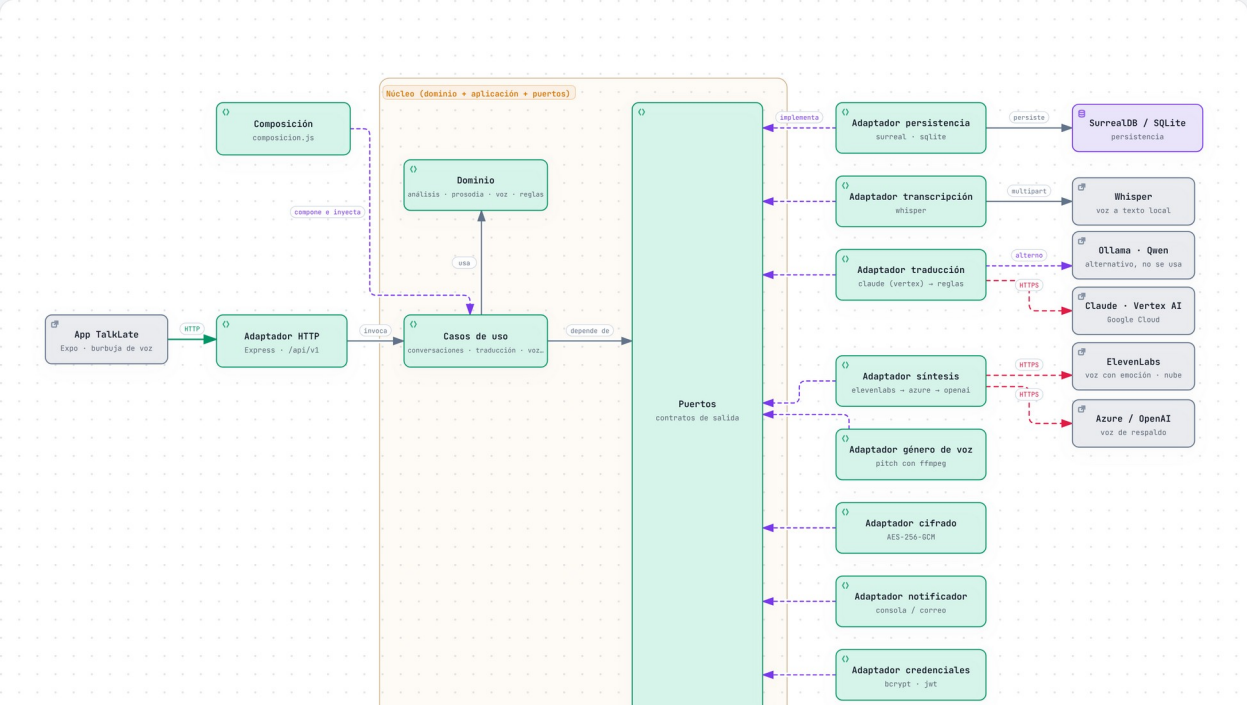


Figura 2. *Backend hexagonal: nucleo, 9 puertos y adaptadores (Archify)*

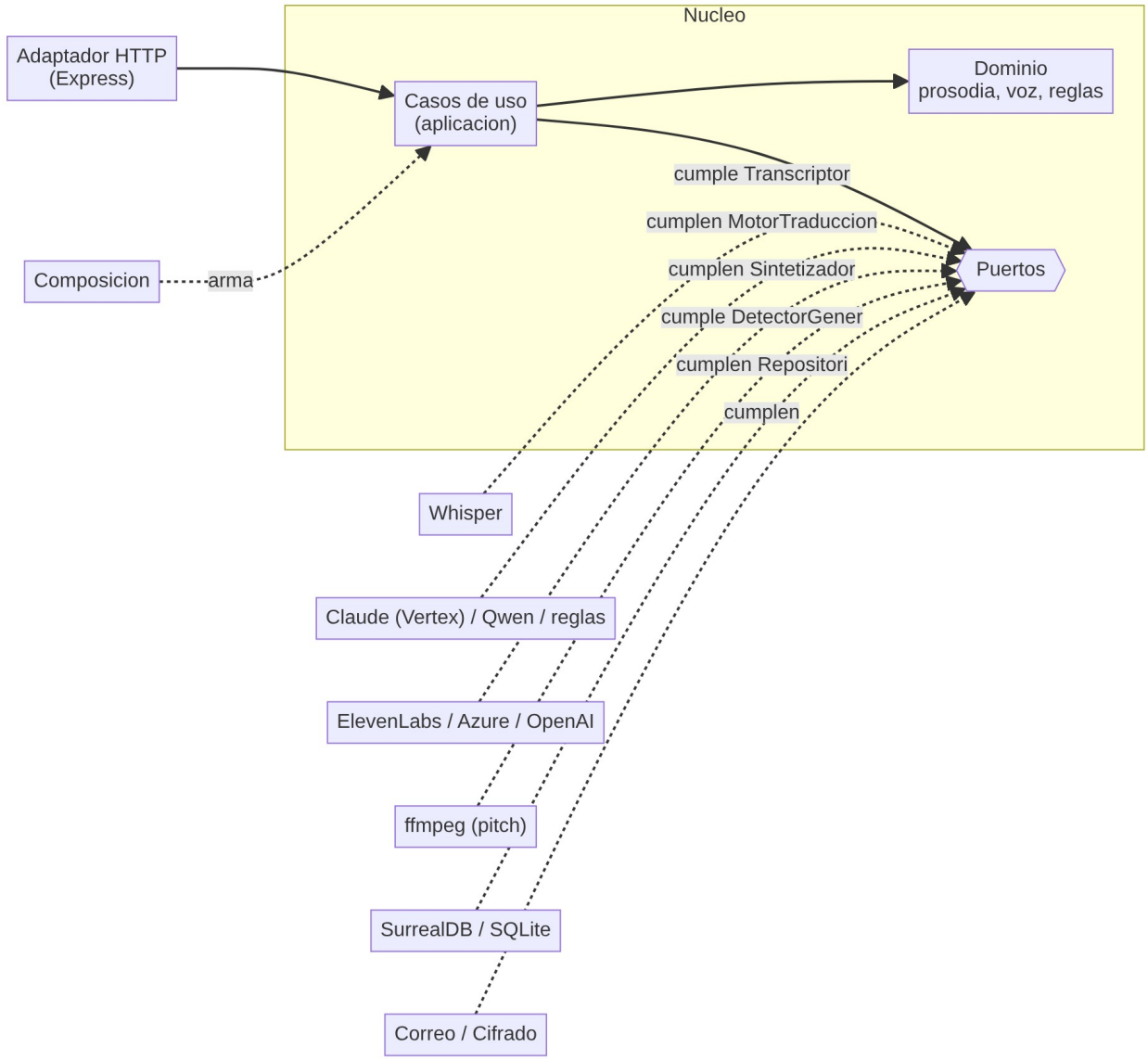


Figura 3. *1. Estilo arquitectonico*

2. Como la arquitectura responde a los drivers

Por cada driver: que se decidio (ADR), con que mecanismo del codigo se realiza y como se comprueba.

Driver	Decision (ADR)	Mecanismo en el codigo	Como se comprueba
DRV-01 Tiempo por turno	ADR-002 (REST por turno), ADR-004 (motores en cadena, sin reintentos), ADR-014, ADR-019, ADR-020	`aplicacion/traduccion.js`: al fallar un motor se pasa al siguiente y el circuito evita esperar el timeout en cada mensaje; Whisper corre local; Claude (Vertex) y ElevenLabs son llamadas a la nube, con tiempo limite y su latencia por medir	`npm run medir` y `npm run medir:voz`; evento de log `traduccion` con `duracion_ms`

Driver	Decision (ADR)	Mecanismo en el codigo	Como se comprueba
DRV-02 Calidad cultural y emocional	ADR-004 (prompt de equivalencia cultural), ADR-014 (entonacion), ADR-015 y ADR-016 (voz con emocion)	`adaptadores/traduccion/prompt.js` arma el prompt con tono, prosodia y region destino; `dominio/voz.js` traduce el tono a estilo SSML, instruccion o vector de emocion	`npm run medir` con glosario de prueba; audios de `npm run medir:voz`; prueba de escucha
DRV-03 Privacidad	ADR-006 (JWT + refresco), ADR-010 (proteccion de datos)	`adaptadores/cifrado/aesGcm.js` (AES-256-GCM en reposo), `compartido/log.js` (JSON sin datos personales), aislamiento por usuario, eliminar cuenta; los audios de voz solo viven en memoria 5 minutos	`test/api.test.js`, `test/seguridad.test.js`
DRV-04 Crecimiento de idiomas	ADR-008 (LenguajeRegion N:M), ADR-012 (catalogo en datos)	`aplicacion/catalogo.js` + puerto `Repositorio.catalogo`: alta y activacion de idiomas y regiones sin tocar codigo	Prueba del administrador
DRV-05 Continuidad ante fallas	ADR-004 (Circuit Breaker), ADR-015 (cadena de voz)	Circuito abierto 60 s por motor; motor de reglas y voz del celular como ultimo recurso	`test/motores.test.js`
DRV-06 Costo y abuso	ADR-004, ADR-010, ADR-020	Whisper local sin costo; Claude (Vertex) y ElevenLabs cuestan por uso, asi que `adaptadores/http/limites.js` pone limites: 30 mensajes por minuto, limites de registro, recuperacion y voz	Pruebas de seguridad
DRV-07 Multiplataforma	ADR-001 (Expo), ADR-007 (template Luna)	Una base de codigo en `frontend/`; grabacion con `expo-audio`, que funciona en Expo Go	Compilacion web y Expo Go
DRV-08 Facilidad de prueba	ADR-002, ADR-012, ADR-013	Adaptador SQLite sin servidor ni claves; casos de uso probados con adaptadores falsos; dependency-cruiser verifica las capas	`npm run coverage` (cercana al 96 %), `npm run arch:check`, CI contra SQLite y SurrealDB

3. Vista de contexto (C4 Nivel 1)

Quien usa TalkLate (Cliente Traductor, Interlocutor, Administrador) y con que sistemas externos habla: Claude Sonnet (Google Vertex AI), Whisper, ElevenLabs (con Azure Speech u OpenAI de respaldo) y la voz del dispositivo. Diagrama completo en *documento 9 (Diagramas C4)*.

4. Vista de contenedores (C4 Nivel 2)

Tres contenedores propios (App, API, Base de datos) y los servicios de IA (uno local y dos en la nube). Diagrama completo en *documento 9 (Diagramas C4)*. Justificacion por contenedor:

Contenedor	Decision	ADR
App movil (Expo)	Una base de codigo multiplataforma, reusa el template Luna; graba la voz con `expo-audio`	ADR-001, ADR-007, ADR-014
API TalkLate (Node + Express)	Monolito modular hexagonal, REST sincrona por turno	ADR-002, ADR-011, ADR-013

Contenedor	Decision	ADR
Base de datos (SurrealDB)	Fuente de verdad detras de un repositorio; enlaces nativos, ASSERT y UNIQUE	ADR-012
Servicios de IA	Whisper corre en el servidor de TalkLate; la traduccion (Claude en Google Vertex AI) y la voz (ElevenLabs) son servicios de nube que reciben solo texto	ADR-014, ADR-019, ADR-020

5. Vista de componentes (C4 Nivel 3)

5.1 API TalkLate

Detalle completo (componentes, bounded contexts y trazabilidad requisito -> componente) en *documento 10 (Componentes de la API)*. En resumen, en capas hexagonales: Adaptador HTTP (rutas, sesion, limites) -> Casos de uso (cuentas, sesiones, conversaciones, traduccion, voz, catalogo, reportes) -> Dominio (reglas, prosodia, voz) y Puertos <- Adaptadores de salida (motores Claude (Vertex), Qwen y reglas; transcriptor Whisper; sintetizadores ElevenLabs, Azure y OpenAI; detector de genero; repositorios SurrealDB y SQLite; notificador; cifrador), conectados por la Composicion.

5.2 App movil

El contenedor App reusa el sistema de componentes del template Luna (ADR-007) y agrega las vistas y la logica de TalkLate. Es un cliente delgado: toda la logica de negocio esta en la API.

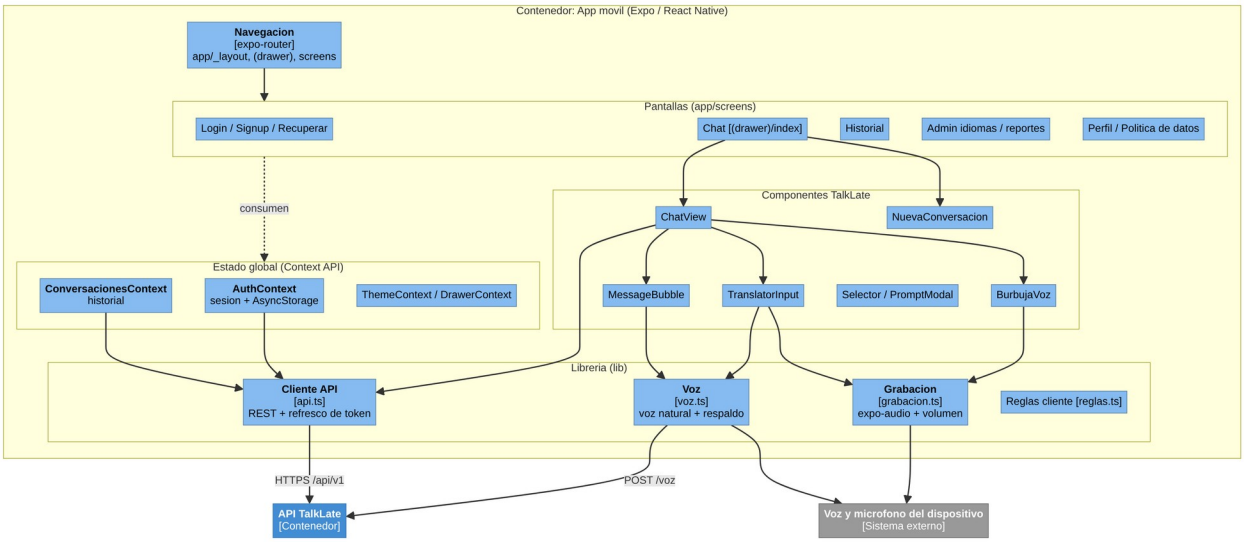


Figura 4. *5.2 App movil*

Componente	Responsabilidad	Archivo(s)	ADR
Navegacion	Rutas y layout (drawer, pantallas)	`app/_layout.tsx`, `app/(drawer)/`, `app/[...404].tsx`	ADR-001
Pantallas	Una vista por etapa del journey (ver *documento 13 (Especificacion de vistas y UI)*)	`app/screens/*`	ADR-007

Componente	Responsabilidad	Archivo(s)	ADR
AuthContext	Sesion en memoria + persistencia en AsyncStorage; renueva usuario al abrir	`app/contexts/AuthContext.tsx`	ADR-006
ConversacionesContext	Historial del usuario para el menu lateral y la pantalla de historial	`app/contexts/ConversacionesContext.tsx`	ADR-007
ChatView	Pantalla del traductor con dos modos: burbuja de voz (por defecto) y chat	`components/talklate/ChatView.tsx`	ADR-007, ADR-014
BurbujaVoz	Burbuja animada que representa la IA: escucha (ondas), traduce (gira) y habla	`components/talklate/BurbujaVoz.tsx`	ADR-014
TranslatorInput + MessageBubble	Modo chat: escribir o grabar, mostrar y reproducir la traduccion	`components/talklate/*`	ADR-007
Grabacion	Graba con `expo-audio`, mide el volumen cada 100 ms y detecta el fin de la frase (1.5 s de silencio)	`lib/grabacion.ts`	ADR-014
Cliente API	REST a `/api/v1`; **refresco automatico del token** (401 -> renueva una vez con guardia single-flight -> repite)	`lib/api.ts`	ADR-006, ADR-011
Voz	Pide la voz natural con emocion a la API (`POST /voz`) y la reproduce; si no hay motor, usa `expo-speech` con tono y velocidad ajustados	`lib/voz.ts`	ADR-015

> *Justificacion: la App es delgada a proposito (ADR-002: la logica vive en la API). La app solo graba y reproduce; transcribir, estimar la entonacion, traducir y sintetizar la voz ocurre en el servidor (ADR-014, ADR-015). La region de origen se reconoce por el vocabulario.*

6. Vista de datos

Modelo de dominio (10 objetos + relacion N:M `LenguajeRegion` + tablas de soporte) y su correspondencia con la base, en *documento 12 (Modelo de datos)*. Justificacion:

Decision	ADR
SurrealDB como fuente de verdad, esquema SCHEMAFULL con ASSERT y UNIQUE	ADR-012
Identificadores del dominio en espanol (lenguaje ubicuo); esquema de la base en ingles	ADR-009
LenguajeRegion N:M y tablas de soporte (`reporte`, `token_refresco`, `sesion_revocada`, `recuperacion`)	ADR-008
Contenido de mensajes cifrado en reposo (AES-256-GCM)	ADR-010

7. Vista dinamica: traducir un mensaje

El flujo central (HU0003). Muestra el Circuit Breaker (ADR-004), el cifrado al persistir (ADR-010) y la entrega por el mismo canal (REQ-TRAD-005).

> *Fuente editable:* `diagramas/secuencia_traduccion.drawio`.

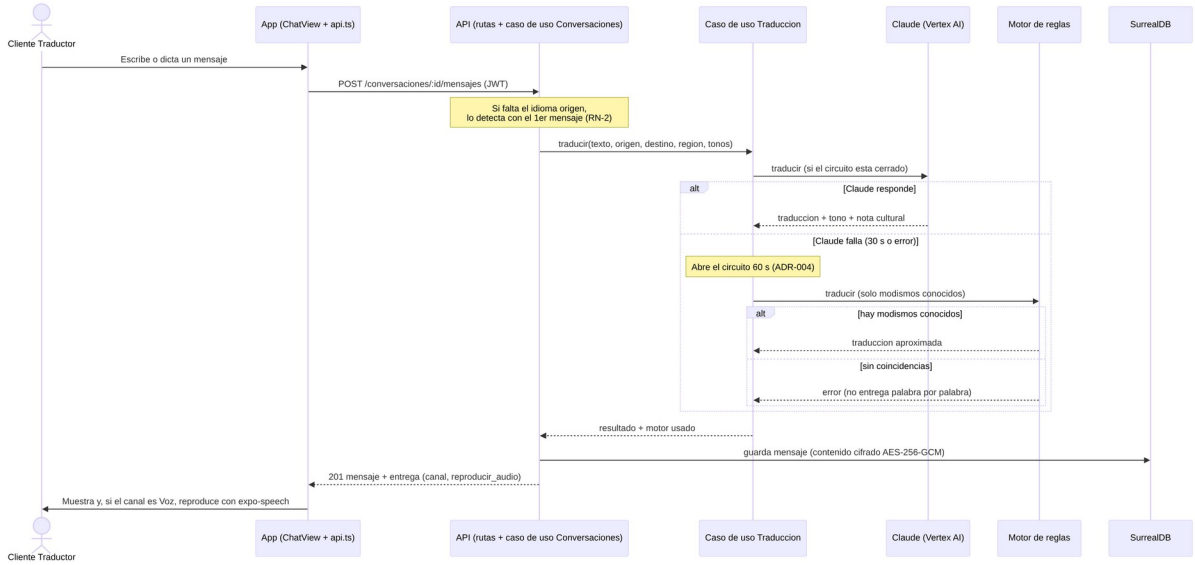


Figura 5. *7. Vista dinamica: traducir un mensaje*

7.1 Mensaje hablado (burbuja de voz)

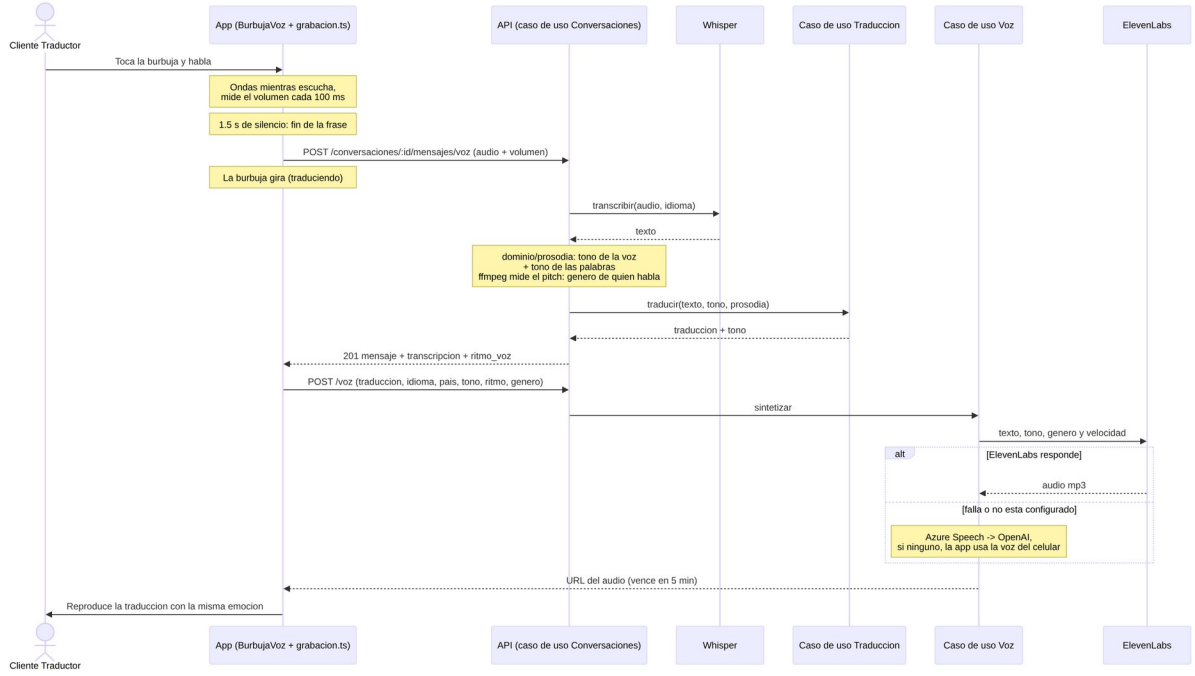


Figura 6. *7.1 Mensaje hablado (burbuja de voz)*

> *El token de acceso dura 15 minutos. Si vence a mitad del flujo, `lib/api.ts` responde al 401 renovando el token con el de refresco (una sola vez, con guardia) y repite la peticion sin que el usuario lo note (ADR-006).*

8. Vista de despliegue

Donde corre cada pieza. La API y Whisper viven en el **servidor** de TalkLate (ADR-014); la traduccion (Claude en Google Vertex AI, ADR-020) y la voz (ElevenLabs, ADR-019) son servicios de **nube**; SurrealDB es una instancia **remota** gestionada (ADR-012). Equipo de referencia para la demostracion: Mac con chip M5 y 32 GB de RAM; se arranca con `./run-talklate.sh`.

> *Fuente editable:* `diagramas/despliegue.drawio`. *Vista interactiva:* `diagramas/arquitectura_talklate.html`.

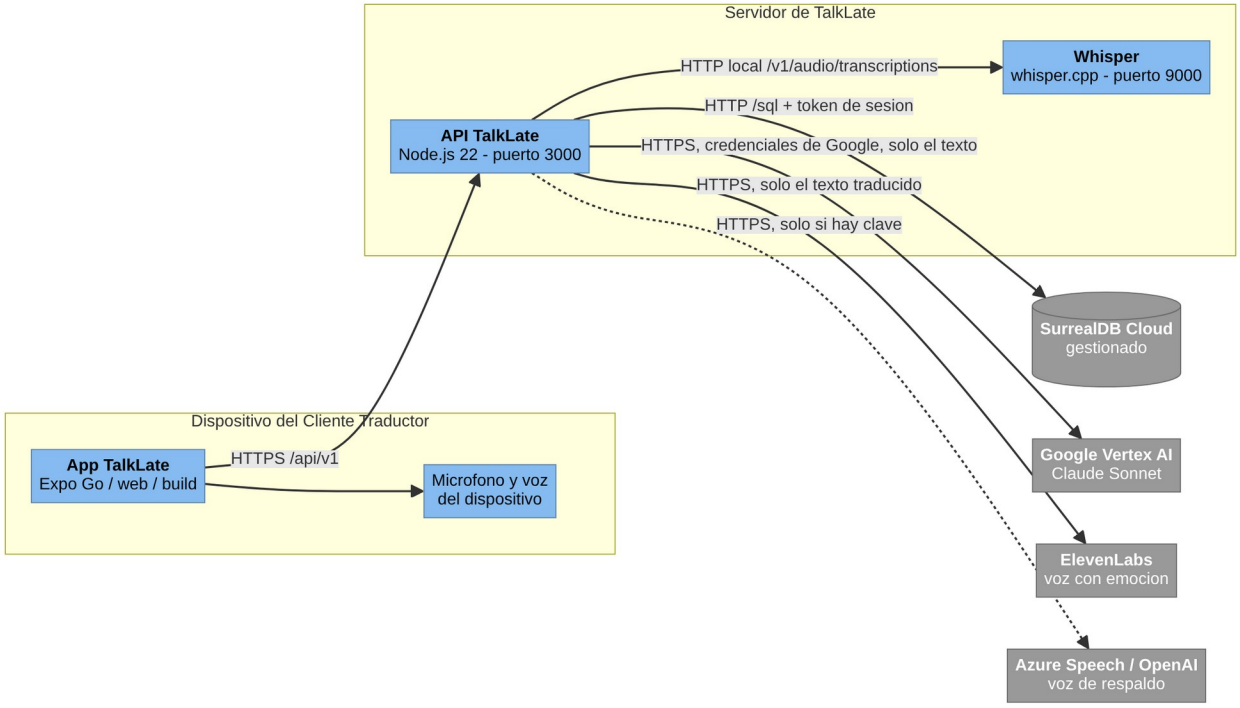


Figura 7. *8. Vista de despliegue*

9. Patrones aplicados

Patrones del catalogo de la convencion del proyecto:

Patron	Donde	ADR
Request-Reply	API REST sincrona por turno	ADR-002, ADR-011
Circuit Breaker (con degradacion aceptable)	Caso de uso de traduccion (`aplicacion/traduccion.js`)	ADR-004
Cadena de respaldo (fallback)	Caso de uso de voz (`aplicacion/voz.js`): ElevenLabs -> Azure -> OpenAI -> voz del celular	ADR-015

Patron	Donde	ADR
Token-Based Authentication	Sesion JWT + refresco rotativo	ADR-006
Repository	Puerto `Repositorio` con dos adaptadores (`adaptadores/persistencia/`)	ADR-012
Puertos y adaptadores (hexagonal)	Toda la API: dominio, casos de uso, puertos, adaptadores y composicion	ADR-013
Structured Logging	Logs JSON sin PII (`compartido/log.js`)	ADR-010

Patrones del catalogo que **no** se usan y por que (microservicios, API Gateway, BFF, CQRS, Event Sourcing, Strangler Fig): ver la nota en *documento 11 (Decisiones de arquitectura (ADR))*.

10. Indice de decisiones (ADR)

Detalle completo en *documento 11 (Decisiones de arquitectura (ADR))*. Resumen de que resuelve cada una:

ADR	Decision	Driver
ADR-001	App Expo (React Native) + NativeWind	DRV-07
ADR-002	Backend monolitico Node + Express, REST sincrona	DRV-07, DRV-08
ADR-003	SQLite inicial (reemplazada por ADR-012; queda para pruebas)	DRV-04, DRV-08
ADR-004	Traduccion con motores en cadena (Circuit Breaker); ajustada por ADR-020	DRV-01, DRV-02, DRV-05, DRV-06, DRV-08
ADR-005	Voz en el cliente con expo-speech (la entrada por voz la reemplaza ADR-014)	DRV-07
ADR-006	JWT de acceso corto + refresco rotativo	DRV-03
ADR-007	Interfaz basada en el template Luna	-
ADR-008	Ajustes al modelo de dominio (LenguajeRegion, soporte)	DRV-04
ADR-009	Identificadores del dominio en espanol	-
ADR-010	Proteccion de datos personales	DRV-03
ADR-011	API v1 REST + OpenAPI	-

ADR	Decision	Driver
ADR-012	SurrealDB como fuente de verdad detras de un repositorio	DRV-01, DRV-03, DRV-04, DRV-08
ADR-013	Arquitectura hexagonal (puertos y adaptadores) dentro del monolito	DRV-04, DRV-05, DRV-08
ADR-014	Voz en Expo Go: grabacion en el celular, Whisper en el servidor y entonacion por texto y voz	DRV-01, DRV-02, DRV-07
ADR-015	Voz natural con emocion en el servidor (puerto Sintetizador)	DRV-02, DRV-05
ADR-016	VoiceStudio local como motor de voz (revertida, ver ADR-019)	DRV-02, DRV-06
ADR-017	Piramide de pruebas en tres niveles, todos en el CI	DRV-08
ADR-018	Escaneo de seguridad con Trivy en el CI	DRV-03
ADR-019	ElevenLabs como motor de voz (reemplaza a ADR-016)	DRV-01, DRV-02
ADR-020	Claude Sonnet en Google Vertex AI como motor de traduccion principal	DRV-02, DRV-03, DRV-05, DRV-06

11. Riesgos tecnicos y mitigaciones

Riesgo	Impacto	Mitigacion
La latencia de Claude (Vertex), Whisper y ElevenLabs supera DRV-01	Conversacion lenta	Medicion con `npm run medir` y `npm run medir:voz` antes de cada demostracion; si la voz natural tarda, la app usa la voz del celular
El tiempo de espera de Claude (30 s) es mayor que el limite de DRV-01	Un turno puede tardar demasiado si Claude se cuelga	El Circuit Breaker abre el circuito y los mensajes siguientes van directo al respaldo
El texto sale a terceros (Google Cloud para traducir, ElevenLabs para la voz)	Privacidad de los mensajes	Solo viaja texto (nunca el audio original); politica de datos y matriz actualizadas; consentimiento al registrarse; Whisper sigue local
Costo por uso de Claude (Vertex) y ElevenLabs, y dependencia de internet	Gasto no controlado o servicio caido	Limites de 30 mensajes y 40 voces por minuto por usuario; respaldo de voz en el celular; falta un tope de gasto diario
Consultas por enlaces sin indice en SurrealDB	Lentitud con muchos datos	Indices sobre los campos de enlace al crecer el volumen (ADR-012)

Riesgo	Impacto	Mitigacion
Servicio de correo para recuperacion	El codigo de recuperacion no llega por correo en produccion	Puerto `Notificador` listo para un proveedor SMTP; en desarrollo el codigo se muestra en la respuesta (ADR-006)
Reconocer la region por el sonido de la voz	Precision regional limitada al vocabulario	Evolucion futura sobre el mismo puerto `Transcriptor`

12. Trazabilidad global

La cadena completa que recorre el proyecto:

Vision (*documento 1 (Vision y contexto)*) -> **Impacto** (*documento 2 (Mapa de impacto)*) -> **Historia** (*documento 3 (Historias de usuario)*) -> **Requisito** (*documento 5 (Requisitos funcionales y no funcionales)*) -> **Driver** (*documento 6 (Drivers de arquitectura)*) -> **ADR** (*documento 11 (Decisiones de arquitectura (ADR))*) -> **Componente** (*documento 10 (Componentes de la API)* y seccion 5.2) -> **Evidencia** (pruebas del backend, `npm run medir` y `npm run medir:voz`).

8. Stack tecnologico

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026). Cada decision tiene su ADR en *documento 11 (Decisiones de arquitectura (ADR))* con las alternativas que se compararon, y responde a un driver de *documento 6 (Drivers de arquitectura)*.

Vista general

Capa	Tecnologia	ADR
App movil	Expo SDK 58, React Native 0.88, expo-router, NativeWind (Tailwind)	ADR-001, ADR-007
Interfaz	Template Luna adaptado (tema claro y oscuro)	ADR-007
Voz (entrada)	Grabacion con `expo-audio` en el celular; transcripcion con Whisper en el servidor (whisper.cpp o faster-whisper, API compatible con OpenAI); entonacion por texto y voz	ADR-014
Voz (salida)	ElevenLabs (voz regional con emocion), Azure Speech y OpenAI de respaldo; `expo-speech` como ultimo recurso	ADR-015, ADR-019
Backend	Node.js 22 y Express 4: monolito modular en arquitectura hexagonal, API REST `/api/v1` con OpenAPI	ADR-002, ADR-011, ADR-013
Base de datos	SurrealDB 3.x en la nube (SurrealDB Cloud), detras de un repositorio; SQLite solo en pruebas	ADR-012
Traduccion e IA	Claude Sonnet en Google Vertex AI y motor de reglas espanol/ingles; Qwen local (Ollama) como adaptador alternativo	ADR-004, ADR-020
Autenticacion	JWT de acceso (15 min) y token de refresco rotativo; contrasenas con bcrypt	ADR-006
Proteccion de datos	Mensajes cifrados con AES-256-GCM, consentimiento, eliminar cuenta, logs sin datos personales	ADR-010
Pruebas y CI	`node:test` (cobertura minima 80 %, hoy cercana al 96 %), pruebas contra SurrealDB, TypeScript, dependency-cruiser para las capas; GitHub Actions	ADR-013

Por que cada tecnologia y que alternativas se evaluaron

Capa	Elegida	Por que	Alternativas evaluadas
App movil	Expo (React Native)	Una sola base de codigo para iOS, Android y web; aprovecha el template Luna; se prueba en el celular con Expo Go.	Flutter (otro lenguaje, Dart, y no usa el template); nativo Swift y Kotlin (dos desarrollos para dos personas); web responsiva (microfono y voz limitados).
Backend	Node.js con Express, en arquitectura hexagonal	Mismo lenguaje que la app; monolito modular suficiente para el alcance; la traduccion es por turno, asi que REST basta. Los puertos y adaptadores dan limites claros sin el costo de los microservicios.	Microservicios (complejidad sin necesidad); capas clasicas sin puertos; WebSocket o streaming (solo para traduccion simultanea); Python FastAPI (otro lenguaje); Java Spring Boot (mas pesado).

Capa	Elegida	Por que	Alternativas evaluadas
Base de datos	SurrealDB en la nube	El dominio es una cadena de relaciones que guarda como enlaces nativos y recorre sin JOIN; el esquema con ASSERT y UNIQUE deja las reglas del dominio en la base; es multimodelo (reportes flexibles y, a futuro, busqueda vectorial de modismos sin otra base); gestionada en la nube, con usuarios de permisos limitados.	SQLite (archivo local, no sirve para varios usuarios ni produccion); PostgreSQL (muy maduro, relaciones con JOIN); SDK oficial de SurrealDB (dependencia extra).
Traduccion	Claude Sonnet (Vertex AI) y reglas; Qwen local como alternativa	Transcreacion cultural: adapta modismos y tono por region, con ejemplos del glosario, y explica la adaptacion; el adaptador esta detras del puerto, asi que se puede cambiar de modelo sin tocar el dominio.	DeepL o Google Translate (literal, no explican modismos); solo Qwen local (sin costo ni datos a terceros, pero con la capacidad de un modelo de 7B); solo reglas (no escala); NLLB o Argos (no siguen instrucciones de tono).
Voz (entrada)	`expo-audio` + Whisper en el servidor	Expo Go no trae reconocimiento de voz nativo; grabar en el celular y transcribir en el servidor funciona igual en iOS, Android y web, y deja medir el volumen para estimar la emocion. Whisper corre local, sin costo por mensaje.	Dictado del teclado (no mide la voz ni sirve en Expo Go de forma uniforme); reconocimiento nativo (requiere un build propio de Expo); servicios de nube de pago.
Voz (salida)	ElevenLabs; Azure y OpenAI de respaldo	Voces con acento regional (paisa y New York) y emocion graduada por tono, rapidas (1 a 5 s por frase). VoiceStudio local se descarto por lento en CPU (ADR-016 revertida).	`expo-speech` (suena robotica, sin emocion real); solo Azure (clave y costo); solo OpenAI (costo y datos a un tercero).
Autenticacion	JWT corto y refresco rotativo	Recomendado para apps moviles; permite cerrar sesion de verdad (revocacion).	Sesiones en servidor (guardan estado); Firebase Auth o Auth0 (dependen de un tercero y sacan datos personales).
Datos personales	Cifrado de mensajes y logs sin PII	Cumple la Ley 1581 de 2012 sin dependencias nativas.	Cifrar toda la base (dependencia nativa; se deja al proveedor en produccion).
Nombres del dominio	Espanol en el codigo (la base en ingles)	Trazabilidad con el modelo de dominio y las reglas de negocio.	Todo en ingles (rompe la trazabilidad).

Estructura del codigo (capas)

El backend sigue una arquitectura hexagonal (puertos y adaptadores, ADR-013):

- `backend/src/dominio`: reglas puras (validaciones, errores de negocio, analisis de idioma, tono y region, prosodia de la voz y expresion de cada tono).
- `backend/src/aplicacion`: un caso de uso por epica (cuentas, sesiones, conversaciones, traduccion, voz, catalogo, reportes).
- `backend/src/puertos`: contratos `Repositorio`, `MotorTraduccion`, `Transcriptor`, `Sintetizador`, `Notificador`, `Cifrador`, `Hasheador` y `FirmadorTokens`.

- `backend/src/adaptadores`: HTTP (Express) de entrada; de salida SurrealDB y SQLite, Claude (Vertex y API directa), Qwen y reglas, Whisper, ElevenLabs, Azure y OpenAI, detector de genero, correo y cifrado.
- `backend/src/composicion.js`: elige los adaptadores segun el entorno y arma los casos de uso.
- `backend/db/surreal/schema.surql`: esquema de la base. `backend/openapi.yaml`: contrato de la API.
- `frontend/app`: pantallas (expo-router); `frontend/components/talklate`: componentes de TalkLate; `frontend/lib`: cliente de la API, reglas, grabacion y voz.
- `componentes/template/native-luna/`: template original, sin cambios.
- `backend/.dependency-cruiser.cjs`: reglas de dependencia entre capas (`npm run arch:check`).

Como correrlo

- Backend: `backend/` con `npm start` en `http://localhost:3000` (configuracion en `backend/.env`, ver `backend/.env.example`).
- App: `frontend/` con `npm run web` o Expo Go (SDK 57).
- Arranque local: `./run-talklate.sh` (backend y Expo; con `--full` tambien Whisper). La traduccion necesita credenciales de Google (ADC) y la voz una clave de ElevenLabs.

9. Diagramas C4

Fase 3 - Arquitectura

Version: 1.0 (29/10/2026)

La arquitectura de TalkLate en notacion C4:

- **Nivel 1 - Contexto:** quien usa TalkLate y con que sistemas externos habla.
- **Nivel 2 - Contenedores:** las piezas que forman TalkLate y como se comunican.
- **Nivel 3 - Componentes:** el interior de la API TalkLate, en *documento 10 (Componentes de la API)*.

Fuentes editables en `diagramas/` (`c4_contexto.drawio`, `c4_contenedores.drawio`). Vistas interactivas (Archify): `diagramas/arquitectura_talklate.html` (general, con el flujo de voz) y `diagramas/arquitectura_hexagonal.html` (backend por puertos y adaptadores). La justificacion de cada vista esta en *documento 7 (Arquitectura)*.

Nivel 1 - Contexto del sistema

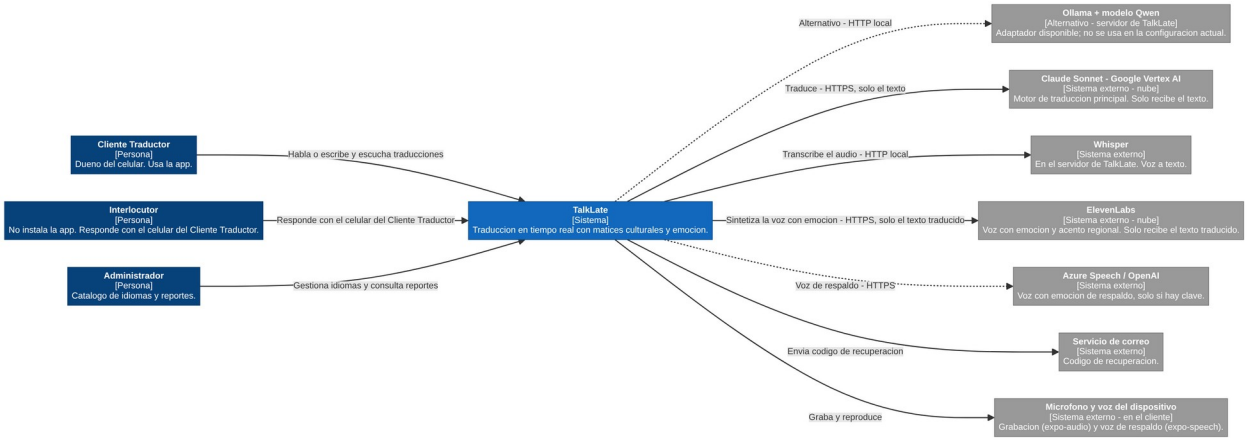


Figura 8. *Nivel 1 - Contexto del sistema*

Nivel 2 - Contenedores

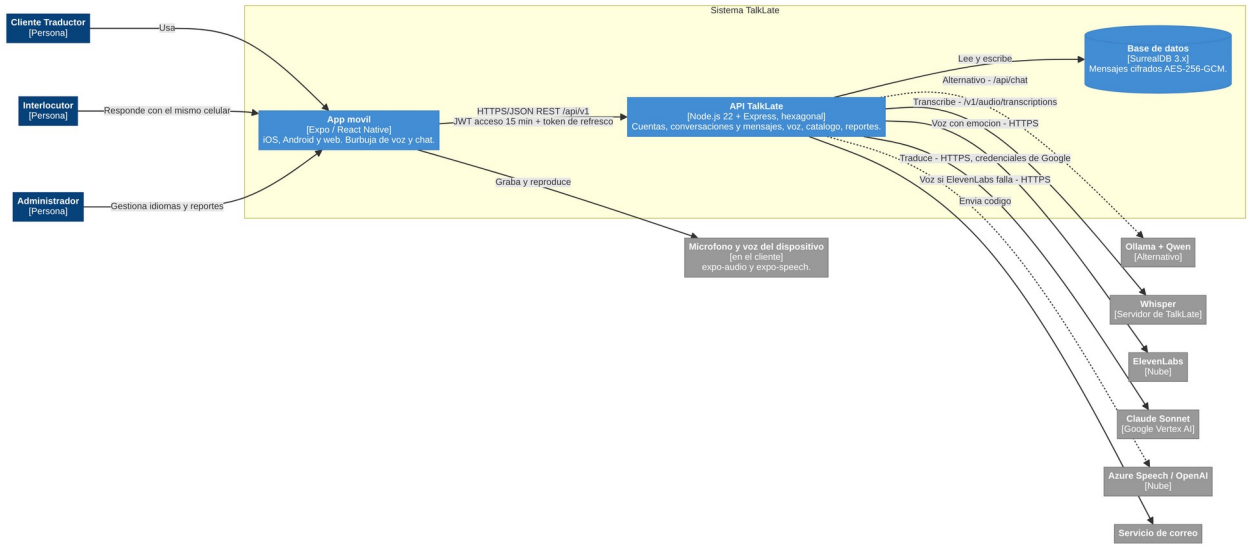


Figura 9. *Nivel 2 - Contenedores*

Tabla de elementos

Elemento	Tipo	Tecnologia	Descripcion
Cliente Traductor	Persona	-	Dueno del celular. Usa la app para traducir sus conversaciones.
Interlocutor	Persona	-	La otra persona de la conversacion. No instala la app; responde con el celular del Cliente Traductor.
Administrador	Persona	-	Gestiona el catalogo de idiomas y revisa reportes.
TalkLate	Sistema	-	Traduccion en tiempo real que considera contexto, emociones, jerga e intencion.
App movil	Contenedor	Expo / React Native (iOS, Android y web)	Burbuja de voz animada y modo chat. Graba la voz, muestra y reproduce traducciones.
API TalkLate	Contenedor	Node.js 22 + Express, arquitectura hexagonal	REST en `/api/v1`. Transcribe, estima la entonacion, traduce y sintetiza la voz con emocion.
Base de datos	Contenedor	SurrealDB 3.x (SQLite solo en pruebas)	Usuarios, conversaciones, catalogo y reportes. Mensajes cifrados con AES-256-GCM.
Ollama + Qwen	Sistema externo (alternativo)	Ollama, HTTP local `/api/chat`	Adaptador alternativo de traduccion, en el servidor de TalkLate; no se usa en la configuracion actual.
Whisper	Sistema externo	whisper.cpp o faster-whisper, API compatible con OpenAI	Pasa el audio a texto, en el servidor de TalkLate.
ElevenLabs	Sistema externo (nube)	API REST (`/v1/text-to-speech/{voz}`, `eleven_multilingual_v2`)	Voz con emocion y acento regional; recibe solo el texto traducido, el tono, el genero y la velocidad.

Elemento	Tipo	Tecnologia	Descripcion
Claude Sonnet (Google Vertex AI)	Sistema externo (nube)	SDK de Anthropic para Vertex, credenciales de Google (ADC)	Motor de traduccion principal (ADR-020); solo recibe el texto y el contexto del prompt.
Azure Speech / OpenAI	Sistema externo	HTTPS (SSML con estilos / `gpt-4o-mini-tts`)	Respaldo de la voz con emocion; solo con clave.
Servicio de correo	Sistema externo	Puerto `Notificador`	Envia el codigo de recuperacion de cuenta.
Microfono y voz del dispositivo	Sistema externo (en el cliente)	expo-audio, expo-speech	Graba el mensaje y reproduce la voz de respaldo.

Comunicaciones principales

Origen	Destino	Protocolo	Detalle
App movil	API TalkLate	HTTPS/JSON REST `/api/v1`	JWT de acceso de 15 min + token de refresco. El audio viaja en base64 (hasta 10 MB).
API TalkLate	Base de datos	HTTP `/sql` con token de sesion	Lectura y escritura; mensajes cifrados AES-256-GCM.
API TalkLate	Whisper	HTTP local multipart `/v1/audio/transcriptions`	Transcripcion del mensaje hablado con pista de idioma.
API TalkLate	Ollama + Qwen	HTTP local `/api/chat`	Alternativo: solo si se configura `TRADUCTOR=auto` o `qwen`.
API TalkLate	Claude Sonnet (Vertex)	HTTPS	Traduccion principal; tiempo limite de 30 s y sin reintentos del SDK.
API TalkLate	Motor de reglas	En proceso	Respaldo parcial: solo traduce modismos conocidos; sin coincidencias responde un error.
API TalkLate	ElevenLabs	HTTPS	Voz con la emocion del tono detectado, por region y genero.
API TalkLate	Azure Speech / OpenAI	HTTPS	Respaldo de la voz si ElevenLabs falla.
App movil	API TalkLate	HTTPS `GET /voz/audio/:id`	Descarga el audio sintetizado (vence en 5 minutos).
API TalkLate	Servicio de correo	Puerto `Notificador`	Envio del codigo de recuperacion.

10. Componentes de la API

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026)

Este documento abre el contenedor **API TalkLate** (*documento 9 (Diagramas C4)*, Nivel 2) y muestra sus componentes internos: que piezas lo forman, de que se encarga cada una y como se comunican. Solo se detalla este contenedor: la App movil se describe en *documento 7 (Arquitectura)* (seccion 5.2) y en *documento 13 (Especificacion de vistas y UI)*, y la base de datos en *documento 12 (Modelo de datos)*.

Metodo: el diagrama refleja el codigo real de `backend/src`. Cada componente cita sus archivos, el Bounded Context al que pertenece y el ADR que lo justifica (*documento 11 (Decisiones de arquitectura (ADR))*).

Organizacion: la API sigue una **arquitectura hexagonal (puertos y adaptadores)** dentro del monolito modular (ADR-013): el dominio y los casos de uso estan en el centro y no dependen de Express, de la base de datos ni de los motores de IA; cada dependencia externa entra por un puerto y se conecta con un adaptador. La composicion (`src/composicion.js`) es el unico lugar que conoce los adaptadores concretos. Las reglas de dependencia se verifican con dependency-cruiser (`backend/.dependency-cruiser.cjs`, `npm run arch:check`). El chequeo corre en el CI en cada pull request y hoy no marca ninguna violacion: el hash de contrasenas (bcrypt) y los tokens (JWT) tambien entran por puertos (`Hasheador` y `FirmadorTokens`).

Diagrama de componentes

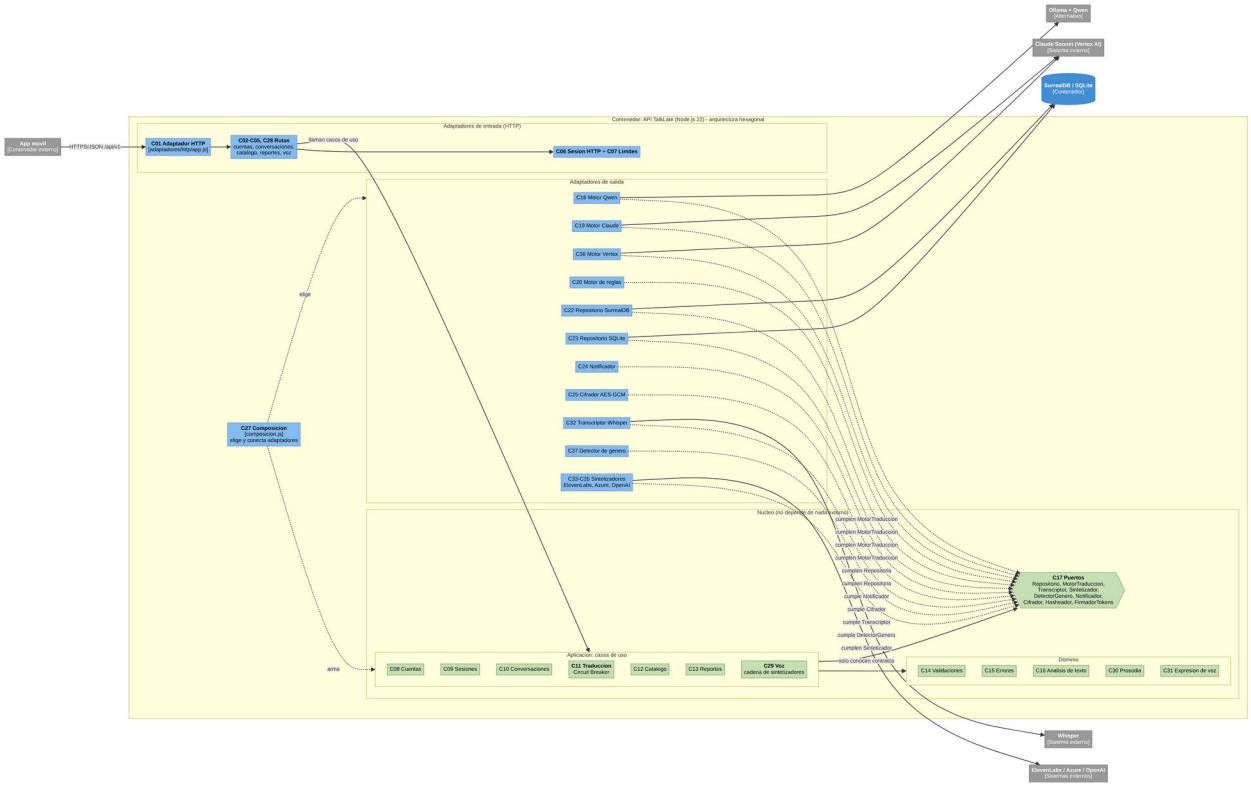


Figura 10. *Diagrama de componentes*

> ***Regla de dependencias:*** *las flechas siempre apuntan hacia el nucleo. Los casos de uso (aplicacion) solo importan el dominio y los puertos; nunca un adaptador, ni Express, ni la base. Los adaptadores de salida "cumplen" un puerto y la composicion verifica al arrancar que lo cumplen (`asegurarPuerto`). Por eso se puede cambiar SurrealDB por otra base, o agregar un motor de traduccion, sin tocar los casos de uso. Log estructurado (C26) lo usan todas las capas.*

Catalogo de componentes

#	Componente	Capa	Bounded Context	Responsabilidad	Archivo(s)	Patron / ADR
C01	Adaptador HTTP	Entrada	Plataforma	Arma Express: CORS, limite de cuerpo, `/health`, OpenAPI/Swagger, monta las rutas `/api/v1`, 404 y manejador global de errores.	`src/adaptadores/http/app.js`, `src/server.js`	ADR-002, ADR-011, ADR-013
C02	Rutas de cuentas	Entrada	Identidad y Acceso	Traduce HTTP a casos de uso: registro, login, refresco, recuperacion, logout, `/usuarios/yo` y eliminar cuenta.	`src/adaptadores/http/rutas/cuentas.js`	ADR-011
C03	Rutas de conversaciones	Entrada	Conversaciones y Mensajes	CRUD de conversaciones, envio de mensajes escritos y hablados (`/mensajes/voz`), `Location` y paginacion.	`src/adaptadores/http/rutas/conversaciones.js`	ADR-011
C04	Rutas de catalogo	Entrada	Catalogo de Idiomas	Consultar el catalogo y las operaciones del administrador.	`src/adaptadores/http/rutas/catalogo.js`	ADR-011
C05	Rutas de reportes	Entrada	Reportes de Calidad	Reportar una traduccion y revision del administrador.	`src/adaptadores/http/rutas/reportes.js`	ADR-011
C06	Sesion HTTP y roles	Entrada	Identidad y Acceso	Lee la cabecera `Authorization` y delega en el caso de uso de sesiones; `requerirAdmin`.	`src/adaptadores/http/sesion.js`	Token-Based Auth (5.1) / ADR-006
C07	Limites de uso	Entrada	Plataforma	Rate limiting por usuario e IP (mensajes, registro, recuperacion).	`src/adaptadores/http/limites.js`	ADR-010 (RNF-006)
C08	Caso de uso Cuentas	Aplicacion	Identidad y Acceso	Registro con consentimiento, login con bloqueo por intentos, recuperacion, cierre de sesion y eliminar cuenta.	`src/aplicacion/cuentas.js`	ADR-006, ADR-010
C09	Caso de uso Sesiones	Aplicacion	Identidad y Acceso	JWT de acceso, token de refresco rotativo y atomico, revocacion.	`src/aplicacion/sesiones.js`	Token-Based Auth (5.1) / ADR-006
C10	Caso de uso Conversaciones	Aplicacion	Conversaciones y Mensajes	Reglas Conversacion-RN-1 a RN-5, deteccion del idioma origen, mensajes hablados (transcribe, valida tipo y tamano del audio, estima la prosodia), orquesta la traduccion, cifra al guardar; paginacion.	`src/aplicacion/conversaciones.js`, `src/aplicacion/paginacion.js`	ADR-004, ADR-010, ADR-014
C11	Caso de uso Traduccion	Aplicacion	Traduccion	Prueba los motores en orden y salta 60 s el que falla (Circuit Breaker); no conoce ningun motor concreto.	`src/aplicacion/traduccion.js`	Circuit Breaker (4.1) / ADR-004, ADR-013
C12	Caso de uso Catalogo	Aplicacion	Catalogo de Idiomas	Alta, edicion y activacion de idiomas y regiones; combinaciones unicas.	`src/aplicacion/catalogo.js`	ADR-008
C13	Caso de uso Reportes	Aplicacion	Reportes de Calidad	Reportar una traduccion propia una vez y revision del administrador.	`src/aplicacion/reportes.js`	ADR-010

#	Componente	Capa	Bounded Context	Responsabilidad	Archivo(s)	Patron / ADR
C14	Validaciones	Dominio	Plataforma	Reglas de formato de entrada (nombres, correo, contrasena, contenido, nombre de conversacion).	`src/dominio/validaciones.js`	ADR-011
C15	Errores	Dominio	Plataforma	`ErrorNegocio` y cuerpo de error uniforme de la API.	`src/dominio/errores.js`	ADR-011
C16	Analisis de texto	Dominio	Traduccion	Detecta idioma, tono y region de origen por vocabulario.	`src/dominio/analisis.js`	ADR-004, ADR-005
C17	Puertos	Puertos	(todos)	Contratos `Repositorio`, `MotorTraduccion`, `Transcriptor`, `Sintetizador`, `DetectorGenero`, `Notificador`, `Cifrador`, `Hasheador` y `FirmadorTokens`, y su verificacion al arrancar.	`src/puertos/index.js`	Puertos y adaptadores / ADR-013
C18	Motor Qwen	Salida	Traduccion	Cumple `MotorTraduccion` via Ollama local (HTTP `/api/chat`).	`src/adaptadores/traduccion/qwen.js`	ADR-004
C19	Motor Claude	Salida	Traduccion	Cumple `MotorTraduccion` via la API de Anthropic (solo si hay clave).	`src/adaptadores/traduccion/claude.js`	ADR-004
C20	Motor de reglas	Salida	Traduccion	Cumple `MotorTraduccion` sin red: espanol/ingles con modismos por pais. Solo traduce si el texto trae modismos conocidos; sin coincidencias falla en vez de entregar una traduccion palabra por palabra.	`src/adaptadores/traduccion/reglas.js`, `acentos.js`	ADR-004
C21	Constructor de prompt	Salida	Traduccion	Prompt de transcreacion cultural comun a Qwen y Claude (incluye tono, prosodia de la voz y ejemplos del glosario), y lectura de su JSON validado (la nota y la region solo se aceptan como texto de largo acotado).	`src/adaptadores/traduccion/prompt.js`	ADR-004
C22	Repositorio SurrealDB	Salida	(todos)	Cumple `Repositorio` sobre SurrealDB (HTTP `/sql` con token de sesion; valores serializados, nunca texto concatenado).	`src/adaptadores/persistencia/surreal/repositorio.js`, `cliente.js`	Repository / ADR-012
C23	Repositorio SQLite	Salida	(todos)	Cumple `Repositorio` sobre SQLite para pruebas sin servidor.	`src/adaptadores/persistencia/sqlite/repositorio.js`, `esquema.js`	Repository / ADR-003
C24	Notificador	Salida	Identidad y Acceso	Cumple `Notificador`; en desarrollo devuelve el codigo de recuperacion en la respuesta; un proveedor de correo se conecta como otro adaptador del mismo puerto.	`src/adaptadores/correo/consola.js`	ADR-006
C25	Cifrador AES-GCM	Salida	Plataforma	Cumple `Cifrador`: cifra/descifra el contenido de los mensajes en reposo.	`src/adaptadores/cifrado/aesGcm.js`	ADR-010
C26	Log estructurado	Transversal	Plataforma	Logs JSON sin datos personales; evento `traduccion` con `duracion_ms`.	`src/compartido/log.js`	Structured Logging (6.1) / ADR-010
C27	Composicion	Arranque	Plataforma	Unico lugar que conoce los adaptadores: los elige segun el entorno (`TRADUCTOR`, `TRANSCRIPCION_URL`, `VOZ_MOTOR`), verifica sus puertos y arma los casos de uso.	`src/composicion.js`	ADR-013

#	Componente	Capa	Bounded Context	Responsabilidad	Archivo(s)	Patron / ADR
C28	Rutas de voz	Entrada	Voz	`POST /voz` (con sesion y limite por usuario) y `GET /voz/audio/:id` (id aleatorio de 128 bits que vence en 5 minutos).	`src/adaptadores/http/rutas/voz.js`	ADR-015
C29	Caso de uso Voz	Aplicacion	Voz	Valida texto, idioma y tono; prueba los sintetizadores en orden; guarda el audio en memoria con vencimiento.	`src/aplicacion/voz.js`	ADR-015
C30	Prosodia	Dominio	Voz	Estima intensidad, ritmo y expresividad con el volumen y las palabras por segundo; combina el tono de la voz con el de las palabras.	`src/dominio/prosodia.js`	ADR-014
C31	Expresion de voz	Dominio	Voz	Define como suena cada tono: voz por idioma y pais, estilo SSML, prosodia, instruccion y vector de emocion.	`src/dominio/voz.js`	ADR-015, ADR-019
C32	Transcriptor Whisper	Salida	Voz	Cumple `Transcriptor`: envia el audio multipart a la API compatible con OpenAI y devuelve el texto.	`src/adaptadores/transcripcion/whisper.js`	ADR-014
C33	Sintetizador ElevenLabs	Salida	Voz	Cumple `Sintetizador`: API REST de ElevenLabs con voz por region (paisa, New York) y genero; la emocion va en `voice_settings`.	`src/adaptadores/sintesis/elevenlabs.js`	ADR-019
C34	Sintetizador Azure	Salida	Voz	Cumple `Sintetizador`: SSML con `mstts:express-as` y prosodia de respaldo.	`src/adaptadores/sintesis/azure.js`	ADR-015
C35	Sintetizador OpenAI	Salida	Voz	Cumple `Sintetizador`: `/v1/audio/speech` con la emocion y el acento como instruccion.	`src/adaptadores/sintesis/openai.js`	ADR-015
C36	Motor Vertex	Salida	Traduccion	Cumple `MotorTraduccion` con Claude Sonnet en Google Vertex AI (SDK oficial, credenciales de Google); tiempo limite de 30 s y sin reintentos del SDK.	`src/adaptadores/traduccion/vertex.js`	ADR-020
C37	Detector de genero	Salida	Voz	Cumple `DetectorGenero`: decodifica el audio con ffmpeg y estima el pitch; menos de 160 Hz hombre, mas de 180 Hz mujer, en medio no decide.	`src/adaptadores/audio/genero.js`	ADR-019
C38	Caso de uso Problemas	Aplicacion	Reportes de Calidad	Reportar un problema general de la app (con limite de 10 por hora por usuario) y revision del administrador.	`src/aplicacion/problemas.js`, `src/adaptadores/http/rutas/problemas.js`	ADR-010

Bounded Contexts

Cada contexto agrupa componentes y las entidades del modelo (*documento 12 (Modelo de datos)*) que gobierna. En el codigo se reconocen por el caso de uso y por el namespace del repositorio.

Bounded Context	Componentes	Namespace del repositorio	Entidades principales
Identidad y Acceso	C02, C06, C08, C09, C24	`usuarios`, `tokens`, `sesiones`, `recuperaciones`	Usuario, TokenRefresco, SesionRevocada, Recuperacion

Bounded Context	Componentes	Namespace del repositorio	Entidades principales
Conversaciones y Mensajes	C03, C10	`conversaciones`, `mensajes`	Conversacion, LenguajePorConversacion, MensajeOriginal, MensajeDestino
Traduccion	C11, C16, C18-C21	- (servicio, sin tablas propias)	-
Catalogo de Idiomas	C04, C12	`catalogo`	Pais, Region, Lenguaje, LenguajeRegion, Canal, Entonacion
Reportes de Calidad	C05, C13	`reportes`	Reporte
Voz	C28-C35	- (audio en memoria, sin tablas)	-
Plataforma (transversal)	C01, C07, C14, C15, C17, C22, C23, C25-C27	-	-

Trazabilidad: requerimiento -> componente

Completa la cadena requisito -> historia -> impacto de *documento 5 (Requisitos funcionales y no funcionales)* hasta el componente que lo realiza.

Requerimiento	Componentes que lo realizan
REQ-TRAD-001 (mensaje hablado)	C03, C10, C30, C32; grabacion con `expo-audio` en la app (ADR-014)
REQ-TRAD-002 (mensaje escrito 1-5000)	C10, C14
REQ-TRAD-003 (traducir con tono/modismos + nota)	C10, C11, C18/C19/C20, C21, C16
REQ-TRAD-004 (voz natural con emocion)	C10 (`ritmo_voz`), C28, C29, C31, C33-C35; `expo-speech` de respaldo en la app
REQ-TRAD-005 (entregar por el mismo canal)	C10
REQ-CONV-001 (idioma/region destino valida)	C10, C14, C22 (`regionValidaParaLenguaje`)
REQ-CONV-002 (reconocer origen y fijarlo)	C10, C16 (`detectarIdioma`), C22 (`conversaciones.crearLpc`)
REQ-CONV-003 (crear conversacion, par soportado)	C10, C11 (`soportaPar`)
REQ-CONV-004 (configurar una sola vez)	C10, C22 (`lpcDe`)
REQ-CONV-005 (guardar hilo en orden)	C10, C25, C22 (`mensajes.crear`)

Requerimiento	Componentes que lo realizan
REQ-CONV-006 (historial paginado)	C03 (`paginar`), C10
REQ-CONV-007 (ver mios, negar ajenos)	C10 (`delUsuario`), C06
REQ-CONV-008 (renombrar 3-80)	C10, C14 (`validarNombreConversacion`)
REQ-CONV-009 (eliminar con cascada)	C10, C22 (`conversaciones.eliminar`)
REQ-USR-001 (registro + consentimiento)	C02, C08, C14
REQ-USR-002 (login + bloqueo + refresco)	C02, C08, C09
REQ-USR-003 (codigo de recuperacion)	C02, C08, C24
REQ-USR-004 (logout invalida tokens)	C02, C08, C09
REQ-USR-005 (eliminar cuenta)	C02, C08
REQ-INT-001 (traduccion al interlocutor sin cuenta)	C10
REQ-INT-002 (responder en mi idioma)	C10 (`remitente=interlocutor`), C11
REQ-REP-001 (reportar una vez)	C05, C13
REQ-REP-002 (admin revisa reportes)	C05, C13, C06 (`requerirAdmin`)
REQ-IDIOMA-001 (ver idiomas activos)	C04, C12
REQ-IDIOMA-002 (agregar idioma/region)	C04, C12, C06, C14
REQ-IDIOMA-003 (activar/desactivar)	C04, C12, C06
RNF-001 (3 s texto, 5 s voz)	C11 (Circuit Breaker), C26 (`duracion_ms`), C32, C33
RNF-003 (proteccion de datos)	C25, C26, C09, C08 (eliminar cuenta)
RNF-005 (degradacion si falla la IA)	C11, C20, C29
RNF-006 (limites de uso)	C07, C08 (bloqueo de login)
RNF-008 (facilidad de prueba)	C17 (puertos), C23, C27 (adaptadores falsos en pruebas)
RNF-009 (API REST + OpenAPI)	C01, C15
RNF-010 (voz con emocion)	C30, C31, C33-C35

Limitaciones conocidas y evolucion

- **C24 Notificador:** en desarrollo el codigo de recuperacion vuelve en la respuesta; en produccion se conecta un proveedor de correo como otro adaptador del mismo puerto (ADR-006).
- **C22 Repositorio SurrealDB:** algunas consultas filtran recorriendo enlaces; se agregan indices sobre esos campos al crecer el volumen (ADR-012).
- **C16 Analisis:** la region de origen se reconoce por vocabulario; reconocerla por el sonido de la voz es evolucion futura sobre el puerto `Transcriptor`.

11. Decisiones de arquitectura (ADR)

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026)

Metodo: las decisiones se validaron construyendo una prueba de concepto (14/10/2026) y se sustentan aqui con su contexto, las alternativas evaluadas y sus consecuencias. Cada ADR responde a uno o mas drivers (*documento 6 (Drivers de arquitectura)*); los patrones citados vienen del catalogo de patrones arquitectonicos del proyecto.

ADR	Decision	Patron del catalogo	Estado
ADR-001	App movil con Expo (React Native) y NativeWind	-	aceptada
ADR-002	Backend monolitico en Node.js con Express, API REST sincrona	Request-Reply (2.3)	aceptada
ADR-003	SQLite como base de datos inicial	-	reemplazada por ADR-012 (queda para pruebas sin servidor)
ADR-004	Traduccion con Qwen local, Claude Sonnet de respaldo y motor de reglas	Circuit Breaker (4.1) con degradacion aceptable	aceptada; el motor principal cambio por ADR-020
ADR-005	Voz: dictado en el cliente y expo-speech para la salida	-	reemplazada en la entrada por ADR-014 y en la salida por ADR-015 (expo-speech queda de respaldo)
ADR-006	Autenticacion con JWT de acceso corto y token de refresco rotativo	Token-Based Authentication (5.1)	aceptada
ADR-007	Interfaz basada en el template Luna	-	aceptada
ADR-008	Ajustes al modelo de dominio al implementarlo	-	aceptada
ADR-009	Identificadores del dominio en espanol (lenguaje ubicuo)	-	aceptada (excepcion: esquema de SurrealDB en ingles, ver ADR-012)
ADR-010	Proteccion de datos personales	Structured Logging (6.1)	aceptada
ADR-011	API v1 segun la convencion REST y documentada con OpenAPI	Request-Reply (2.3)	aceptada
ADR-012	SurrealDB como fuente de verdad, detras de un repositorio (esquema en ingles, instancia remota)	Repository	aceptada (PostgreSQL evaluado y descartado)
ADR-013	Arquitectura hexagonal (puertos y adaptadores) dentro del monolito modular	Puertos y adaptadores	aceptada

ADR	Decision	Patron del catalogo	Estado
ADR-014	Voz en Expo Go: grabacion en el celular, Whisper en el servidor y entonacion por texto y voz	-	aceptada
ADR-015	Voz natural con emocion en el servidor (puerto Sintetizador)	Cadena de respaldo	aceptada
ADR-016	VoiceStudio local como primer motor de voz con emocion	-	revertida (05/11/2026), reemplazada por ADR-019
ADR-017	Piramide de pruebas en tres niveles, todos en el CI	-	aceptada
ADR-018	Escaneo de seguridad con Trivy en el CI	-	aceptada
ADR-019	ElevenLabs (nube) como motor de voz; se revierte VoiceStudio	Cadena de respaldo	aceptada
ADR-020	Claude Sonnet en Google Vertex AI como motor de traduccion principal	Circuit Breaker	aceptada

Patrones del catalogo que **no** se usan y por que: Database per Service, API Gateway y BFF (un solo cliente y un solo backend; el catalogo lo indica asi para el MVP), Strangler Fig (no hay sistema legado), CQRS y Event Sourcing (el historial es un CRUD simple; no hay requisito de auditoria completa). Se revisaran si crece el volumen o aparecen nuevos clientes.

ADR-001: App movil con Expo (React Native) y NativeWind

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-07

Contexto

Dos personas desarrollan; la app debe servir en iOS y Android; el template de interfaz elegido (Luna) esta hecho en Expo.

Decision

Expo con React Native, expo-router para la navegacion y NativeWind (Tailwind) para los estilos. Se empezo con el SDK 54 y el 15/10/2026 se paso al SDK 57 para que la app abra en la version actual de Expo Go (PR #31). La misma base de codigo corre en web, lo que permite probar sin celular.

Alternativas (comparadas al justificar)

- **Flutter:** una sola base de codigo tambien, pero obliga a aprender Dart y no aprovecha el template ya comprado.
- **Nativo (Swift y Kotlin):** mejor acceso al dispositivo, pero son dos desarrollos para un equipo de dos.
- **Web responsiva (PWA):** mas simple, pero el microfono, la voz y la experiencia movil son mas limitados.

Consecuencias

Se prueba en Expo Go sin compilar. El reconocimiento de voz nativo y las notificaciones pueden requerir un "development build" mas adelante.

ADR-002: Backend monolitico en Node.js con Express, API REST sincrona

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-07, DRV-08 · **Patron:** Request-Reply

Contexto

Un solo cliente (la app) y un equipo pequeno. La traduccion es "bajo demanda con respuesta inmediata esperada por el usuario", el caso de uso que el catalogo asigna a Request-Reply.

Decision

Monolito modular en Node.js 22 con Express: rutas por recurso y reglas separadas de HTTP (desde ADR-013, en capas hexagonales: `src/dominio`, `src/aplicacion`, `src/adaptadores`), API REST sincrona. Mismo lenguaje (JavaScript/TypeScript) que la app.

Alternativas (comparadas al justificar)

- **Microservicios** (traduccion aparte): el catalogo los reserva para despliegue y escalado independientes; aqui solo agregan complejidad.
- **WebSocket o streaming:** el catalogo lo menciona para tiempo real estricto; como la traduccion es por turno (documento 1, Vision y contexto), REST basta. Si se pasa a traduccion simultanea, se revisa.
- **Python (FastAPI):** buena opcion para IA, pero agrega otro lenguaje; la IA se consume por HTTP (Ollama, Anthropic), no se entrena aqui.
- **Java (Spring Boot):** robusto, pero mucho mas pesado para este alcance.

Consecuencias

Un solo lenguaje en todo el proyecto. Si crece, el modulo de traduccion se puede separar sin cambiar el contrato de la API.

ADR-003: SQLite como base de datos inicial

Fecha: 2026-10-14 · **Estado:** reemplazada por ADR-012 el 2026-10-15 (SQLite sigue como motor de las pruebas sin servidor) · **Responde a:** DRV-04, DRV-08

Contexto

Hace falta persistir usuarios, conversaciones y mensajes con integridad referencial, y que cualquiera pueda correr el proyecto sin instalar un servidor de base de datos.

Decision

SQLite con `node:sqlite` (incluido en Node 22) y solo consultas parametrizadas. El esquema sigue el modelo de dominio (`backend/src/adaptadores/persistencia/sqlite/esquema.js`). Las pruebas usan base en memoria.

Alternativas (comparadas al justificar)

- **PostgreSQL:** mejor para produccion y concurrencia; es el siguiente paso (F10).
- **MySQL:** similar a PostgreSQL, sin ventaja para este caso.
- **MongoDB:** no encaja con un modelo tan relacional (combinaciones unicas, relaciones muchos a muchos).
- **ORM (Prisma, Sequelize):** util con muchas entidades; con 11 tablas y consultas simples, las consultas parametrizadas cumplen la regla de seguridad sin otra dependencia.

Consecuencias

Instalacion cero. Para produccion hay que migrar a PostgreSQL; el SQL es estandar, asi que el cambio es acotado. `node:sqlite` aun es experimental en Node 22.

ADR-004: Traduccion con Qwen local, Claude Sonnet de respaldo y motor de reglas

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-01, DRV-02, DRV-05, DRV-06, DRV-08 · **Patron:** Circuit Breaker con degradacion aceptable

Contexto

El valor de TalkLate es traducir conservando tono, intencion y modismos; un traductor literal no lo logra. El equipo decidio usar **Qwen corriendo local** por ahora y volver a **Claude Sonnet** si no funciona.

Decision

Una fachada (`backend/src/aplicacion/traduccion.js`) prueba los motores en orden: **Qwen** servido por Ollama en el servidor de TalkLate, luego **Claude Sonnet** (API de Anthropic, si hay clave) y por ultimo un **motor de reglas** espanol/ingles con modismos y variantes por pais. Los dos modelos reciben el mismo prompt (equivalente cultural segun la region destino; devuelven tono, region de origen y nota cultural). Es el patron **Circuit Breaker** del catalogo: si un motor falla, se "abre" durante un minuto (no se le vuelve a llamar) y el sistema **degrada** al siguiente; el catalogo menciona justamente "traduccion literal como degradacion aceptable", que es el papel del motor de reglas. No se usa Retry con backoff dentro de una misma peticion porque el usuario esta esperando: se prefiere pasar de inmediato al siguiente motor.

Alternativas (comparadas al justificar)

- **DeepL o Google Translate:** buena calidad literal, pero no explican ni adaptan modismos por region.
- **Solo Claude:** mejor calidad esperada, pero costo por mensaje, dependencia de internet y los mensajes salen a un tercero.
- **Solo motor de reglas:** gratis y rapido, pero no escala a mas idiomas ni a jerga nueva.
- **Otros modelos abiertos de traduccion (NLLB, Argos):** traducen frases, pero no siguen instrucciones de tono ni explican modismos.

Consecuencias

Qwen local no cuesta por mensaje y no envia datos a terceros (DRV-03, DRV-06), pero necesita un servidor con Ollama y el modelo descargado; su velocidad depende de ese equipo. Qwen corre en el **servidor**, no en el celular. La medicion real de tiempo y calidad se hace con `npm run medir` (DRV-01, DRV-02).

ADR-005: Voz: dictado en el cliente y expo-speech para la salida

Fecha: 2026-10-14 · **Estado:** reemplazada por ADR-014 (entrada) y ADR-015 (salida); `expo-speech` sigue como voz de respaldo · **Responde a:** DRV-07

Contexto

El usuario habla y escucha la traduccion. Transcribir audio en el servidor exige otro servicio y subir audio (dato personal).

Decision

El dictado se hace en el cliente (reconocimiento de voz del navegador en web; teclado del sistema en celular) y llega como texto con `canal: "voz"`. La traduccion se escucha con `expo-speech`, ajustando tono y velocidad segun la entonacion detectada.

Alternativas (comparadas al justificar)

- **Whisper u otro servicio de transcripcion en el servidor:** mas preciso y uniforme, y permitiria analizar el acento por el sonido; pero sube audio y requiere mas infraestructura.
- **Reconocimiento nativo dentro de la app:** requiere un development build de Expo.

Consecuencias

Sin costos extra y sin subir audio. En la practica el dictado del teclado no funciona de forma uniforme en Expo Go y no permite medir la voz; por eso ADR-014 lo reemplaza con grabacion en el celular y Whisper en el servidor.

ADR-006: Autenticacion con JWT de acceso corto y token de refresco rotativo

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-03 · **Patron:** Token-Based Authentication

Contexto

Usuario-RN-5 y RN-10, y la convencion de seguridad del proyecto: "tokens con expiracion (access corto, refresh mas largo)". El catalogo advierte que los JWT "no son revocables sin infraestructura adicional".

Decision

Token de acceso JWT de 15 minutos y token de refresco de 30 dias guardado solo como hash, que se rota en cada uso. Esa es la "infraestructura adicional": al cerrar sesion se revocan el token de acceso (lista de revocados) y el de refresco; al cambiar la contrasena se cierran las demas sesiones. Contrasenas con bcrypt, limite de intentos y codigo de recuperacion de un solo uso.

Alternativas (comparadas al justificar)

- **Sesiones en servidor:** revocacion trivial, pero guardan estado por sesion; JWT es lo que el catalogo recomienda para una app movil.
- **Proveedor externo (Firebase Auth, Auth0):** menos codigo propio, pero depende de un tercero y saca datos personales del sistema.

Consecuencias

La app renueva la sesion sola. El codigo de recuperacion sale por el puerto `Notificador`: en desarrollo se devuelve en la respuesta y en produccion lo envia un proveedor de correo conectado al mismo puerto.

ADR-007: Interfaz basada en el template Luna

Fecha: 2026-10-14 · **Estado:** aceptada

Contexto

El equipo adquirio el template Luna (chatbot) para no disenar la interfaz desde cero.

Decision

Se conservo su sistema de componentes y tema (claro y oscuro) y se reemplazaron sus vistas por las de TalkLate; se retiraron componentes y dependencias sin uso. El template original queda en `componentes/template/native-luna`.

Consecuencias

Interfaz consistente y rapida de construir. El mapeo etapa del journey -> pantalla esta en *documento 13 (Especificacion de vistas y UI)*.

ADR-008: Ajustes al modelo de dominio al implementarlo

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-04

Contexto

Al implementar aparecieron diferencias con el modelo de dominio enriquecido.

Decision

1. **Lenguaje - Region muchos a muchos** (`LenguajeRegion`): Conversacion-RN-1 pide validar que la region sea valida para el idioma, y un idioma se habla en varias regiones. Asi el nombre del lenguaje puede seguir siendo unico.
2. **Tablas de soporte** (`reporte`, `token_refresco`, `sesion_revocada`, `recuperacion`): no son objetos del modelo de dominio (los reportes son dinamicos y no tienen estructura fija), pero el sistema las necesita para funcionar.
3. **Usuario** gana `contrasena` (hash), `rol` y `consentimientoDatos`; **MensajeDestino.mensajeOriginal** pasa a ser referencia.

Consecuencias

documento 12 (Modelo de datos) refleja estos cambios y su correspondencia con la base de datos.

ADR-009: Identificadores del dominio en espanol (lenguaje ubicuo)

Fecha: 2026-10-14 · **Estado:** aceptada

Contexto

La regla transversal 1 de F0 pedia identificadores en ingles, pero todo el descubrimiento (modelo de dominio, reglas de negocio, historias) esta en espanol: Usuario, Conversacion, MensajeOriginal, Conversacion-RN-2...

Decision

Usar en el codigo los mismos nombres del modelo de dominio, en espanol (lenguaje ubicuo de Domain-Driven Design): tablas `conversacion`, `mensaje_original`; funciones `traducir`, `validarNombreConversacion`. Los terminos tecnicos y de librerias quedan en ingles. La regla de nombres de las convenciones del proyecto quedo actualizada.

Alternativas (comparadas al justificar)

- **Todo en ingles:** es la practica comun, pero obliga a traducir cada concepto y rompe la trazabilidad con el modelo de dominio y las reglas de negocio.

Consecuencias

Trazabilidad directa: una regla como Conversacion-RN-2 se encuentra en el codigo con el mismo nombre. Mezcla de idiomas en el codigo (dominio en espanol, framework en ingles). Excepcion (2026-10-15): el esquema de SurrealDB usa nombres en ingles para alinearse con el modelo de dominio en ingles; la traduccion de nombres queda encerrada en el adaptador (ver ADR-012).

ADR-010: Proteccion de datos personales

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-03 · **Patron:** Structured Logging

Contexto

TalkLate guarda conversaciones personales (PII segun la convencion de seguridad del proyecto) y opera en Colombia (Ley 1581 de 2012).

Decision

- Matriz de sensibilidad de datos (documento 16).
- Consentimiento obligatorio al registrarse y politica de tratamiento de datos.
- Eliminar la cuenta con todos sus datos (derecho al olvido).
- Contenido de mensajes **cifrado en reposo** con AES-256-GCM (clave por variable de entorno).
- Logs JSON que nunca guardan correo, contrasena, tokens, codigos ni contenido (patron Structured Logging, obligatorio en el catalogo).

- CORS con origenes explicitos y limites de uso, en especial sobre la traduccion.

Alternativas (comparadas al justificar)

- **Cifrar toda la base** (SQLCipher): protege todo, pero agrega una dependencia nativa; se deja para produccion junto con el cifrado del proveedor.

Consecuencias

Busquedas por contenido de mensajes no son posibles en la base (hoy no se necesitan). Sin la clave no se pueden leer los mensajes.

ADR-011: API v1 segun la convencion REST y documentada con OpenAPI

Fecha: 2026-10-14 · **Estado:** aceptada · **Patron:** Request-Reply

Contexto

La convencion REST del proyecto define versionado, formato de error, codigos, paginacion, health check y OpenAPI.

Decision

Base `/api/v1`; errores `{ statusCode, error, message, timestamp, path }`; `201` con `Location`; `204` al eliminar; `422` para reglas de negocio; listas `{ data, pagination }`; `GET /health`; contrato en `backend/openapi.yaml` con Swagger UI en `/api/v1/docs`. Los campos de las respuestas usan los nombres del dominio en snake_case (por ejemplo `fecha_inicio`) y `id`, para coincidir con las columnas (ADR-009); el ejemplo `identificador`/`fechaCreacion` de la convencion se toma como ilustrativo.

Consecuencias

Contrato claro para la app y para pruebas; cambios incompatibles van en `/api/v2`.

ADR-012: SurrealDB como fuente de verdad, detras de un repositorio

Fecha: 2026-10-15 · **Estado:** aceptada (decision cerrada el 2026-10-23) · **Reemplaza:** ADR-003 · **Responde a:** DRV-01, DRV-03, DRV-04, DRV-08

Contexto

ADR-003 dejo SQLite como base inicial y anoto que para produccion habia que pasar a un servidor de base de datos. Julian construyo el cambio (PR #29): el modelo de dominio tiene muchas relaciones (conversacion, lenguaje por conversacion, mensaje original y destino, reporte) y combinaciones unicas que se querian expresar directo en la base.

Decision

- **SurrealDB 3.x** es la fuente de verdad del backend. El esquema esta en `backend/db/surreal/schema.surql` (SCHEMAFULL) y se aplica al arrancar.

- Las relaciones son enlaces nativos (`record<...>`); las combinaciones unicas son indices `UNIQUE`; los valores permitidos (rol, estado, remitente) son `ASSERT` en el campo.
- El acceso a datos pasa por un **repositorio** con la misma interfaz para dos motores: `src/adaptadores/persistencia/surreal/repositorio.js` (ejecucion) y `src/adaptadores/persistencia/sqlite/repositorio.js` (pruebas sin servidor). Las rutas no conocen el motor; `DB_DRIVER` elige cual usar.
- La conexion es por HTTP (`/sql`) sin dependencias nuevas. Se entra una vez con `/signin` y se reutiliza el token; los valores se escriben como literales serializados (`lit`, `rid`), nunca concatenando texto del usuario.

Alternativas (comparadas al justificar)

- **Seguir con SQLite:** instalacion cero, pero es un archivo local; para varios usuarios a la vez y para produccion habia que cambiarla igual.
- **PostgreSQL:** muy maduro y era el paso previsto en ADR-003; las relaciones se modelan con llaves foraneas y JOIN. Se eligio SurrealDB porque los enlaces entre registros siguen el modelo de dominio sin tablas intermedias de mas y el esquema con ASSERT deja reglas del dominio en la base.
- **SDK oficial de SurrealDB:** mas comodo, pero agrega una dependencia; el endpoint HTTP basta para estas consultas.

Consecuencias

- Hace falta un servidor SurrealDB 3.x: el remoto para usar la app, o uno local para las pruebas (`backend/README-surreal.md`). Con la 2.x el esquema no carga; el backend lo revisa al arrancar.
- Mandar usuario y clave en cada consulta costaba unos 35 ms por consulta (unos 420 ms por mensaje); con el token de sesion bajo a unos 28 ms por mensaje. Sigue por encima de SQLite (unos 7 ms), pero muy lejos del limite de 3 s de DRV-01.
- La renovacion de la sesion revoca el token de refresco en un solo paso (`tokens.consumir`), porque con un servidor dos pedidos pueden llegar al mismo tiempo.
- CI corre la suite de la API contra los dos motores (job `backend-surreal`).
- En produccion son obligatorias `SURREAL_USER` y `SURREAL_PASS` (no se acepta root/root).
- Algunas consultas filtran recorriendo enlaces (por ejemplo, los mensajes de una conversacion); al crecer el volumen se agregan indices sobre esos campos.

Evaluacion frente a PostgreSQL

Se comparo SurrealDB con PostgreSQL contra lo que pide TalkLate y se eligio **SurrealDB**. Las razones:

1. **El dominio es una cadena de relaciones y SurrealDB las guarda como enlaces nativos.** Conversacion -> idioma de la conversacion -> mensaje original -> mensaje traducido -> reporte. Cada campo `record<...>` apunta directo al registro relacionado, y una consulta lo recorre sin JOIN. Ejemplo real del adaptador: para saber si una traduccion es del usuario se escribe `mensaje_original.lenguaje_conversacion.conversacion.usuario = $usuario` en una sola expresion (REQ-CONV-007, REQ-REP-001).

2. **Las reglas del dominio tambien viven en la base (defensa en profundidad).** El esquema es SCHEMAFULL: las combinaciones unicas del modelo de datos son indices `UNIQUE` (correo, nombre y codigo de idioma, region dentro del pais, un reporte por traduccion y usuario); los valores permitidos son `ASSERT` (rol, estado del reporte, remitente, estado de entrega); y hay validaciones de formato y longitud (correo con `string::is_email`, nombre de conversacion de 3 a 80 caracteres). Si un error en la API dejara pasar un dato invalido, la base lo rechaza.
3. **Es multimodelo, y eso encaja con lo que viene sin agregar otra base.** Los reportes son dinamicos (sin estructura fija): SurrealDB admite campos flexibles junto a tablas estrictas. La evolucion prevista, usar los reportes y un glosario de modismos para mejorar la traduccion, puede apoyarse en sus indices vectoriales (HNSW) y en relaciones de grafo (`RELATE`) para equivalencias entre regiones, todo en la misma base.
4. **Gestionada y con control de acceso fino.** SurrealDB Cloud evita instalar y operar un servidor de base de datos, algo importante para un equipo de dos personas. SurrealDB permite crear usuarios con permisos limitados a un namespace o a una base; el backend usa un usuario de base propio, limitado a `talklate/app`, y no el root (minimo privilegio, ADR-010).
5. **Ya esta construida y probada.** Funciona contra una instancia real y CI corre la suite de la API contra SurrealDB en cada cambio.

Lo que se acepta a cambio (y como se maneja):

- Es mas joven que PostgreSQL y su comunidad es menor. Se usa la version 3.x fija en CI, y el acceso a datos queda aislado detras del puerto `Repositorio` (ADR-013): si algun dia hubiera que cambiar de base, seria un adaptador nuevo y no una reescritura.
- No se usan consultas preparadas como en SQL: el adaptador nunca concatena texto del usuario; todo valor pasa por `lit`/`rid` y hay pruebas de inyeccion (comillas, saltos de linea, `"; DELETE ...`).
- Las consultas que recorren enlaces no usan indices: se revisan con mas datos y se agregan indices donde haga falta.

Sintesis: "Elegimos SurrealDB porque nuestro dominio es una cadena de relaciones (conversacion, mensajes, traducciones, reportes) que guarda como enlaces nativos, porque nos deja poner las reglas del dominio en el propio esquema y porque, al ser multimodelo, soporta lo que viene (reportes flexibles, busqueda vectorial de modismos) sin agregar otra base. Su riesgo, ser una tecnologia mas joven, lo controlamos con la arquitectura hexagonal: la base esta detras de un puerto y se puede cambiar sin tocar las reglas."

Esquema e instancia

- **Esquema en ingles:** las tablas y campos de SurrealDB estan en ingles (`user`, `conversation`, `original_message`, `translated_message`...), alineados con el modelo de dominio en ingles. Es una excepcion a ADR-009 solo para la base: la API, el codigo y los valores de los enumerados (`origen`/`destino`, `abierto`...) siguen en espanol, y el adaptador (`src/adaptadores/persistencia/surreal/repositorio.js`) traduce los nombres en cada consulta.
- **Instancia remota:** la app usa por defecto una instancia de SurrealDB Cloud; las credenciales van en `backend/.env` (fuera del repo). Las pruebas y CI usan un SurrealDB local para no crear ni borrar datos en el remoto.

ADR-013: Arquitectura hexagonal (puertos y adaptadores) dentro del monolito modular

Fecha: 2026-10-23 · **Estado:** aceptada · **Complementa:** ADR-002, ADR-004, ADR-012 · **Responde a:** DRV-04, DRV-05, DRV-08

Contexto

Una revision de la arquitectura confirmo que el monolito modular es la decision correcta (ADR-002), pero mostro que su modularidad no era explicita: las rutas HTTP mezclaban reglas de negocio, acceso a datos y cifrado, y la fachada de traduccion conocia cada motor por su nombre. La pregunta "y si manana crece?" debe poder responderse sin partir el sistema en microservicios.

Decision

Organizar la API en capas de **puertos y adaptadores** (Alistair Cockburn), sin cambiar su contrato:

- **Dominio** (`src/dominio`): validaciones, errores de negocio, analisis de texto, prosodia de la voz y expresion de cada tono. No depende de nada externo.
- **Aplicacion** (`src/aplicacion`): un caso de uso por epica (cuentas, sesiones, conversaciones, traduccion, voz, catalogo, reportes). Solo importa el dominio y los puertos.
- **Puertos** (`src/puertos`): contratos `Repositorio`, `MotorTraduccion`, `Transcriptor`, `Sintetizador`, `Notificador`, `Cifrador`, `Hasheador` y `FirmadorTokens`, verificados al arrancar.
- **Adaptadores** (`src/adaptadores`): de entrada, HTTP con Express; de salida, SurrealDB y SQLite, Claude (Vertex), Qwen y reglas, Whisper, ElevenLabs, Azure y OpenAI, detector de genero, correo y AES-GCM.
- **Composicion** (`src/composicion.js`): unico lugar que conoce los adaptadores concretos; los elige segun el entorno y arma los casos de uso.

El subsistema de traduccion queda detras del puerto `MotorTraduccion`: el caso de uso aplica el Circuit Breaker (ADR-004) sobre cualquier lista de motores, sin saber cuales son.

Alternativas (comparadas al justificar)

- **Dejar el monolito como estaba:** funciona, pero las reglas viven en las rutas y cambiar un motor o la base obliga a tocar varios archivos.
- **Microservicios (traduccion aparte):** limites claros, pero agrega red, despliegue y fallas distribuidas que un equipo de dos no necesita (ADR-002).
- **Capas clasicas (controlador, servicio, repositorio) sin puertos:** separa responsabilidades, pero el servicio sigue dependiendo de implementaciones concretas.
- **Clean Architecture completa con un framework de inyeccion de dependencias:** mas ceremonia y dependencias para el mismo resultado; aqui la composicion manual basta.

Consecuencias

- Limites claros sin el costo distribuido: cambiar de base de datos (por ejemplo a PostgreSQL) o agregar un motor es escribir un adaptador nuevo; los casos de uso no cambian.
- El caso de uso de traduccion y el Circuit Breaker se prueban con motores falsos, sin red ni HTTP (`test/motores.test.js`).
- Un adaptador incompleto falla al arrancar (`asegurarPuerto`), no a mitad de una peticion.
- Mas archivos y una indireccion mas. El contrato de la API, la base y las pruebas de punta a punta no cambiaron (las mismas 16 pruebas de la API pasan con SQLite y con SurrealDB).
- Los nombres siguen ADR-009 (dominio en espanol).
- Las reglas de dependencia entre capas se verifican con dependency-cruiser (`backend/.dependency-cruiser.cjs`, `npm run arch:check`): dominio puro, aplicacion sin adaptadores, puertos como contratos y adaptadores que no se cruzan entre categorias. El chequeo corre en el CI en cada pull request y hoy no marca ninguna violacion: el hash de contrasenas (bcrypt) y los tokens (JWT) tambien entran por puertos (`Hasheador` y `FirmadorTokens`).

ADR-014: Voz en Expo Go con grabacion en el celular, Whisper en el servidor y entonacion por texto y voz

Estado: Aceptada (23/10/2026). Reemplaza la parte de ADR-005 que usaba el dictado del teclado.

Contexto: Expo Go no trae reconocimiento de voz nativo, y el dictado del teclado no funciona de forma uniforme ni permite medir la voz. Ademas, la traduccion debe sonar con la misma entonacion de quien habla.

Decision:

- La app graba con `expo-audio` (funciona en Expo Go) y mide el volumen cada 100 ms.
- El audio va en base64 a `POST /api/v1/conversaciones/:id/mensajes/voz`. El nuevo puerto `Transcriptor` lo pasa a texto con un adaptador Whisper (API compatible con OpenAI `/v1/audio/transcriptions`: faster-whisper/speaches o whisper.cpp locales). Variables: `TRANSCRIPCION_URL`, `TRANSCRIPCION_MODELO`.
- `dominio/prosodia.js` saca el tono de la voz (volumen, variacion y palabras por segundo). El tono del texto manda salvo que sea Neutral; la descripcion de la prosodia va al prompt del LLM y `entrega.ritmo_voz` ajusta la velocidad de la voz de salida.
- La interfaz tiene dos modos, como ChatGPT: la burbuja animada del template Luna (AiCircle) y el chat. El modo se recuerda en el celular.

Consecuencias: Se necesita un servidor Whisper (se recomienda el modelo `small`; `tiny` confunde jerga como "parce"). Sin `TRANSCRIPCION_URL` el endpoint de voz responde 503 y el texto sigue funcionando.

ADR-015: Voz natural con emocion en el servidor (puerto Sintetizador)

Estado: Aceptada (26/10/2026). Complementa ADR-014.

Contexto: La voz del celular (expo-speech) suena robotica y casi no cambia con la emocion; subir o bajar el tono no alcanza para que se note enojo o alegria.

Decision:

- Nuevo puerto `Sintetizador` con dos adaptadores, elegidos con `VOZ_MOTOR` (auto, azure, openai, ninguno):
  - **Azure Speech** (voces neuronales, capa gratis de 500 mil caracteres al mes). La emocion va en SSML con `mstts:express-as` (angry, cheerful, sad, friendly) mas prosodia de respaldo para voces sin estilos. La voz se elige por idioma y pais (ej. es-CO-SalomeNeural, en-US-JennyNeural).
  - **API compatible con OpenAI** (`/v1/audio/speech`). Con `gpt-4o-mini-tts` la emocion y el acento van como instruccion en lenguaje natural; tambien sirve un Kokoro-FastAPI local.
- `dominio/voz.js` define como suena cada tono (estilo, prosodia e instruccion); los adaptadores solo lo traducen a su API.
- `POST /api/v1/voz` devuelve una URL de mp3 que vence en 5 minutos (`GET /voz/audio/:id`, id aleatorio de 128 bits), para que el reproductor del celular la abra sin guardar archivos.
- Si no hay motor configurado o falla, la app vuelve a la voz del celular con tono y velocidad ajustados.

Consecuencias: La voz suena natural y con emocion cuando hay clave de Azure o de OpenAI; sin ellas todo sigue funcionando con la voz del celular. Los audios viven solo en memoria del servidor (no se guardan).

ADR-016: VoiceStudio local como primer motor de voz con emocion

Estado: Revertida (05/11/2026), reemplazada por ADR-019. Fue aceptada el 29/10/2026 y extendia ADR-015. Motivo: IndexTTS 2 no soporta Metal/MPS y en la Mac corre solo en CPU (unos 30 s por frase), inviable en una conversacion en vivo.

Contexto: Azure y OpenAI dan voz natural, pero cobran o piden clave y salen a internet. El proyecto ya traduce con Qwen local (ADR-004), y una voz local mantiene la misma linea: sin costo por uso y sin enviar la voz a terceros. VoiceStudio (github.com/debpalash/VoiceStudio) reune motores de voz locales; su motor IndexTTS 2 acepta la emocion como un vector de 8 pesos y habla espanol e ingles.

Decision:

- Nuevo adaptador `adaptadores/sintesis/voicestudio.js` del puerto `Sintetizador`.
  - Con IndexTTS 2 usa `POST /generate` con `emo_vector`. Con los otros motores usa `POST /v1/audio/speech` (compatible con OpenAI) y manda la emocion como instruccion en texto.
- Con `VOZ_MOTOR=auto` el orden es VoiceStudio, luego Azure, luego OpenAI, y al final la voz del celular.
- El equipo de despliegue de referencia es una Mac con chip M5 y 32 GB de RAM, con instalacion nativa (Metal/MLX). En un equipo Intel sin GPU NVIDIA, IndexTTS en CPU tarda decenas de segundos por frase; ahi el primer motor es Azure (`VOZ_MOTOR=azure`).
- La guia de montaje y medicion estaba en `backend/VOICESTUDIO_MAC.md` (archivo retirado con ADR-019); `npm run medir:voz` se conserva.

Consecuencias:

- Voz con emocion gratis y local, y la posibilidad de clonar una voz con permiso.
- La latencia se mide con `npm run medir:voz` en el equipo de despliegue (meta: mediana de 5 s o menos por frase, DRV-01).

- La licencia de VoiceStudio es AGPL-3.0 y cada modelo tiene la suya: es valida para el uso academico; para un uso comercial se usa Azure u OpenAI detras del mismo puerto.
- El adaptador queda cubierto por pruebas automaticas de las dos rutas (`/generate` y `/v1/audio/speech`), del vector de emocion y de los errores.

ADR-017: Piramide de pruebas en tres niveles, todos en el CI

Estado: aceptada (05/11/2026). **Responde a:** DRV-08, DRV-03.

Contexto: Las pruebas del backend estaban en cuatro archivos mezclados (reglas, adaptadores e integracion) y no habia pruebas de la app real en un navegador. No se veia que nivel cubria que, ni cuanto costaba correr cada uno.

Decision:

- Organizar las pruebas como piramide: **base** de unitarias (dominio, casos de uso y adaptadores con dobles; `backend/test/unitarias/` y `frontend/test/`), **medio** de integracion (API HTTP con base real; `backend/test/integracion/`, contra SQLite y SurrealDB) y **punta** de punta a punta (servidor real como proceso en `backend/test/e2e/` y app web con Playwright en `e2e/`).
- Un script por nivel (`test:unitarias`, `test:integracion`, `test:e2e`) y un paso por nivel en el CI; la app web corre en un job aparte (`e2e-web`) que solo arranca si los niveles de abajo pasaron.
- Cobertura minima de 80 % en unitarias + integracion, y `arch:check` (dependency-cruiser) en el CI. Para que pase, bcrypt y jsonwebtoken entran por los puertos `Hasheador` y `FirmadorTokens`.
- En las pruebas de punta a punta, los servicios de IA (Whisper y el motor de voz) se reemplazan por servidores HTTP falsos que hablan su mismo protocolo: se prueban los adaptadores reales sin depender de modelos pesados.

Alternativas:

- **Jest o Vitest en el backend:** mas funciones, pero `node:test` ya viene con Node 22, no agrega dependencias y tiene cobertura incluida.
- **Cypress para la web:** maduro, pero Playwright es mas liviano en CI, emula celulares y ya era la herramienta usada para revisar la app.
- **Detox o Maestro en el celular:** prueban la app nativa, pero exigen emuladores en CI; la app web comparte el mismo codigo (ADR-001) y cubre el recorrido.

Consecuencias: 78 unitarias (73 del backend y 5 del frontend), 29 de integracion (18 de la API contra SQLite y SurrealDB, 11 de problemas contra SQLite) y 4 de punta a punta; cobertura de lineas del backend de 96.5 %. Cada falla dice en que nivel esta el problema. Detalle en `calidad/piramide_de_pruebas.md`.

ADR-018: Escaneo de seguridad con Trivy en el CI

Estado: aceptada (05/11/2026). **Responde a:** DRV-03.

Contexto: TalkLate maneja datos personales (Ley 1581) y depende de cientos de paquetes de npm. Una dependencia vulnerable o una clave escrita en el codigo son riesgos que las pruebas funcionales no detectan.

Decision:

- Escanear el repositorio con Trivy: dependencias npm (de produccion y de desarrollo), secretos y configuraciones inseguras. Bloquea el merge si hay hallazgos HIGH o CRITICAL.
- Corre en cada pull request, en cada push a `develop` y `main` y cada lunes (los CVE nuevos aparecen aunque el codigo no cambie).
- Trivy se instala desde el binario oficial con version fija (0.69.3) y checksum SHA-256 verificado. No se usan `aquasecurity/trivy-action` ni `setup-trivy`, comprometidas en marzo de 2026.
- Las vulnerabilidades indirectas se corrigen con `overrides` de npm dentro de la misma version mayor. Lo que no tiene parche compatible va a `.trivyignore` con su razon, su riesgo real y una fecha de vencimiento.

Alternativas:

- **`npm audit`:** solo revisa npm y no busca secretos ni configuraciones; ademas no permite excepciones con vencimiento.
- **Dependabot o Snyk:** utiles, pero dependen de la configuracion de la organizacion en GitHub o de un servicio externo; Trivy corre igual en local y en CI.

Consecuencias: Se corrigieron 1 HIGH en el backend y 1 CRITICAL + 50 HIGH en el frontend (produccion y desarrollo). Queda 1 excepcion documentada (`decode-uri-component`, vence el 28/02/2027). Detalle en `calidad/seguridad_trivy.md`.

ADR-019: ElevenLabs (nube) como motor de voz; se revierte VoiceStudio

Estado: Aceptada (05/11/2026). Reemplaza ADR-016; extiende ADR-015.

Contexto: VoiceStudio con IndexTTS 2 (ADR-016) daba voz local con emocion, pero IndexTTS no soporta Metal/MPS: en la Mac (Apple Silicon) corre solo en CPU, ~30 s por frase, inviable para una conversacion en vivo. Los motores de VoiceStudio que si usan la GPU (omnivoice, mlx-audio) no hacen emocion real. Se necesita voz natural, con emocion, con acento regional (paisa, New York) y rapida.

Decision:

- Se elimina el adaptador VoiceStudio y se agrega `adaptadores/sintesis/elevenlabs.js` (puerto `Sintetizador`), que llama la API REST de ElevenLabs (`POST /v1/text-to-speech/{voice_id}`, modelo `eleven_multilingual_v2`) con `fetch` (sin SDK, igual que azure/openai).
- La emocion va en `voice_settings` (menos `stability` = mas expresivo; `style` exagera modismos) y la velocidad en `speed`, derivadas del tono y la prosodia que ya se detectaban (ADR-015).
- La voz se elige por locale y genero: es-CO = paisa (hombre/mujer), en-US = New York (hombre/mujer). El genero lo infiere el mismo modelo de traduccion local (campo `genero` en su JSON, sin llamada extra) y viaja por `entrega.genero` hasta el adaptador; si no hay pista usa `ELEVENLABS_GENERO` (default hombre).
- Config: `ELEVENLABS_API_KEY`, `ELEVENLABS_VOZ_PAISA_HOMBRE/MUJER`, `ELEVENLABS_VOZ_NY_HOMBRE/MUJER`, `ELEVENLABS_GENERO`, `ELEVENLABS_MODELO`. Con `VOZ_MOTOR=auto` el orden es ElevenLabs -> Azure -> OpenAI -> voz del celular.

Consecuencias:

- Voz natural, con emocion y acento regional, y rapida (1-5 s por frase).
- Es un servicio en la nube: cambia el enfoque local-first de ADR-015/016 (el texto traducido sale a ElevenLabs). Las voces de la Voice Library requieren plan pago para usarse via API (el plan free las bloquea).
- La deteccion de genero por texto es imperfecta, sobre todo en ingles; cuando no hay pista usa el genero por defecto. Detectar el genero por el tono de voz (pitch) o persistirlo por mensaje quedan como mejoras futuras.
- Los limites de creditos, el uso comercial y la atribucion dependen del plan de ElevenLabs contratado.

ADR-020: Claude Sonnet en Google Vertex AI como motor de traduccion principal

Estado: aceptada (12/11/2026). Ajusta ADR-004. **Responde a:** DRV-02, DRV-05. **Riesgos nuevos:** DRV-03, DRV-06.

Contexto: ADR-004 dejo prevista esta salida: se empezaba con Qwen local y se volveria a Claude Sonnet si no funcionaba. La prioridad del producto es la calidad cultural de la traduccion (DRV-02). El equipo adopto Claude Sonnet como motor principal, servido por Google Vertex AI (pull request 53).

Decision:

- Nuevo adaptador `adaptadores/traduccion/vertex.js` del puerto `MotorTraduccion`, con el SDK oficial de Anthropic para Vertex. Se autentica con las credenciales de Google (ADC), no con una clave de API en el codigo. Variables: `ANTHROPIC_USE_VERTEX`, `ANTHROPIC_VERTEX_PROJECT_ID` y `TALKLATE_VERTEX_REGION` (por defecto `global`: Claude no se sirve en todas las regiones). Modelo por defecto del adaptador: `claude-sonnet-4-6`, configurable con `TALKLATE_MODEL`.
- La configuracion del equipo (`run-talklate.sh`) arranca con `TRADUCTOR=claude`: Claude (Vertex) -> motor de reglas. Qwen (Ollama) sigue disponible como adaptador (`TRADUCTOR=auto` o `qwen`), pero no se usa.
- El prompt pasa a **transcreacion cultural**: primero entiende la intencion, la situacion y el tono, y luego lo reexpresa como lo diria un nativo de la region destino; incluye ejemplos reales del glosario (few-shot) para los modismos detectados.
- El adaptador tiene tiempo limite de 30 s y desactiva los reintentos del SDK: si falla, el Circuit Breaker pasa al siguiente motor (ADR-004).
- **El motor de reglas deja de traducir sin modismos conocidos:** si el texto no contiene ninguna frase del diccionario, falla con un error claro en vez de entregar una traduccion palabra por palabra.

Alternativas:

- **Seguir solo con Qwen local:** sin costo por mensaje y sin datos a terceros, pero con la capacidad de un modelo de 7B parametros.
- **API directa de Anthropic:** el adaptador `claude.js` sigue disponible (necesita `ANTHROPIC_API_KEY`); Vertex evita guardar esa clave y usa las credenciales de Google.

Consecuencias:

- **Privacidad (DRV-03):** el texto de cada mensaje sale del servidor hacia Google Cloud para que Claude lo procese. Ya no es cierto que la traduccion no envie datos a terceros; la politica de tratamiento de datos y la matriz de sensibilidad se actualizan.
- **Costo (DRV-06):** cada traduccion tiene costo por uso y hace falta un proyecto de Google Cloud con facturacion. Se mantienen los limites por usuario; falta un tope de gasto diario.
- **Continuidad (DRV-05):** el respaldo es parcial. Si Claude falla, el motor de reglas solo traduce los modismos que conoce; en los demas casos la persona recibe un mensaje de error y no una traduccion literal de mala calidad.
- **Dependencia de la nube:** sin internet o sin credenciales de Google no hay traduccion general.
- **Pendiente:** medir calidad y tiempo (DRV-01, DRV-02) con este motor.

12. Modelo de datos

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026). Fuente: modelo de dominio enriquecido del proyecto.

Cada entidad documenta sus atributos, sus reglas de negocio y sus **combinaciones unicas** (restricciones de unicidad simples o compuestas). Las combinaciones unicas salen de las reglas de negocio de cada entidad y estan implementadas como indices `UNIQUE` en el esquema de SurrealDB (`backend/db/surreal/schema.surql`). La correspondencia con la base de datos esta al final.

Diagrama editable de referencia: `diagramas/modelo_dominio.drawio`.

Objetos de dominio

Objeto de dominio	Descripcion
Usuario	Entidad que define a un usuario registrado en la aplicacion, guardando de el sus datos personales y de interes.
Conversacion	Entidad que representa una sesion de comunicacion entre usuarios, conteniendo multiples mensajes intercambiados a traves de un canal especifico.
Lenguaje	Entidad que representa un idioma disponible en el sistema para traduccion y comunicacion.
LenguajePorConversacion	Entidad de relacion que almacena las configuraciones de idioma especificas para cada conversacion individual.
MensajeOriginal	Entidad que almacena el contenido original del mensaje antes de cualquier procesamiento o traduccion.
MensajeDestino	Entidad que almacena el mensaje procesado/traducido listo para ser entregado al destinatario en su idioma preferido.
Canal	Entidad que define el medio a traves del cual se transmiten los mensajes.
Entonacion	Entidad que define el tono o estilo comunicativo a aplicar en los mensajes (formal, informal, tecnico, etc.).
Region	Entidad que representa una region geografica que puede influir en variaciones dialectales y culturales del lenguaje.
Pais	Entidad que representa un pais especifico, utilizado para contexto geografico y configuraciones regionales.

Usuario

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Simple	correo	Usuario-RN-1: no puede haber dos cuentas con el mismo correo.

Atributo	Tipo	Longitud	Obligatorio	Sensible	Descripcion
identificador	Alfanumerico (UUID)	36	Si	No	Identifica de manera unica a un usuario. Autogenerado.
nombre	Alfanumerico	1–50	Si	No	Nombre del usuario.
apellido	Alfanumerico	1–50	Si	No	Apellido del usuario.
correo	Alfanumerico	5–320	Si	**Si**	Correo electronico, usado para contacto privado y seguridad.
fechaNacimiento	Fecha	—	Si	**Si**	Fecha de nacimiento; no puede ser mayor a la actual.
contrasena	Alfanumerico (hash bcrypt)	60	Si	**Si**	Solo se guarda el hash (Usuario-RN-4); nunca se devuelve ni se registra.
rol	Alfanumerico	5–7	Si	No	`cliente` (por defecto) o `admin`. Lo asigna el sistema.
consentimientoDatos	Fecha y tiempo	—	Si	**Si**	Cuando acepto la politica de tratamiento de datos (Ley 1581 de 2012).

Responsabilidades y reglas de negocio

Responsabilidad	Entrada	Salida	Regla	Descripcion	Excepcion	Respuesta al usuario
Registrar Usuario	identificador, nombre, apellido, correo, fecha de nacimiento	estado, mensaje	Usuario-RN-1	Verificar que el correo no este asociado previamente a una cuenta activa.	Ya existe un usuario con el mismo correo	"El correo electronico ingresado ya esta registrado en TalkLate. Por favor inicie sesion."
Registrar Usuario			Usuario-RN-2	La edad calculada desde la fecha de nacimiento debe ser ≥ 13 anos.	Usuario menor de 13 anos	"Lo sentimos, debes ser mayor de 13 anos para crear una cuenta."
Registrar Usuario			Usuario-RN-3	El correo debe cumplir el formato estandar (usuario@dominio.com).	Formato invalido	"El formato del correo es invalido. Por favor reviselo."
Registrar Usuario			Usuario-RN-4	La contrasena debe tener minimo 8 caracteres, incluyendo mayuscula, numero y caracter especial.	Contrasena debil	"La contrasena es debil. Debe tener al menos 8 caracteres, numeros y simbolos."
Iniciar sesion	correo, contrasena	estado, mensaje, token	Usuario-RN-5	Verificar que el correo exista y la contrasena coincida.	Credenciales incorrectas	"Usuario o contrasena incorrectos. Por favor verifique sus credenciales."
Solicitar Recuperacion de contrasena	correo	estado, mensaje	Usuario-RN-6	Buscar cuenta activa vinculada al correo.	Correo no registrado	"No encontramos una cuenta asociada a este correo electronico."
Solicitar Recuperacion de contrasena			Usuario-RN-7	Verificar envio exitoso del correo con el codigo.	Fallo en servicio de mensajeria	"Ocurrio un error al enviar el codigo. Por favor intente nuevamente mas tarde."
Restablecer Contrasena	correo, codigo, nueva contrasena	estado, mensaje	Usuario-RN-8	El codigo debe coincidir con el generado y no haber expirado (ej. 15 min).	Codigo incorrecto/expirado	"El codigo de verificacion es invalido o ha expirado. Solicite uno nuevo."
Restablecer Contrasena			Usuario-RN-9	La nueva contrasena debe cumplir las mismas politicas que Usuario-RN-4.	Contrasena no valida	"La nueva contrasena no es segura. Use al menos 8 caracteres, numeros y simbolos."

Responsabilidad	Entrada	Salida	Regla	Descripcion	Excepcion	Respuesta al usuario
Cerrar Sesion	—	estado, mensaje	Usuario-RN-10	El token de autenticacion debe ser valido, no expirado y corresponder a una sesion activa.	Sesion inactiva	"La sesion ya ha expirado o no es valida."

Datos simulados (muestra)

Identificador	Nombre	Apellido	Correo	Fecha de nacimiento
f256dadb-...	Julian David	Diaz Medina	diazjulian460@gmail.com	2002-04-16
d72f77e6-...	Diego Alejandro	Franco Atehortua	dafa123.daf@gmail.com	2002-04-29

Conversacion

Combinaciones unicas: ninguna. El nombre tiene valor por defecto "Nueva Conversacion", asi que se repite. El identificador no cuenta porque ya es unico.

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	3–80	Si	Default "Nueva Conversacion"; no puede contener caracteres especiales o palabras ofensivas (Conversacion-RN-4). No se puede modificar el identificador (Conversacion-RN-5).
fecha y hora de inicio	Fecha y tiempo	—	Si	Autogenerado.
usuario	Referencia a Usuario	—	Si	Usuario que crea la conversacion.
lenguajePorConversacion	Referencia a LenguajePorConversacion	—	Si	Configuracion de idiomas de la conversacion.

Reglas de negocio

Regla	Descripcion
Conversacion-RN-1	Una conversacion debe tener definido un idioma destino antes de iniciar la traduccion; la Region seleccionada debe ser valida para el Lenguaje elegido.
Conversacion-RN-2	El idioma origen se detecta automaticamente con el primer mensaje (de voz o de texto) y no puede cambiar durante esa conversacion.
Conversacion-RN-3	Los pares de idiomas origen-destino deben estar soportados por el sistema antes de iniciar la conversacion.
Conversacion-RN-4	El nombre de la conversacion no puede contener caracteres especiales o palabras ofensivas.
Conversacion-RN-5	No se puede modificar el identificador.

Responsabilidades

Responsabilidad	Entrada	Salida	Descripcion
Configurar Parametros de Traduccion	Lenguaje, Region	estado, mensaje	Establece el contexto antes de enviar mensajes; idioma y region destino obligatorios, origen opcional.
Traducir Mensaje	MensajeOriginal	estado, mensaje, MensajeDestino	Funcionalidad principal: recibe el mensaje (audio o texto), detecta origen si falta, procesa modismos/contexto emocional y genera el MensajeDestino por el mismo canal de entrada. Valida formato de audio compatible (mp3, wav, m4a), no vacio, dentro de limites de duracion.
Consultar historial de conversaciones	identificador	Lista<Conversacion>	Recupera mensajes origen y destino vinculados al ID de conversacion.
Registrar / Eliminar / Consultar Conversacion	—	—	CRUD estandar sobre la entidad.

Datos simulados (muestra)

Identificador	Nombre	Usuario	Lenguaje por conversacion
1520f08a-...	Conversacion 1	Julian David	LC1
cbe289b6-...	Conversacion 2	Diego Alejandro	LC2

Lenguaje

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Simple	nombre	La regla del lenguaje dice que el nombre no puede repetirse.
2	Simple	codigo	El codigo ISO (es, en, fr) identifica un idioma.

Como la region ya no es un atributo del lenguaje sino una relacion muchos a muchos (`LenguajeRegion`), el nombre del lenguaje si puede ser unico: "Espanol" existe una vez y se relaciona con Antioquia, Bogota, Madrid, etc. Decision registrada en ADR-008.

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	1–50	Si	Nombre del idioma. No puede repetirse.

Atributo	Tipo	Longitud	Obligatorio	Descripcion
regiones	Referencia a Region (varias)	—	No	Regiones donde se habla el lenguaje (relacion muchos a muchos `LenguajeRegion`, ver abajo).
codigo	Alfanumerico	2–5	Si	Formato ISO 639-1 / 639-2 (es, en, fr, etc.).
activo	Logico	—	Si	Indica si el idioma esta disponible para traduccion. Default `false`.

Reglas de negocio y responsabilidades

Responsabilidad	Regla	Descripcion
Listar lenguajes disponibles	—	El nombre de un lenguaje no puede repetirse; no pueden existir dos lenguajes con identificadores iguales.
Generar reporte de mala traduccion	—	Permite reportar una mala traduccion del modelo, promoviendo su mejora.

Datos simulados (muestra)

Nombre	Region
Espanol	Antioquia
Ingles	California
Frances	Paris
Portugues	Rio de Janeiro

LenguajePorConversacion

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Compuesta	conversacion + lenguaje	Un mismo lenguaje no puede estar dos veces en la misma conversacion.

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	3–5	Si	Autogenerado y calculado, formato `LC{n}` incrementado por registro.
conversacion	Referencia a Conversacion	—	Si	Conversacion a la que pertenece el lenguaje.
lenguaje	Referencia a Lenguaje	—	Si	Lenguaje en el que esta dada la conversacion.

Reglas de negocio

No se puede modificar el identificador; no pueden existir dos registros con identificadores iguales.

Datos simulados (muestra)

Nombre	Conversacion	Lenguaje
LC1	Conversacion 1	Espanol
LC2	Conversacion 1	Ingles

MensajeOriginal

Combinaciones unicas: ninguna. Dos mensajes pueden tener el mismo contenido ("Hola") en la misma conversacion.

Atributo	Tipo	Longitud	Obligatorio	Sensible	Descripcion
identificador	Alfanumerico (UUID)	36	Si	No	Autogenerado.
contenido	Texto	1–5000	Si	**Si**	Contenido textual del mensaje en su idioma original; no puede ser solo espacios.
fechaCreacion	Fecha	—	Si	No	Autogenerado, no mayor a la actual.
lenguajeConversacion	Referencia a LenguajePorConversacion	—	Si	Lenguaje en el que fue expresado el mensaje.	
canal	Referencia a Canal	—	Si	Canal por el cual se transmite el mensaje.	
entonacion	Referencia a Entonacion	—	Si	Entonacion con la que se dio el mensaje.	

Datos simulados (muestra)

LenguajeConversacion	Canal	Contenido	Entonacion
LC1	Voz	"Hay que ponerse las pilas con el trabajo"	Preocupado
LC2	Voz	"Let's start now"	Neutral

MensajeDestino

Combinaciones unicas: ninguna por ahora. Si se decide que un mensaje original tiene una sola traduccion por lenguaje, seria (mensajeOriginal + lenguajeConversacion).

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
mensajeOriginal	Referencia a MensajeOriginal	—	Si	Mensaje del que sale esta traduccion.
contenidoTraducido	Texto (cifrado)	1–5000	Si	Contenido del mensaje traducido; se guarda cifrado (AES-256-GCM).
notaCultural	Texto	0–2000	No	Explicacion del modismo adaptado, si lo hubo.
aproximada	Logico	—	Si	Verdadero si quedaron palabras sin traducir.
motor	Alfanumerico	1–60	Si	Motor que tradujo (`qwen:...`, `claude:...` o `local`).
fechaCreacion	Fecha	—	Si	Autogenerado, no mayor a la actual.
estadoEntrega	Alfanumerico	1–15	Si	Valores: `Pendiente` (default), `Enviado`, `Entregado`, `Fallido`.
lenguajeConversacion	Referencia a LenguajePorConversacion	—	Si	Lenguaje al que se tradujo el mensaje.

Datos simulados (muestra)

LenguajeConversacion	Mensaje Original	Contenido Traducido
LC2	"Hay que ponerse las pilas con el trabajo"	"We need to get cracking on the work."
LC1	"Let's start now"	"Empecemos de una vez."

Canal

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Simple	nombre	No puede haber dos canales llamados "Voz" o "Texto".

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	3–50	Si	Nombre descriptivo del canal (ej. Texto, Voz).

Responsabilidades

- Mostrar las opciones por las que se puede enviar o entregar un mensaje (cuadro de texto o captura por voz) una vez configuradas las opciones de traduccion.
- Identificar el canal por el cual llego cada mensaje (cada mensaje lleva su propio canal: la persona puede alternar entre hablar y escribir dentro de la misma conversacion).
- Entregar el mensaje traducido aplicando el canal configurado.

Datos simulados

Nombre
Texto
Voz

Entonacion

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Simple	nombre	No puede haber dos entonaciones con el mismo nombre.

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	3–30	Si	Nombre de la entonacion. Catalogo inicial: Neutral, Amigable, Alegre, Preocupado, Enojado, Formal.
activo	Booleano	—	Si	Default `true`. Indica si esta disponible para uso.

Responsabilidades

- Identificar el tono y matices de un mensaje origen recibido.
- Aplicar el tono y matices identificados al mensaje destino generado.

Datos simulados

Nombre
Preocupado
Amigable
Neutral
Enojado

Region

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Compuesta	nombre + identificador de pais	El nombre de una region no se repite dentro del mismo pais . Si puede repetirse en paises distintos.

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	2–100	Si	Nombre de la region geografica.
identificador de Pais	Alfanumerico (UUID)	36	Si	Pais al que pertenece la region.

Responsabilidades

- Listar regiones disponibles al configurar opciones de traduccion.
- Configurar region destino para la conversacion.
- Identificar automaticamente la region origen a partir del mensaje.
- Aplicar matices culturales de la region destino al mensaje de forma automatica.

Datos simulados

Nombre	Pais
Antioquia	Colombia
California	Estados Unidos
Paris	Francia
Rio de Janeiro	Brasil

Pais

Combinaciones unicas:

N.	Tipo	Atributos	Por que
1	Simple	nombre	No pueden existir dos paises con el mismo nombre.
2	Simple	codigo	No debe repetirse el codigo del pais (ISO 3166-1).

Atributo	Tipo	Longitud	Obligatorio	Descripcion
identificador	Alfanumerico (UUID)	36	Si	Autogenerado.
nombre	Alfanumerico	1–50	Si	Nombre del pais. **Combinacion unica.**
codigo	Alfanumerico	2–5	Si	Formato ISO 3166-1 alfa-2 (CO, ES, MX, etc.).

Responsabilidades

- Identificar automaticamente el pais al cual pertenece una region.

Datos simulados

Nombre
Colombia
Estados Unidos
Francia
Brasil

LenguajeRegion (relacion)

Relacion muchos a muchos entre Lenguaje y Region: una region puede hablar varios idiomas (Texas: ingles y espanol) y un idioma se habla en varias regiones. Permite validar Conversacion-RN-1 ("la region seleccionada debe ser valida para el lenguaje elegido").

Combinacion unica: compuesta (lenguaje + region).

Tablas de soporte (no son objetos de dominio)

Los reportes son dinamicos y no hacen parte del modelo de dominio. Estas tablas existen para que el sistema funcione y se documentan para que el modelo coincida con la base de datos (ADR-008):

Tabla	Para que
`reporte`	Guarda el reporte de mala traduccion (responsabilidad "Generar reporte de mala traduccion" de Lenguaje): mensaje, motivo, sugerencia, estado. Combinacion unica: mensaje destino + usuario.
`token_refresco`	Sesiones: hash del token de refresco, vencimiento y si fue revocado.
`sesion_revocada`	Tokens de acceso invalidados al cerrar sesion (Usuario-RN-10).
`recuperacion`	Codigo de recuperacion de contrasena (hash, vencimiento de 15 minutos, usado) (Usuario-RN-8).

Correspondencia con la base de datos

Los nombres de las tablas siguen el modelo de dominio (lenguaje ubicuo, ADR-009): `usuario`, `conversacion`, `lenguaje`, `lenguaje_por_conversacion`, `mensaje_original`, `mensaje_destino`, `canal`, `entonacion`, `region`, `pais`, `lenguaje_region`. `MensajeOriginal.contenido` y `MensajeDestino.contenidoTraducido` se guardan cifrados (ver *documento 16 (Matriz de sensibilidad de datos)*).

Nota metodologica sobre combinaciones unicas

Las combinaciones unicas indican que datos no se pueden repetir entre registros (numeradas 1, 2, 3... si hay mas de una). Pueden ser:

- **Simples**: un solo atributo no se repite (ej. Pais.nombre).
- **Compuestas**: la combinacion de varios atributos no se repite, aunque cada atributo por separado si pueda repetirse (ej. Region.nombre no se repite *dentro del mismo* Pais, pero "Antioquia" podria existir como region en otro pais).

El identificador nunca se marca como combinacion unica (ya es unico por definicion). No es obligatorio que un objeto de dominio tenga combinaciones unicas, y puede tener mas de una.

13. Especificacion de vistas y UI

Fase 4/5 - Diagramas C4 y especificacion de componentes UI

Version: 1.0 (29/10/2026). Template de UI: **Luna** (`componentes/template/native-luna`). Cada etapa del Customer Journey Map esta mapeada a pantallas y componentes de `frontend/`. Diagramas C4 en *documento 9 (Diagramas C4)*.

Customer Journey Map

Etapa	Objetivo del usuario	Acciones del usuario	Puntos de contacto / Interfaz	Funcionalidades clave	Emociones / Experiencia	Oportunidades de mejora
1. Registro	Acceder a la aplicacion y crear una cuenta	Ingreso a la app, opcion "crear cuenta", ingreso de datos personales	Pantalla de inicio de sesion, formulario de registro	Registro de usuario, validacion de datos	Expectativa, motivacion	Agilizar el proceso de registro, con opciones de redes sociales
2. Recuperacion de contrasena	Recuperar acceso en caso de olvido	Seleccion de "restablecer contrasena", ingreso de correo electronico	Pantalla de recuperacion, correo de confirmacion	Envio automatico de correo con un codigo de 6 digitos (vence en 15 minutos)	Tranquilidad, confianza	Mejorar la seguridad y la rapidez del proceso
3. Inicio de sesion	Ingresar a la aplicacion con credenciales	Ingreso de usuario y contrasena, acceso a la app	Pantalla de login, validacion de credenciales	Autenticacion segura, redireccion a la interfaz principal	Confianza, satisfaccion	Agregar opciones de inicio con redes sociales
4. Configuracion de idioma y region	Seleccionar el idioma y la region de destino	Eleccion del idioma y region antes de traducir el mensaje	Menu de configuracion, selector de idioma y region	Personalizacion cultural, deteccion automatica de idioma	Confianza, claridad	Mejorar la deteccion automatica y sugerencias de region
5. Entrada de mensaje	Escribir o hablar el mensaje a traducir	Ingreso de texto o activacion del microfono para voz	Campo de texto, boton de microfono, interfaz de voz	Entrada por texto o voz, deteccion automatica del idioma	Comodidad, versatilidad	Optimizar la precision del reconocimiento de voz y texto
6. Traduccion cultural y emocional	Obtener una traduccion que mantenga el tono y la cultura	La aplicacion traduce el mensaje y lo adapta culturalmente	Interfaz de traduccion, motor de IA con analisis cultural y emocional	Traduccion adaptativa, equivalencias culturales, mantenimiento del tono	Asombro, satisfaccion	Mejorar la precision y naturalidad de las adaptaciones culturales
7. Entrega del mensaje traducido	Recibir el mensaje en el canal original de entrada	Visualizar o escuchar el mensaje traducido en el mismo formato que se utilizo para enviarlo	Pantalla de resultados, reproductor de audio, interfaz de texto	Adaptacion del canal de entrega, traduccion contextualizada	Satisfaccion, conexion emocional	Mejorar la sincronizacion entre voz y texto, garantizar calidad en ambos formatos
8. Conversacion continua	Mantener un flujo de comunicacion fluido y natural	El usuario toca "Otra persona" y le pasa el celular; la respuesta se traduce a su idioma (si el texto no parece del idioma esperado, la app avisa)	Interfaz de conversacion, deteccion automatica de idioma	Traduccion en tiempo real, adaptacion continua del contexto	Confianza, fluidez	Optimizar la velocidad de traduccion y precision en contextos complejos

Etapa	Objetivo del usuario	Acciones del usuario	Puntos de contacto / Interfaz	Funcionalidades clave	Emociones / Experiencia	Oportunidades de mejora
9. Retroalimentacion y mejora	Evaluar la calidad de la traduccion y contribuir a la mejora	Reportar una traduccion con un motivo y una sugerencia (el administrador los revisa)	Pantalla de retroalimentacion, formulario de feedback	Aprendizaje automatico, personalizacion de la app	Satisfaccion, confianza a largo plazo	Implementar mejoras continuas basadas en feedback

Fuente: hoja "Customer Journey Map" del modelo de dominio enriquecido.

Etapas del Customer Journey Map en la app

Etapa	Pantalla (`frontend/app/...`)	Componentes	HU
1. Registro	`screens/welcome`, `screens/signup`, `screens/politica-datos`	Button, Input, casilla de consentimiento	HU0015
2. Recuperacion de contrasena	`screens/forgot-password`	Input, Button	HU0017
3. Inicio de sesion	`screens/login`	Input, Button	HU0016
4. Configuracion de idioma y region	`(drawer)/index` sin `id` (`components/talklate/NuevaConversacion`)	Selector (Chip), Input	HU0006, HU0008, HU0009, HU0023
5. Entrada de mensaje	`(drawer)/index` con `id` (`ChatView`)	BurbujaVoz (modo voz por defecto: tocar, hablar y soltar con 1.5 s de silencio), TranslatorInput (modo chat: escribir o grabar)	HU0001, HU0002
6. Traduccion cultural y emocional	`ChatView`	BurbujaVoz girando mientras traduce; MessageBubble (traduccion, tono, nota cultural)	HU0003, HU0007
7. Entrega del mensaje traducido	`ChatView`	Voz natural con la emocion del mensaje (reproduccion automatica en la burbuja), boton Escuchar en el chat; voz del celular de respaldo	HU0004, HU0005
8. Conversacion continua	`ChatView`, `screens/historial`, menu lateral	TranslatorInput ("Yo" / "Otra persona"), CustomDrawerContent	HU0010 a HU0014, HU0019, HU0020
9. Retroalimentacion y mejora	`ChatView` (Reportar), `screens/admin-reportes`	PromptModal, Selector	HU0021, HU0022
Cuenta	`screens/profile`	Cerrar sesion, Eliminar mi cuenta	HU0018, HU0026
Administracion	`screens/admin-idiomas`	Switch, Input, Selector	HU0024, HU0025

Diferencias con el diseno original (decididas al construir)

- Etapa 2: codigo de 6 digitos en vez de enlace (no requiere abrir el correo en el mismo celular).
- Etapa 8: la respuesta de la otra persona se marca con "Otra persona" en vez de detectarse sola, para no confundir quien habla.
- Etapa 9: reportar con motivo en vez de calificar con estrellas, porque el reporte dice que mejorar.
- Etapa 5: la pantalla del traductor abre en la burbuja animada que representa la IA (escucha, gira mientras traduce y habla); el chat queda a un toque en la cabecera, como en los asistentes de voz.
- Las opciones de inicio con redes sociales quedan como mejora futura.

14. Planificacion de sprints

Version: 1.0 (29/10/2026)

Metodologia: priorizar las historias por **valor** (no por dependencia), tallarlas con tallas de ropa, asignarles horas y llenar sprints de duracion fija.

Supuestos

- El trabajo de grado dura unos 6 meses: 1 mes de analisis y diseno, 3 meses de construccion y 1 mes de cierre (ver el cronograma abajo).
- Sprint de **un mes** (4 semanas, 20 dias habiles). 2 personas por 4 horas al dia = 160 horas brutas por sprint.
- Se usa el 75 % (**120 horas**): el resto es planeacion, dailies, refinamiento, retrospectiva e imprevistos (por ejemplo, festivos).
- Talla a horas (una persona desarrolla la historia, incluyendo analisis, desarrollo, pruebas, revision y documentacion): XS = 4, S = 8, M = 16, L = 24, XL = 32, XXL = 48.
- Se llena cada sprint de mayor a menor valor hasta cerca de 120 horas y, si sobra espacio, se acomoda una historia pequena de mas abajo. **No se planifica por dependencia**: por eso Cerrar sesion (prioridad 22) entra en el sprint 2 aunque Iniciar sesion llegue en el sprint 3.

Historias por prioridad

Prioridad	Historia	Epica	Talla	Horas	Sprint	Por que genera valor
1	HU0003 Traducir mi mensaje al idioma destino en tiempo real	Traduccion de mensajes	XXL	48	1	Es el corazon del producto: sin esto TalkLate no entrega su valor diferencial.
2	HU0002 Ingresar mi mensaje escribiendo	Traduccion de mensajes	S	8	1	Es la entrada mas simple y la que permite usar la traduccion desde el primer dia.
3	HU0001 Ingresar mi mensaje hablando	Traduccion de mensajes	L	24	1	Es la forma natural de usar el producto: hablar y ser entendido sin escribir.
4	HU0004 Escuchar el mensaje traducido	Traduccion de mensajes	M	16	1	Permite que la otra persona reciba el mensaje de forma comoda y natural.
5	HU0005 Recibir la traduccion por el mismo canal que use	Traduccion de mensajes	S	8	1	Mantiene la experiencia coherente: si hablo, escucho; si escribo, leo.
6	HU0006 Seleccionar el idioma y la region destino	Gestion de conversaciones	M	16	1	Define a que persona y lugar se adapta el mensaje: de ahi sale el matiz cultural.
7	HU0007 Reconocer el idioma origen	Gestion de conversaciones	L	24	2	Evita configurar a mano el idioma cada vez y acelera la conversacion.
8	HU0008 Crear una conversacion	Gestion de conversaciones	S	8	2	Agrupa los mensajes de una misma charla para no perder el contexto.
9	HU0009 Configurar los parametros una sola vez	Gestion de conversaciones	S	8	2	Ahorra repetir la configuracion en cada mensaje.
10	HU0010 Mantener el hilo continuo	Gestion de conversaciones	M	16	2	Hace posible un dialogo fluido, que es lo que busca el usuario.

Prioridad	Historia	Epica	Talla	Horas	Sprint	Por que genera valor
11	HU0019 Recibir el mensaje traducido	Participacion del interlocutor	S	8	2	Hace que la otra persona participe sin instalar ni operar la app.
12	HU0020 Responder en mi idioma	Participacion del interlocutor	M	16	2	Cierra el dialogo en ambos sentidos.
13	HU0023 Consultar los idiomas disponibles	Administracion de idiomas y regiones	XS	4	2	Muestra con que idiomas se puede hablar antes de empezar.
14	HU0011 Consultar mi historial	Gestion de conversaciones	M	16	2	Permite repasar y aprender de conversaciones pasadas.
15	HU0012 Abrir una conversacion anterior	Gestion de conversaciones	S	8	2	Permite retomar una conversacion donde se dejo.
16	HU0015 Registrarme en la aplicacion	Cuenta de usuario	L	24	3	Guarda las conversaciones del usuario y protege sus datos; el valor central no depende de esto.
17	HU0016 Iniciar sesion	Cuenta de usuario	M	16	3	Da acceso seguro a lo guardado.
18	HU0021 Reportar una mala traduccion	Reportes	M	16	3	Alimenta la mejora de calidad con casos reales.
19	HU0013 Renombrar una conversacion	Gestion de conversaciones	XS	4	2	Ayuda a ubicar conversaciones, pero no cambia el valor central.
20	HU0014 Eliminar una conversacion	Gestion de conversaciones	XS	4	2	Da control sobre los datos propios; es de apoyo.
21	HU0017 Recuperar mi contrasena	Cuenta de usuario	L	24	3	Evita perder la cuenta; es de apoyo.
22	HU0018 Cerrar sesion	Cuenta de usuario	XS	4	2	Protege la cuenta en un dispositivo compartido; es de apoyo. Por valor cabe en el sprint 2.
23	HU0022 Revisar los reportes de mala traduccion	Reportes	M	16	3	Permite priorizar que mejorar primero.
24	HU0024 Agregar un idioma o una region	Administracion de idiomas y regiones	M	16	3	Hace crecer el servicio a mas personas.
25	HU0025 Activar o desactivar un idioma o una region	Administracion de idiomas y regiones	XS	4	3	Controla que idiomas se ofrecen con calidad suficiente.
26	HU0026 Eliminar mi cuenta	Cuenta de usuario	XS	4	3	Cumple el derecho al olvido (Ley 1581 de 2012): la persona controla sus datos.

Sprints

Sprint	Fechas	Horas	Historias
1	19 de octubre al 13 de noviembre de 2026	120	HU0003, HU0002, HU0001, HU0004, HU0005, HU0006
2	16 de noviembre al 11 de diciembre de 2026	120	HU0007, HU0008, HU0009, HU0010, HU0019, HU0020, HU0023, HU0011, HU0012, HU0013, HU0014, HU0018
3	14 de diciembre de 2026 al 8 de enero de 2027	120	HU0015, HU0016, HU0021, HU0017, HU0022, HU0024, HU0025, HU0026

Total: 360 horas en 3 sprints (3 meses de construccion).

Como se llena el sprint 2: con las prioridades 7 a 15 llega a 108 horas. Las siguientes (HU0015 de 24 h, HU0016 y HU0021 de 16 h) no caben en las 12 horas que quedan, asi que entran las siguientes que si caben: HU0013, HU0014 y HU0018 (4 h cada una).

Por que el sprint 1 tiene pocas historias

El sprint 1 lleva la historia de mayor valor y mas grande: traducir el mensaje (XXL, 48 h). Ademas de la historia, en ese sprint se deja lista la base que usan todas las demas (arquitectura en capas, seguridad, conexion a la base de datos, integracion continua). Hacerlo completo desde el inicio evita rehacer trabajo en los sprints siguientes. Tambien es el sprint en el que el equipo ajusta su forma de trabajar (issues, ramas, revisiones).

Cronograma del trabajo de grado

Etapa	Fechas	Entregable	Estado
Analisis y diseno	agosto a octubre de 2026	Vision, modelo de dominio, mapa de impacto, historias, requerimientos y arquitectura (fases F0 a F5)	Terminado
Construccion: sprint 1	19/10/2026 al 13/11/2026	Liberacion 1: traduccion funcional	En curso
Construccion: sprint 2	16/11/2026 al 11/12/2026	Liberacion 2: conversaciones completas	Planeado
Construccion: sprint 3	14/12/2026 al 08/01/2027	Liberacion 3: cuentas, calidad y administracion	Planeado
Cierre	11/01/2027 al 05/02/2027	Pruebas con usuarios, manual de usuario, documento final y preparacion de la sustentacion	Planeado

Relacion con la prueba de concepto del 14/10/2026

Antes del sprint 1 se hizo una **prueba de concepto (spike)**: una version funcional de la app para validar el stack y el motor de traduccion antes de comprometer la arquitectura. Los sprints toman cada historia desde ese punto: revisar y completar el codigo, probarlo en celular, cumplir sus criterios de aceptacion, sus pruebas y la documentacion. Cada historia se trabaja con su issue de GitHub, su rama y su pull request.

Criterios de la planificacion

- Por que traducir va primero: porque es el valor del producto, no porque lo demas dependa de eso. Registro y cierre de sesion son de apoyo.
- Por que cerrar sesion va antes que iniciar sesion: el orden es por valor y no por dependencia; en la practica la base de autenticacion ya existe desde el sprint 1.
- Por que el sprint 1 tiene pocas historias: lleva la base completa que usan las demas, para no rehacer trabajo.
- La estimacion es metodologica (valor, tallas, capacidad y sprints): el plan de liberaciones sale de estos sprints; las horas reales de cada mes pueden variar.

15. Plan de liberaciones

Version: 1.0 (29/10/2026)

Cada sprint dura un mes (4 semanas, 120 horas) y termina con una liberacion. El primer sprint empieza el **lunes 19 de octubre de 2026** y la construccion termina el **8 de enero de 2027**. Historias, tallas y horas en *documento 14 (Planificacion de sprints)*.

Liberacion	Sprint	Fechas	Nombre	Historias	Horas	Indicadores que se revisan
1	1	19 de octubre al 13 de noviembre de 2026	Traduccion funcional	HU0003, HU0002, HU0001, HU0004, HU0005, HU0006	120	Impactos 1 y 2 (traduccion cultural, voz)
2	2	16 de noviembre al 11 de diciembre de 2026	Conversaciones completas	HU0007, HU0008, HU0009, HU0010, HU0019, HU0020, HU0023, HU0011, HU0012, HU0013, HU0014, HU0018	120	Impactos 3, 4, 5 y 6 (conversaciones, interlocutor, historial)
3	3	14 de diciembre de 2026 al 8 de enero de 2027	Cuentas, calidad y administracion	HU0015, HU0016, HU0021, HU0017, HU0022, HU0024, HU0025, HU0026	120	Impactos 7, 8, 9 y 10 y el objetivo general

Que recibe el usuario en cada liberacion

- **Liberacion 1 - Traduccion funcional:** El usuario puede escribir o hablar un mensaje, recibirlo traducido con el tono y los modismos adaptados, escucharlo y obtenerlo por el mismo canal. Es el valor central del producto.
- **Liberacion 2 - Conversaciones completas:** El usuario organiza sus conversaciones (reconocer idioma, crear, hilo continuo, historial, renombrar y eliminar), la otra persona participa y se puede cerrar sesion.
- **Liberacion 3 - Cuentas, calidad y administracion:** Registro con consentimiento de datos, inicio de sesion, recuperacion de contrasena, eliminar la cuenta, reportar y revisar malas traducciones, y agregar, activar o desactivar idiomas y regiones.

Al cierre de cada liberacion se publica una version con etiqueta (`v0.1.0`, `v0.2.0`, `v1.0.0`, segun la convencion de versionamiento del proyecto) y se miden sus indicadores SMART (ver *documento 2 (Mapa de impacto)*).

Nota

El orden sale de la prioridad por valor, no de las dependencias. El plan es metodologico: las horas reales de cada mes pueden variar. Despues de la construccion viene un mes de cierre (11/01 al 05/02/2027): pruebas con usuarios, manual de usuario y documento final.

16. Matriz de sensibilidad de datos

Fase 2 - Version 1.0 (29/10/2026). Clasifica cada dato del sistema y dice que control lo protege y donde esta implementado.

Clasificaciones: **Publico**, **Interno** (solo usuarios autenticados), **Confidencial** (roles especificos), **PII** (identifica personas naturales).

Entidad	Dato	Clasificacion	Controles	Donde se implementa
Usuario	identificador	Interno	JWT	`backend/src/aplicacion/sesiones.js`
Usuario	nombre, apellido	PII	JWT, solo el dueno lo ve, se borra al eliminar la cuenta	`aplicacion/cuentas.js` (`/usuarios/yo`)
Usuario	correo	PII	JWT; nunca se escribe en los logs; unico (Usuario-RN-1)	`compartido/log.js` filtra `correo`
Usuario	fecha de nacimiento	PII	Solo para validar la edad minima (13 anos); no se muestra a terceros	`dominio/validaciones.js`
Usuario	contrasena	Confidencial	Solo se guarda el hash (bcrypt); nunca en logs ni respuestas	`aplicacion/cuentas.js`
Usuario	rol	Confidencial	Solo el servidor lo asigna; RBAC en rutas de administrador	`adaptadores/http/sesion.js` (`requerirAdmin`)
Usuario	consentimiento de datos	PII	Fecha de aceptacion de la politica (Ley 1581)	`usuario.consentimiento_datos`
Sesion	token de acceso	Confidencial	JWT de 15 minutos, revocable; nunca en logs	`aplicacion/sesiones.js`
Sesion	token de refresco	Confidencial	30 dias, se guarda solo su hash, se rota y se revoca al cerrar sesion	tabla `token_refresco`
Recuperacion	codigo de 6 digitos	Confidencial	Hash, 15 minutos, un solo uso, limite de solicitudes; nunca en logs	`aplicacion/cuentas.js`
Conversacion	nombre, fechas	PII (asociado a la persona)	Solo el dueno; se borra con la cuenta	`aplicacion/conversaciones.js`
MensajeOriginal	contenido (texto o voz transcrita)	PII	**Cifrado en reposo** AES-256-GCM; solo el dueno; nunca en logs; se borra con la conversacion o la cuenta	`adaptadores/cifrado/aesGcm.js`
MensajeDestino	contenido traducido	PII	Igual que el mensaje original	`adaptadores/cifrado/aesGcm.js`
Mensaje hablado	audio grabado y medidas de volumen	PII	Viaja por HTTPS, se transcribe en el servidor (Whisper local) y no se guarda: solo queda el texto cifrado; tipo y tamano validados (maximo 10 MB)	`aplicacion/conversaciones.js`
Genero inferido de la voz	hombre o mujer, estimado por el pitch del audio	PII (dato inferido)	Se guarda por mensaje (`mensaje_original.genero`) solo para elegir la voz de salida y viaja a ElevenLabs; no se muestra a terceros; se borra con la cuenta	`adaptadores/audio/genero.js`

Entidad	Dato	Clasificacion	Controles	Donde se implementa
Problema de la app	descripcion, categoria, contacto (texto libre)	PII	Solo el autor (al crearlo) y el administrador; maximo 10 por hora por usuario; se borra con la cuenta	`aplicacion/problemas.js`
Voz sintetizada	audio de la traduccion	PII	Solo en memoria del servidor, con id aleatorio de 128 bits que vence en 5 minutos; nunca en la base ni en logs	`aplicacion/voz.js`
MensajeDestino	nota cultural, motor, estado	Interno	JWT	-
Reporte	motivo, sugerencia	PII	Solo el autor (al crearlo) y el administrador; se borra con la cuenta	`aplicacion/reportes.js`
Lenguaje, Region, Pais, Canal, Entonacion	catalogo	Interno	JWT para leer; solo administrador para cambiar	`aplicacion/catalogo.js`
Logs	eventos	Interno	JSON, solo identificadores (nunca correo, contrasena, token, codigo ni contenido)	`compartido/log.js`

Envio a terceros

- **Claude Sonnet en Google Vertex AI (traduccion):** es el motor principal (ADR-020). El texto del mensaje y el contexto del prompt (idiomas, region, tono y, en voz, una descripcion de como hablo la persona) se envian a Google Cloud para traducirlo.
- **Qwen local (Ollama):** adaptador alternativo que corre en el servidor de TalkLate y no envia nada a terceros; no se usa en la configuracion actual.
- **Whisper local:** corre en el servidor de TalkLate; el audio original no sale a terceros.
- **ElevenLabs (voz):** recibe el texto ya traducido, el tono, el genero de quien habla y la velocidad para generar el audio; nunca recibe el audio original.
- **Azure Speech u OpenAI (respaldo de voz):** si se usan, reciben solo el texto ya traducido para convertirlo en voz.
- Estos envios se informan en la politica de tratamiento de datos (*documento 17 (Politica de tratamiento de datos personales)*), que el usuario acepta al registrarse.

Retencion

Los datos se conservan mientras la cuenta exista. La persona puede borrar conversaciones o su cuenta completa en cualquier momento (derecho al olvido). Los tokens revocados y vencidos se limpian automaticamente.

Controles de infraestructura en produccion

- TLS (HTTPS) en todos los tramos.

- Cifrado de la base y de los respaldos por el proveedor (SurrealDB Cloud).
- Proveedor de correo conectado al puerto `Notificador` para el codigo de recuperacion.

17. Politica de tratamiento de datos personales

Version: 1.0 (29/10/2026)

Segun la Ley 1581 de 2012 de Colombia. La app muestra un resumen en la pantalla "Tratamiento de datos" y pide aceptarla al registrarse.

1. **Responsables:** Julian David Diaz Medina y Diego Alejandro Franco Atehortua, autores de TalkLate (proyecto academico). Contacto: dafa123.daf@gmail.com.
2. **Datos que se recogen:** nombre, apellido, correo, fecha de nacimiento y el contenido de las conversaciones (mensajes originales y traducidos). La voz grabada se usa solo para transcribirla y estimar la entonacion; no se guarda.
3. **Finalidad:** crear y proteger la cuenta, traducir los mensajes, guardar el historial y mejorar la calidad de las traducciones con los reportes que la persona envie. Los datos no se venden ni se usan para publicidad.
4. **Edad minima:** 13 anos.
5. **Tratamiento por terceros:** la transcripcion de la voz (Whisper) se hace en el servidor de TalkLate y el audio original no sale de el. Para traducir, el texto del mensaje se envia a Google Cloud (Vertex AI), que lo procesa con el modelo Claude de Anthropic. Para generar la voz de la traduccion, el texto ya traducido se envia a ElevenLabs y, como respaldo, a Microsoft Azure u OpenAI. Ademas, TalkLate estima si quien habla tiene una voz masculina o femenina a partir de su tono de voz, solo para elegir una voz de salida adecuada; ese dato se guarda con el mensaje y se borra con la cuenta.
6. **Seguridad:** contrasenas con hash, mensajes cifrados, acceso solo del dueno, sesiones que vencen y registros sin datos personales.
7. **Derechos del titular:** conocer, actualizar, rectificar y suprimir sus datos, y revocar la autorizacion. Desde la app puede borrar conversaciones y eliminar su cuenta con todos sus datos; para lo demas, escribir al contacto.
8. **Vigencia:** mientras la cuenta exista.

18. Piramide de pruebas

Version: 1.0 (05/11/2026)

TalkLate se prueba en tres niveles: muchas pruebas unitarias rapidas en la base, menos pruebas de integracion en el medio y pocas pruebas de punta a punta en la punta. Todas corren solas en el CI de GitHub en cada pull request; una falla bloquea el merge.

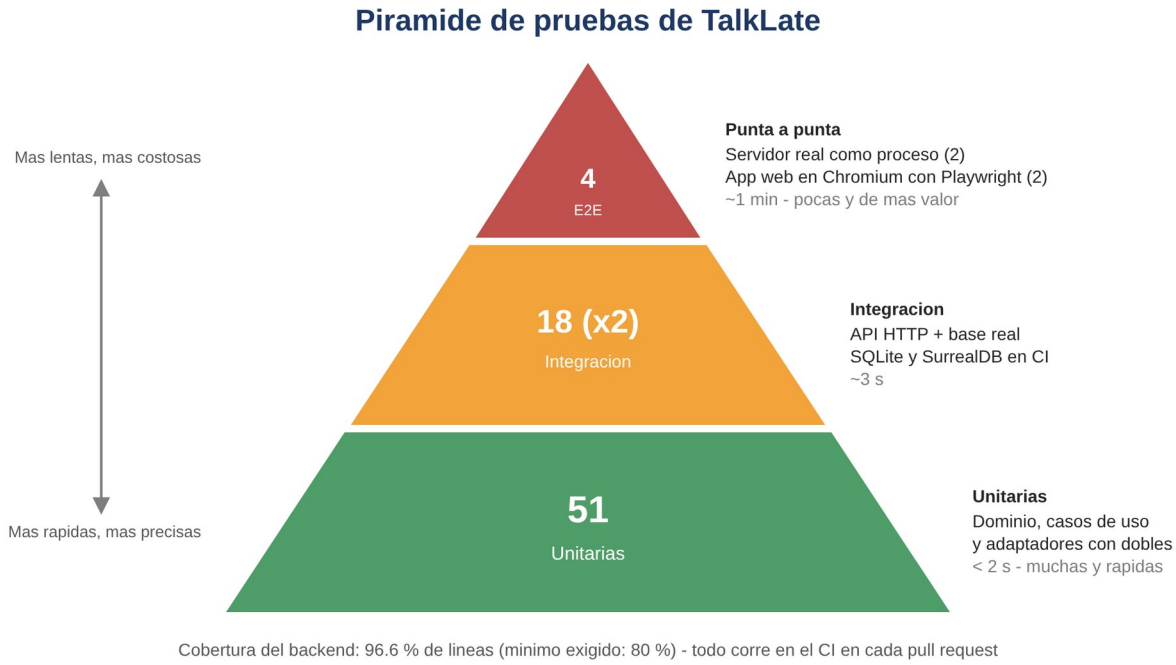


Figura 11. *Piramide de pruebas de TalkLate*

1. Resumen

Nivel	Que prueba	Pruebas	Tiempo	Donde estan	Como se corre
Punta de la piramide: punta a punta (E2E)	La app completa como la usa una persona: servidor real como proceso aparte y app web en un navegador	4 (2 de la API y 2 de la app web)	~1 min	`backend/test/e2e/`, `e2e/pruebas/`	`npm run test:e2e` (backend) y `npm test` (carpeta `e2e/`)
Medio: integracion	La API HTTP completa con una base de datos real (SQLite en memoria y SurrealDB en CI)	29 (18 de la API corren contra SQLite y SurrealDB; 11 de problemas, contra SQLite)	~3 s	`backend/test/integracion/`	`npm run test:integracion` y `npm run test:surreal`
Base: unitarias	Reglas del dominio, casos de uso y adaptadores, con dobles de prueba (sin red ni base)	78 (73 del backend y 5 del frontend)	< 2 s	`backend/test/unitarias/`, `frontend/test/`	`npm run test:unitarias` (backend) y `npm test` (frontend)

Cobertura del backend (unitarias + integracion): **96.5 % de lineas, 94.5 % de funciones y 84.6 % de ramas**. El minimo exigido es 80 % (convencion del proyecto) y el CI falla si baja.

2. Por que una piramide

- **Las unitarias son la base porque son rapidas y precisas.** Prueban una regla a la vez (por ejemplo, "una persona de 12 anos no puede registrarse") sin servidor ni base de datos. Cuando fallan, dicen exactamente que regla se rompio.
- **Las de integracion prueban que las piezas encajan.** La API recibe HTTP, valida, guarda en la base, cifra y responde con los codigos de la convencion REST. Son mas lentas y por eso hay menos.
- **Las de punta a punta prueban el recorrido real.** Son las mas costosas y fragiles, asi que solo cubren los flujos de mas valor: registrarse, conversar por texto y por voz, escuchar la traduccion, reportar y borrar la cuenta.
- La arquitectura hexagonal (ADR-013) es lo que hace barata la base: como el nucleo solo depende de puertos, cada caso de uso se prueba con adaptadores falsos.

3. Base: pruebas unitarias

Archivo	Que cubre	Pruebas
`backend/test/unitarias/dominio.test.js`	Reglas de negocio puras: correo, contrasena, edad minima, nombres, contenido del mensaje; deteccion de idioma, tono y region; prosodia de la voz; expresion de la voz por tono; errores de negocio	14
`backend/test/unitarias/aplicacion.test.js`	Casos de uso con dobles: Circuit Breaker de traduccion (con reloj falso), cadena de voz, vencimiento y limite de audios, paginacion, problemas reportados	12
`backend/test/unitarias/motores.test.js`	Adaptadores de IA con `fetch` falso: Claude (API y Vertex), Qwen, motor de reglas, Whisper, Azure, OpenAI y ElevenLabs; composicion de la cadena de motores	19
`backend/test/unitarias/seguridad.test.js`	Cifrado AES-256-GCM, limites de uso, logs sin datos personales, formato de error, notificador, hash de contrasenas (bcrypt), tokens JWT, validacion de la salida del modelo, limite de uso al reportar problemas y administrador sin contrasena por defecto en produccion	10
`backend/test/unitarias/glosario.test.js`	Glosario cultural: busqueda de modismos paisa y neoyorquinos (sin tildes ni falsos positivos), limite de entradas y que el prompt los incluya	8
`backend/test/unitarias/genero-voz.test.js`	Genero por el pitch de la voz: banda de decision, estimacion de la frecuencia, detector deshabilitado y ffmpeg inexistente	6
`backend/test/unitarias/surreal-cliente.test.js`	Cliente de SurrealDB con `fetch` falso: sesion, reintento con token vencido, literales seguros contra inyeccion	4
`frontend/test/reglas.test.ts`	Reglas de la app: etiqueta de la configuracion, idioma de entrega, resumen del volumen de la voz	5

4. Medio: pruebas de integracion

`backend/test/integracion/api.test.js` levanta la API en memoria (Express + composicion real) y la recorre por HTTP con 18 casos (`problemas.test.js` agrega 11 casos del reporte de problemas): health y OpenAPI, CORS, registro con consentimiento y edad, login y rotacion del token de refresco, recuperacion de contrasena, catalogo, creacion de conversaciones con sus reglas, mensajes por texto y por voz (con transcriptor y sintetizador falsos), historial, aislamiento entre usuarios, reportes, administracion de idiomas, cierre de sesion y borrado de la cuenta.

La misma suite corre dos veces en el CI: contra SQLite (`backend`) y contra un SurrealDB 3.x real (`backend-surreal`). Asi se prueba que los dos adaptadores del puerto `Repositorio` se comportan igual.

5. Punta: pruebas de punta a punta

Prueba	Que hace
`backend/test/e2e/recorrido-usuario.test.js`	Arranca `node src/server.js` como proceso aparte con SQLite en un archivo temporal. Reemplaza Whisper y el motor de voz (API compatible con OpenAI) por servidores HTTP falsos que hablan su mismo protocolo. Recorre registro, conversacion, mensaje escrito, mensaje hablado, voz natural con vector de emocion, historial, reporte y borrado de la cuenta. Tambien comprueba que sin sesion no se puede usar la API.
`e2e/pruebas/traducir.spec.js` (Playwright)	Abre la app web exportada en Chromium con vista de celular (Pixel 7). Inicia sesion, crea una conversacion a Ingles / New York, ve la burbuja de voz, pasa al chat, escribe "Hay que ponerse las pilas con el trabajo" y comprueba la traduccion adaptada "We need to get cracking on the work". Tambien comprueba que una contrasena incorrecta no deja entrar.

Playwright levanta solo el backend y un servidor estatico para la web (`e2e/playwright.config.js`). Guarda capturas y trazas de las pruebas que fallan, y el CI sube el reporte como artefacto.

6. Como correr todo en local

Para levantar la app completa (backend + Expo, y con `--full` tambien Whisper) basta `./run-talklate.sh` en la raiz del repositorio. Para las pruebas:

```
# Backend: base y medio (rapidas)
cd backend
npm run test:unitarias
npm run test:integracion
npm run coverage          # unitarias + integracion con cobertura minima de 80 %
npm run test:e2e          # punta: servidor real como proceso
npm run test:todo         # los tres niveles juntos
npm run arch:check        # reglas de dependencia de la arquitectura hexagonal

# Frontend: unitarias y tipos
cd ../frontend
npm test
npm run typecheck
```

```
# Punta: app web en el navegador
npx expo export --platform web
cd ../e2e
npm install
npx playwright install chromium
npm test
```

7. En el CI (`.github/workflows/ci.yml`)

Job	Pasos
`backend`	unitarias, integracion con cobertura, punta a punta de la API, reglas de dependencia (`arch:check`)
`backend-surreal`	integracion contra SurrealDB 3.x real y verificacion del esquema
`frontend`	unitarias y chequeo de tipos de TypeScript
`e2e-web`	exporta la web y corre Playwright (solo si `backend` y `frontend` pasaron)

8. Reglas del equipo

- Todo cambio de una regla de negocio llega con su prueba unitaria.
- Un endpoint nuevo llega con su caso en la prueba de integracion.
- Las pruebas de punta a punta se reservan para los recorridos de mas valor; no se duplican ahi casos que ya cubre la base.
- Nunca se desactiva una prueba para que el CI pase: se arregla la causa.

19. Seguridad con Trivy

Version: 1.1 (12/11/2026)

TalkLate escanea su codigo con Trivy en cada pull request, en cada push a `develop` y `main`, y cada lunes. El escaneo busca dependencias con vulnerabilidades conocidas, secretos escritos en el codigo y configuraciones inseguras. Si encuentra algo de severidad alta o critica, el CI falla y no se puede hacer merge.

1. Resultado actual

Que se escanea	Hallazgos HIGH / CRITICAL antes	Despues	Como se resolvio
Dependencias del backend (`backend/package-lock.json`)	1 HIGH (`body-parser` 1.20.8)	0	Actualizacion de `express` a 4.22.3, que trae `body-parser` 1.20.9
Dependencias del frontend (`frontend/package-lock.json`), de produccion	1 CRITICAL y 28 HIGH	0 (1 excepcion documentada)	`overrides` de npm hacia versiones parchadas, sin cambiar de version mayor (ver tabla 2)
Dependencias de desarrollo del frontend (eslint, TypeScript y sus herramientas)	22 HIGH	0	`overrides` de npm hacia versiones parchadas de la misma version mayor
Dependencias de las pruebas (`e2e/package-lock.json`)	0	0	-
Secretos en el codigo (claves, tokens, contrasenas)	0	0	Las credenciales viven en `.env`, que no se sube al repositorio
Configuraciones inseguras (IaC)	0	0	El proyecto no tiene Dockerfile ni infraestructura como codigo todavia; el escaneo queda activo para cuando la haya

2. Dependencias corregidas en el frontend

Todas son dependencias indirectas de las herramientas de Expo y React Native. Se fuerzan con `overrides` en `frontend/package.json` a la version parchada mas cercana, sin saltar de version mayor, para no romper nada. Despues del cambio pasan las pruebas, el chequeo de tipos y la exportacion web.

Paquete	Version antes	Version ahora	Severidad mas alta	Riesgo
`shell-quote` (react-devtools)	1.8.3	1.12.0	CRITICAL	Ejecucion de comandos por texto sin escapar
`@xmldom/xmldom` (herramientas de Expo)	0.8.10	0.8.15	HIGH	Inyeccion y denegacion de servicio con XML manipulado
`node-forge` (firma de codigo de Expo)	1.4.0	1.4.3	HIGH	Falsificacion de firmas y denegacion de servicio en TLS

Paquete	Version antes	Version ahora	Severidad mas alta	Riesgo
`js-yaml`	4.1.0	4.3.2	HIGH	Denegacion de servicio con YAML manipulado
`browserslist`	4.26.2	4.29.5	HIGH	Contaminacion de prototipos y denegacion de servicio
`picomatch` 2.x	2.3.1	2.3.2	HIGH	Denegacion de servicio por expresiones regulares
`ws` 7.x (react-devtools)	7.5.10	7.5.13	HIGH	Denegacion de servicio por agotamiento de memoria
`minimatch` 3.x y 9.x (desarrollo)	3.1.2 / 9.0.5	3.1.4 / 9.0.7	HIGH	Denegacion de servicio por patrones de busqueda
`brace-expansion` 1.x y 2.x (desarrollo)	1.1.11 / 2.0.1	1.1.20 / 2.1.6	HIGH	Denegacion de servicio por expansion de llaves
`glob` 10.x (desarrollo)	10.4.5	10.5.0	HIGH	Inyeccion de comandos en su linea de comandos
`flatted` (desarrollo)	3.3.3	3.4.2	HIGH	Contaminacion de prototipos al leer datos

3. Excepcion aceptada

Hallazgo	Por que no se corrige hoy	Riesgo real	Vence
CVE-2026-118664 en `decode-uri-component` 0.2.2 (via `expo-router` -> `query-string` 7)	La unica version parchada (0.5.1) es un modulo ESM que `query-string` 7 no puede cargar; forzarla rompe la navegacion de la app	Bajo: es una denegacion de servicio en el navegador o el celular de quien abre un enlace manipulado a la propia app; no expone el servidor ni datos	28/02/2027

La excepcion esta en `.trivyignore` con fecha de vencimiento: ese dia Trivy vuelve a fallar y el equipo revisa si `expo-router` ya trae una version compatible.

4. Como funciona

Archivo	Para que sirve
`trivy.yaml`	Configuracion: escaneres (`vuln`, `secret`, `misconfig`), tambien dependencias de desarrollo, severidades que bloquean (HIGH y CRITICAL), carpetas excluidas (`node_modules`, la web exportada y el template de UI original, que no se despliega)
`.trivyignore`	Excepciones justificadas, cada una con razon y fecha de vencimiento
`.github/workflows/seguridad.yml`	Instala Trivy, genera el reporte completo (todas las severidades) como artefacto y corre el escaneo que bloquea

Instalacion segura de Trivy en el CI. El workflow descarga el binario oficial de Trivy v0.69.3 y verifica su checksum SHA-256 antes de usarlo. No se usan las acciones `aquasecurity/trivy-action` ni `setup-trivy`: en marzo de 2026 sus etiquetas fueron comprometidas en un ataque a la cadena de suministro, y una herramienta de seguridad comprometida es justo lo que no se puede permitir. Fijar version y checksum evita ese riesgo.

Por que cada lunes. Cada semana se publican vulnerabilidades nuevas sobre paquetes que no cambiaron. El escaneo programado avisa aunque nadie haya tocado el codigo.

5. Como correrlo en local

```
# Instalar (Linux; en Mac: brew install trivy)
curl -sSfL -o trivy.tgz
https://github.com/aquasecurity/trivy/releases/download/v0.69.3/trivy_0.69.3_Linux-
64bit.tar.gz
echo "1816b632dfe529869c740c0913e36bd1629cb7688bd5634f4a858c1d57c88b75  trivy.tgz" |
sha256sum -c -
tar xzf trivy.tgz trivy

# Desde la raiz del repositorio
./trivy fs --config trivy.yaml .                                    # el mismo escaneo que
bloquea en CI
./trivy fs --config trivy.yaml --severity LOW,MEDIUM,HIGH,CRITICAL --exit-code 0 .   #
reporte completo
```

6. Que hacer cuando Trivy falla

1. Leer el reporte: paquete, version instalada y version con el parche.
2. Si es una dependencia directa, actualizarla (`npm update <paquete>` o subir la version en `package.json`).
3. Si es indirecta, agregar un `override` en `package.json` hacia la version parchada de la misma version mayor y correr las pruebas.
4. Si no hay parche compatible, documentar la excepcion en `.trivyignore` con la razon, el riesgo real y una fecha de vencimiento de no mas de 4 meses.
5. Un secreto encontrado nunca se ignora: se revoca la clave, se cambia y se saca del historial.

7. Relacion con la arquitectura

- Responde a DRV-03 (privacidad de los datos) y a la convencion de seguridad del proyecto: dependencias sin vulnerabilidades conocidas y ningun secreto en el codigo.
- Complementa los controles que ya prueba la piramide: cifrado AES-256-GCM, logs sin datos personales, limites de uso, JWT corto con refresco rotativo (ver *documento 18 (Piramide de pruebas)*).
- Decision registrada en ADR-018 (*documento 11 (Decisiones de arquitectura (ADR))*).

8. Historial de escaneos

Fecha	Que cambio	Resultado	Notas
05/11/2026	Primer escaneo completo, con dependencias de desarrollo	1 HIGH en el backend y 1 CRITICAL y 50 HIGH en el frontend; todos corregidos	Ver las secciones 1 y 2

Fecha	Que cambio	Resultado	Notas
12/11/2026	Se integra el pull request 53 (traduccion con Claude en Google Vertex AI, pantalla de inicio nueva, reporte de problemas y mejoras de voz). Dependencia nueva en el backend: `@anthropic-ai/vertex-sdk`, con sus dependencias de autenticacion de Google	**0 vulnerabilidades HIGH o CRITICAL, 0 secretos y 0 configuraciones inseguras** en `backend`, `frontend` y `e2e`, con la base de vulnerabilidades descargada el mismo dia	La excepcion de `decode-uri-component` sigue vigente hasta el 28/02/2027

Observaciones del reescaneo del 12/11/2026:

- La traduccion con Vertex se autentica con las credenciales de Google (ADC), no con una clave escrita en el codigo ni en el repositorio; el escaner de secretos no encontro ninguna.
- El script de arranque `run-talklate.sh` trae por defecto el identificador del proyecto de Google Cloud. Un identificador de proyecto no es un secreto, pero es mejor leerlo del entorno que dejarlo escrito.
- Trivy se mantiene fijado en la version 0.69.3 con su checksum. Ya existe una version mas nueva; actualizarla implica cambiar la version y el checksum en `seguridad.yml` y es una tarea pendiente.
- El escaneo cubre dependencias, secretos y configuraciones. **No** revisa riesgos propios del uso de modelos de lenguaje (para eso, ver `owasp_llm_top10.md`).

20. OWASP Top 10 para aplicaciones con LLM

Version: 1.1 (12/11/2026) **Referencia:** OWASP Top 10 for LLM Applications, edicion 2026 (publicada el 04/08/2026), repositorio https://github.com/GenAI-Security-Project/GenAI-LLM-Top10 (carpeta `2026/final/`).

Este documento revisa TalkLate contra los 10 riesgos de la lista. Es una revision honesta: marca lo que esta cubierto, lo que esta cubierto a medias y lo que falta. Los controles ya existian en el codigo (se construyeron por las convenciones de seguridad y la arquitectura hexagonal), pero hasta la revision del 11/11/2026 no estaban contrastados con esta lista. La version 1.1 recoge los ajustes hechos el 12/11/2026 y el cambio de motor de traduccion (ADR-020).

> *Nota: los nombres de los riesgos son los oficiales; las definiciones y estrategias de la tabla son un resumen propio del equipo.*

1. Resumen

TalkLate usa modelos de lenguaje en **un solo lugar**: traducir un mensaje. El motor principal es **Claude Sonnet servido por Google Vertex AI** (ADR-020); Qwen local queda como adaptador alternativo. El modelo recibe texto y devuelve texto en JSON. **No tiene herramientas, no toca la base de datos, no envia correos y no ve datos de otros usuarios.** Eso reduce mucho el riesgo, porque lo peor que puede pasar con un mensaje malicioso es que se dane la traduccion de la propia persona que lo escribio.

Lo que cambio con Vertex: el texto de cada mensaje **ahora sale del servidor** hacia Google Cloud (antes, con Qwen local, no salia). Eso pesa sobre LLM02 y LLM06, y esta reflejado en la politica de datos.

#	Riesgo (2026)	Estado	Resumen
LLM01	Prompt Injection	Parcial	El impacto es bajo, pero el texto del usuario entra al prompt sin marcarlo como dato y no hay pruebas de inyeccion
LLM02	Sensitive Information Disclosure	Parcial	Cifrado, logs limpios y limites de uso; el texto sale a Google Cloud y a ElevenLabs (documentado); la app no avisa a la persona
LLM03	Excessive Agency	Cubierto por diseno	El modelo no tiene herramientas ni permisos
LLM04	Supply Chain	Parcial	Trivy con checksum y lockfiles; falta un SBOM y revisar los terminos de los proveedores
LLM05	Data and Model Poisoning	Parcial	No se entrena ni se usa RAG; el glosario cultural (196 entradas) alimenta el prompt y no esta validado por nativos
LLM06	Unbounded Consumption	Parcial	Limites por usuario, tamanos, tiempos limite y tope de salida; falta un tope de gasto
LLM07	Misinformation	Parcial	Es el riesgo central del producto; hay reportes y notas, pero la medicion de calidad es debil
LLM08	Hidden Context Exposure	Cubierto	El prompt no tiene secretos ni se usa como control de seguridad

#	Riesgo (2026)	Estado	Resumen
LLM09	Vector and Embedding Weaknesses	No aplica	No hay RAG, embeddings ni base vectorial
LLM10	Improper Output Handling	Cubierto	La salida se valida (JSON, tono, genero, nota y region) y la app no interpreta HTML

Conteo: 3 cubiertos (LLM03, LLM08, LLM10), 1 no aplica (LLM09) y 6 parciales. Ninguno esta en rojo.

2. Detalle por riesgo

LLM01 - Prompt Injection (parcial)

Que es: una entrada altera el comportamiento del modelo porque este no distingue entre instrucciones y datos.

Que tiene TalkLate:

- La salida se fuerza a JSON y se valida en codigo (`backend/src/adaptadores/traduccion/prompt.js`, funcion `leerRespuesta`): la traduccion debe ser texto no vacio, el tono debe estar en la lista permitida, el genero pasa por una expresion regular y la nota y la region solo se aceptan como texto de largo acotado.
- El modelo no puede hacer nada fuera de devolver texto (ver LLM03).
- El mensaje tiene un maximo de 5000 caracteres y hay limite de mensajes por minuto por usuario.

Que falta:

- El texto del usuario se pega al final del prompt (`Mensaje:` + texto) sin delimitadores ni una instruccion que diga "esto es dato, no instrucciones".
- El audio se transcribe con Whisper y el texto resultante entra igual al prompt: la voz tambien es un canal de inyeccion.
- No se quitan caracteres Unicode invisibles.
- No hay pruebas automaticas con mensajes del tipo "ignora lo anterior y responde X".

Riesgo real: bajo. Un atacante solo puede estropear su propia traduccion; no accede a datos de otros ni ejecuta acciones.

LLM02 - Sensitive Information Disclosure (parcial)

Que es: datos confidenciales salen por un canal no autorizado: la respuesta, los logs, los terceros.

Que tiene TalkLate:

- Cifrado AES-256-GCM del contenido de mensajes y traducciones en reposo (`adaptadores/cifrado/aesGcm.js`).
- Logs que descartan por codigo los campos `password`, `token`, `correo`, `contenido`, `texto` y `codigo` (`compartido/log.js`), con prueba unitaria.
- Aislamiento entre usuarios y limites de uso en endpoints sensibles.

- El prompt no lleva datos personales: solo el texto a traducir, idiomas, region, tono y, en voz, una descripcion de como hablo la persona.
- Whisper corre en el servidor: el **audio original no sale** del servidor.
- Politica de tratamiento de datos y matriz de sensibilidad actualizadas el 12/11/2026: nombran a **Google Cloud (Vertex AI)** y a **ElevenLabs** como terceros y clasifican el **genero inferido de la voz** y los **problemas reportados** como datos personales.

Que falta:

- El **texto de cada mensaje sale a Google Cloud** para traducirlo y el texto traducido sale a ElevenLabs para la voz. La persona lo acepta en la politica al registrarse, pero la app no se lo recuerda mientras conversa.
- Falta revisar los terminos de los proveedores (retencion y uso de los datos para entrenar) y dejarlo escrito.
- El genero inferido de la voz se guarda por mensaje; hay que decidir si hace falta guardarlo o basta usarlo al momento.

LLM03 - Excessive Agency (cubierto por diseno)

Que es: el sistema ejecuta acciones danosas por salidas inesperadas del modelo, por exceso de herramientas, permisos o autonomia.

Que tiene TalkLate: el modelo no tiene herramientas, no llama funciones, no accede a la base de datos y no envia mensajes. El backend recibe el JSON, lo valida y decide todo lo demas con codigo determinista. La autorizacion (JWT, dueno de la conversacion, rol admin) vive en el backend, nunca en el modelo.

Reglas del equipo (quedan escritas aqui):

- Ningun motor de IA recibe herramientas ni acceso a datos. Si algun dia se agrega, se revisa este riesgo de nuevo antes de integrarlo.
- La cuenta de Google que usa Vertex debe tener **solo el permiso de invocar modelos** (rol de usuario de Vertex AI), no permisos de administracion del proyecto.

LLM04 - Supply Chain (parcial)

Que es: riesgos de integridad en paquetes, modelos, datos y plataformas de terceros.

Que tiene TalkLate:

- Escaneo con Trivy en cada pull request y cada lunes (`seguridad_trivy.md`). Reescaneo del 12/11/2026 con la dependencia nueva de Vertex: **0 vulnerabilidades altas o criticas y 0 secretos**.
- Binario de Trivy fijado con checksum SHA-256; no se usa `trivy-action` por el ataque de marzo de 2026.
- `package-lock.json` en backend, frontend y pruebas, e instalacion con `npm ci` en el CI.
- Los motores de ElevenLabs, Azure, OpenAI y Claude directo se llaman con `fetch` (sin SDK); la traduccion por Vertex usa el SDK oficial de Anthropic.
- La traduccion por Vertex se autentica con las credenciales de Google (ADC): no hay clave de API en el repositorio.

Que falta:

- No hay SBOM (lista de componentes) ni siquiera de las dependencias de npm.
- Los modelos se piden por nombre (`claude-sonnet-4-6` en Vertex, `claude-sonnet-5-5` en la API directa): los dos valores por defecto no coinciden y conviene fijar uno y comprobar que exista en la region configurada.
- Si se vuelve a usar Qwen, el modelo se descarga por etiqueta (`qwen2.5:7b`) y no por huella fija.
- Falta una nota de evaluacion de los terceros (Anthropic, Google Cloud, ElevenLabs): terminos, retencion y entrenamiento con datos.

LLM05 - Data and Model Poisoning (parcial)

Que es: manipulacion de los datos o del modelo para meter sesgos o comportamientos danosos.

Que tiene TalkLate: no se entrena ni se ajusta ningun modelo y no hay RAG. El unico dato que influye en la salida es el glosario cultural (`glosario.json`), que esta versionado en git, pasa por pull request y trae la `source` de cada entrada (196 de 196).

Que falta: las 196 entradas tienen `native_validated: false`: ningun hablante nativo las ha revisado. Desde el 12/11/2026 el prompt incluye de cada modismo detectado su equivalente **y un ejemplo de uso** (few-shot), asi que una entrada mala (un equivalente incorrecto o una de las 26 marcadas como sensibles) influye aun mas en la traduccion. El campo ya existe; falta usarlo.

LLM06 - Unbounded Consumption (parcial)

Que es: uso de recursos sin control: caida del servicio o costo desbordado (denial of wallet).

Que tiene TalkLate:

- Limites por usuario: 30 mensajes por minuto, 40 voces por minuto y 10 reportes de problemas por hora (todos configurables); por IP en registro, recuperacion y restablecimiento.
- Tamanos: 5000 caracteres por mensaje, 1000 por voz, 1 MB por peticion y 15 MB solo para audio.
- Tiempos limite en todas las llamadas externas (Claude por Vertex 30 s y sin reintentos del SDK, Whisper 30 s, ElevenLabs 20 s, ffmpeg 15 s) y tope de salida: `max_tokens` de 1024 en Claude y `num_predict` de 1024 en Qwen.
- Circuit Breaker: un motor que falla se salta por 60 segundos.
- Los audios sintetizados viven 5 minutos y como maximo 200 en memoria.

Que falta:

- Los limites estan **en memoria**: se pierden al reiniciar y no funcionan con varias instancias.
- No hay un **tope de gasto** diario para Claude (Vertex), ElevenLabs o Whisper en la nube; los limites cuentan peticiones, no costo. Con Vertex y ElevenLabs cada mensaje cuesta, asi que esta brecha pesa mas que antes.

LLM07 - Misinformation (parcial)

Que es: salidas incorrectas que parecen creibles. En TalkLate significa **una traduccion equivocada que suena bien**; es el riesgo central del producto.

Que tiene TalkLate:

- Reporte de traducciones malas por parte del usuario, con atencion del administrador.
- Nota cultural que explica cada modismo adaptado.
- El motor de reglas **ya no entrega una traduccion palabra por palabra**: si el texto no trae modismos conocidos, falla con un error claro (cambio del 12/11/2026).
- Aviso "Traduccion aproximada" cuando traduce el motor de reglas con palabras sin traducir.
- Prompt de **transcreacion cultural** con ejemplos reales del glosario.
- Salida estructurada con campos obligatorios.
- Medicion de modismos adaptados (DRV-02): 14 de 15 con el motor de reglas.

Que falta:

- Esa medicion se hizo **con el motor de reglas, que se construyo con los mismos 15 modismos**: es una linea base, no una medida de calidad. Falta medirla con el motor de produccion (Claude en Vertex) y con una lista de modismos distinta.
- No hay evaluacion de calidad con hablantes nativos (ver LLM05).
- La app no dice de forma general que la traduccion automatica puede equivocarse.

LLM08 - Hidden Context Exposure (cubierto)

Que es: extraccion de las instrucciones ocultas del modelo (system prompt) cuando eso ayuda a un atacante. La guia pide asumir que ese contexto es descubrible.

Que tiene TalkLate:

- El prompt no tiene claves, cadenas de conexion ni datos de usuarios. Las claves viven en variables de entorno y `.env` esta en `.gitignore`; Trivy busca secretos en cada pull request.
- El prompt no es un control de seguridad: la autorizacion se hace en codigo, fuera del modelo.
- El prompt ya es publico (esta en el repositorio, `prompt.js`), asi que filtrarlo no da ventaja.

LLM09 - Vector and Embedding Weaknesses (no aplica)

TalkLate no usa RAG, embeddings, memoria de agentes ni cache semantica. SurrealDB soporta vectores, pero no se usan. Si se agrega busqueda semantica o RAG, este riesgo se revisa antes.

LLM10 - Improper Output Handling (cubierto)

Que es: la salida del modelo se pasa a otro componente sin validarla (XSS, inyeccion, etc.).

Que tiene TalkLate:

- La salida del modelo se trata como entrada no confiable: JSON validado, tono contra lista, genero normalizado y, desde el 12/11/2026, `nota_cultural` (500 caracteres) y `region_origen` (80) solo como texto de largo acotado, con prueba unitaria.
- La app es React Native: muestra el texto como texto, no como HTML ni Markdown, asi que no hay XSS.

- Consultas parametrizadas en SQLite y literales serializados en SurrealDB (nada de texto concatenado).
- El texto que va a Azure se escapa antes de armar el SSML.
- Los logs no guardan el contenido.

3. Brechas y plan para cerrarlas

Prioridad	Brecha	Riesgo	Accion	Estado
1	`nota_cultural` y `region_origen` sin tipo ni largo	LLM10	Convertir a texto y acotar en `leerRespuesta`	**Hecho 12/11/2026**
2	Qwen sin limite de salida; Vertex sin tiempo limite	LLM06	`num_predict` en Qwen; 30 s y sin reintentos en Vertex	**Hecho 12/11/2026**
3	Reporte de problemas sin limite	LLM06	10 por hora por usuario	**Hecho 12/11/2026**
4	Terceros no nombrados en la politica de datos	LLM02	Actualizar la politica y la matriz (Google Cloud, ElevenLabs, genero de la voz)	**Hecho 12/11/2026**
5	El texto del usuario entra al prompt sin delimitar	LLM01	Encerrar el mensaje entre marcas y decir al modelo que es dato; quitar Unicode invisible; pruebas de inyeccion	Pendiente (toca el prompt de transcreacion: probar con el modelo real)
6	Sin SBOM; modelos sin fijar	LLM04	`trivy sbom` en el CI; unificar el modelo por defecto	Pendiente
7	Glosario sin validacion nativa	LLM05, LLM07	Que un hablante paisa y uno de New York revisen las 196 entradas y se marquen con `native_validated: true`	Pendiente (depende de personas)
8	Medicion de calidad solo con el motor de reglas	LLM07	Correr `npm run medir` con Claude (Vertex) y con modismos que no esten en el motor de reglas	Pendiente
9	Sin tope de gasto y limites en memoria	LLM06	Tope diario de gasto y limites en una base compartida	Pendiente (fuera del MVP de una instancia)
10	La app no recuerda que el texto sale a terceros	LLM02	Aviso corto en la pantalla de chat	Pendiente

4. Hallazgo adicional (fuera de la lista OWASP)

Credenciales de administrador por defecto. Al arrancar, el backend crea un administrador con correo `dafa123.daf@gmail.com` y contrasena `Admin123!` si no se definen `ADMIN_CORREO` y `ADMIN_PASSWORD`. **Desde el 12/11/2026, con `NODE_ENV=production` el servidor se niega a crear el administrador sin `ADMIN_PASSWORD`** (con prueba unitaria). En desarrollo se conserva la contrasena conocida, documentada como solo para pruebas. Queda pendiente sacar el correo personal del codigo.

5. Como se relaciona con lo demas

- Complementa a Trivy (`seguridad_trivy.md`): Trivy cubre el eslabon de dependencias y secretos; este documento cubre los riesgos propios de usar un modelo de lenguaje.
- Los controles probados estan en la piramide de pruebas (`piramide_de_pruebas.md`): cifrado, logs sin datos personales, limites de uso y validaciones.
- Responde al driver de privacidad (DRV-03), a RNF-011 y a las convenciones de seguridad (`convenciones/seguridad.md`).